

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
УРАЛЬСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ ИМЕНИ ПЕРВОГО ПРЕЗИДЕНТА
РОССИИ Б.Н. ЕЛЫЦИНА
(УрФУ)

Физико-технологический институт

Кафедра технической физики

ДОПУСТИТЬ К ЗАЩИТЕ В ГЭК

Зав. кафедрой ТФ

(подпись) В.И. Токманцев

« ____ » _____ 2026 г.

**РАЗВИТИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ
ПОДДЕРЖКИ СОВРЕМЕННОГО ИНТЕРНЕТ-МАРКЕТИНГА
НА ОСНОВЕ АВТОМАТИЧЕСКОГО РАСПОЗНАВАНИЯ
ОБЛАСТЕЙ ИНТЕРЕСА НА РЕКЛАМНЫХ
ИЗОБРАЖЕНИЯХ В ВИДЕОПОТОКЕ**

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Пояснительная записка

09.03.02 63.11.11 114 ПЗ

Руководитель _____ И.В. Кашин
Доцент, к. ф.-м. н.

Нормоконтролер _____ И.Э. Новоселов
Ассистент

Студент гр ФТ-420008 _____ М.А. Ветров

Екатеринбург
2026

РЕФЕРАТ

Пояснительная записка: 142 страницы, 47 рисунков, 14 таблиц, 54 источника, 3 приложения.

В работе рассматривается актуальная задача автоматизации анализа внимания пользователей к рекламным изображениям с использованием айтрекинга и машинного обучения. Традиционные методы оценки эффективности рекламы, такие как опросы и фокус-группы, требуют значительных ресурсов и не всегда объективны. Автоматизация этого процесса позволяет быстро и точно выявлять области интереса, оптимизируя размещение ключевых элементов и повышая вовлеченность аудитории.

Целью работы является разработка информационной системы, способной автоматически распознавать и анализировать области интереса на рекламных изображениях. Для этого используются технологии айтрекинга для сбора данных о движении взгляда пользователей и алгоритмы машинного обучения, для предсказания областей интереса на изображениях.

В результате была создана система, автоматически выявляющая области интереса и визуализирующая их с помощью тепловых карт. Обученные алгоритмы показали высокую точность предсказания, что подтверждает эффективность предложенного подхода. Новизна работы заключается в применении агентного подхода для накопления контекста с предыдущих кадров видео и автоматизации анализа внимания в маркетинге в потоковом режиме с помощью машинного обучения.

Результаты могут применяться в цифровом маркетинге, веб-дизайне и UX-анализе, позволяя компаниям оперативно адаптировать рекламные материалы под предпочтения аудитории. Значимость работы состоит в сокращении времени и ресурсов на тестирование рекламы, а также в создании основы для дальнейшего развития автоматизированных маркетинговых систем.

СОДЕРЖАНИЕ

НОРМАТИВНЫЕ ССЫЛКИ	6
ОПРЕДЕЛЕНИЯ.....	7
УСЛОВНЫЕ ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	9
ВВЕДЕНИЕ.....	10
1 ПРОБЛЕМАТИКА АВТОМАТИЗАЦИИ АНАЛИЗА ВНИМАНИЯ ПОЛЬЗОВАТЕЛЕЙ К РЕКЛАМНЫМ ИЗОБРАЖЕНИЯМ С ИСПОЛЬЗОВАНИЕМ ТЕХНОЛОГИЙ АЙТРЕКИНГА И МАШИННОГО ОБУЧЕНИЯ	12
1.1 Ключевые термины.....	12
1.2 Технология поиска информации	12
1.3 Литературно–аналитический обзор	17
1.4 Группы аналогов	18
1.4.1 Группа аналогов: системы программного айтрекинга.....	20
1.4.2 Группа аналогов: агентные методы восстановления изображения.	21
1.4.3 Группа аналогов: алгоритмы машинного обучения для распознавания областей интереса.....	23
1.5 Оценка групп аналогов.....	26
1.5.1 Выбор критериев оценки для группы аналогов: системы программного айтрекинга.....	27
1.5.2 Выбор критериев оценки для группы аналогов: агентных методов восстановления изображения	29
1.5.3 Выбор критериев оценки для группы аналогов: алгоритмы машинного обучения	31
1.5.4 Определение весовых коэффициентов критериев оценки	32

1.5.5	Сравнение аналогов в рамках каждой из групп	35
1.5.6	Выбор прототипа для каждой из групп аналогов, критика	37
1.6	Гипотеза о предлагаемом решении	38
1.7	Цели и задачи исследования	39
1.8	Результаты и выводы по первой главе	40
2	МОДЕЛИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ АНАЛИЗА ВНИМАНИЯ НА ОСНОВЕ АЙТРЕКИНГА И МАШИННОГО ОБУЧЕНИЯ	41
2.1	Концептуальная модель	41
2.2	Структурная модель	45
2.3	Алгоритмическая модель	49
2.4	Функционально–структурная модель	52
2.5	Иерархическая модель	54
2.6	Математическая модель	55
2.6.1	Определения и обозначения	56
2.6.2	Предобработка	57
2.6.3	Обновление динамического распределения	58
2.6.4	Метрики оценки качества	59
2.7	Результаты и выводы по второй главе	63
3	ПРОЕКТИРОВАНИЕ	64
3.1	Внешнее проектирование	64
3.2	Внутреннее проектирование	64
3.2.1	Выбор программной платформы и среды разработки	68
3.3	Результаты и выводы по третьей главе	69
4	ИНЖЕНЕРНАЯ РЕАЛИЗАЦИЯ	70

4.1 Сбор набора данных	70
4.2 Обучение модели	75
4.3 Агентный подход	78
4.4 Интерфейс	90
ЗАКЛЮЧЕНИЕ	95
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	97
ПРИЛОЖЕНИЕ А. Техническое задание	102
ПРИЛОЖЕНИЕ Б. Код библиотеки agent_sim	113
ПРИЛОЖЕНИЕ В. Код основной программы	120

НОРМАТИВНЫЕ ССЫЛКИ

В пояснительной записке использованы ссылки на следующие стандарты:

ГОСТ 19.701-90 ЕСПД	Схема алгоритмов, программ, данных и систем.
ГОСТ 34.602-89 ИТ. ЕКСАС	Виды, комплектность и обозначение документов при создании автоматизированных систем.
ГОСТ 34.602-2020	Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы.
ГОСТ 34.603-92 ит. ЕКСАС	Виды испытаний автоматизированных систем.
ГОСТ Р ИСО/МЭК 9126-93 ИТ	Оценка программной продукции. Характеристика качества и руководства по их применению.
ГОСТ Р ИСО/МЭК ТО 15910-2002 ИТ	Процесс создания документации пользователя программного средства.
ГОСТ Р ИСО/МЭК 12207-99 ИТ	Процессы жизненного цикла программных средств.
Р 50.1.028-2001 ИТПЖЦП	Рекомендация по стандартизации. Методология функционального моделирования.

ОПРЕДЕЛЕНИЯ

Айтрекинг

метод и процесс регистрации движений глаз человека для определения направления взгляда и анализа визуального внимания. Айтрекинг позволяет фиксировать, какие участки изображения привлекают наибольший интерес, и используется для оптимизации дизайна, улучшения пользовательского опыта и повышения эффективности рекламных материалов.

Информационная система поддержки маркетинга

программно-аппаратный комплекс, предназначенный для автоматизации анализа внимания пользователей к рекламным материалам. Система интегрирует технологии айтрекинга, машинного обучения и визуализации данных для оптимизации маркетинговых стратегий.

Машинное обучение в анализе внимания

набор методов и алгоритмов, используемых для автоматического распознавания и предсказания областей интереса на изображениях на основе данных айтрекинга. Эти алгоритмы обучаются на наборах данных, содержащих информацию о движении взгляда, и применяются для оптимизации рекламных кампаний.

Область интереса

участок рекламного изображения, который привлекает повышенное внимание пользователей, определяемое на основе

данных айтрекинга. Эти области выявляются с помощью анализа траектории взгляда и визуализируются в виде тепловых карт для дальнейшей оптимизации контента.

Пользовательский опыт

совокупность впечатлений и реакций пользователя при взаимодействии с продуктом или системой. Айтрекинг и анализ внимания используются для улучшения UX за счет оптимизации визуальных элементов интерфейса и рекламных материалов.

Тепловая карта

визуальное представление распределения внимания пользователей на изображении, где интенсивность цвета соответствует степени интереса к определенным участкам. Тепловые карты создаются на основе данных айтрекинга и используются для анализа эффективности дизайна и размещения ключевых элементов.

УСЛОВНЫЕ ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

BCE	–	(англ. Binary Cross-Entropy) – Бинарная перекрёстная энтропия
ET	–	(англ. Eye Tracking) – Отслеживание взгляда
MSE	–	(англ. Mean Squared Error) – Среднеквадратичная ошибка
OpenCV	–	(англ. Open Source Computer Vision Library) – Библиотека компьютерного зрения с открытым исходным кодом
UX	–	(англ. User Experience) – Пользовательский опыт
ИС	–	Информационная система
КОИ	–	Карта областей интереса
КТВ	–	Карта тепловой визуализации
МО	–	Машинное обучение
ПО	–	Программное обеспечение

ВВЕДЕНИЕ

В современных условиях цифровой экономики эффективность рекламных кампаний [1] напрямую зависит от способности быстро и точно анализировать реакцию целевой аудитории на визуальные материалы. Традиционные методы оценки внимания пользователей [2], такие как фокус-группы, анкетирование или экспертные оценки, требуют значительных временных и финансовых затрат. В последние годы активно развиваются технологии автоматизированного анализа внимания, основанные на айтрекинге [3] и машинном обучении [4]. Эти методы позволяют собирать данные о движении взгляда пользователей в реальном времени, строить тепловые карты интереса [5] и предсказывать наиболее привлекательные зоны на рекламных изображениях. Однако существующие решения часто ограничены высокой стоимостью оборудования, сложностью интеграции в маркетинговые процессы или недостаточной точностью анализа [6].

Актуальность данной работы обусловлена необходимостью создания доступной и эффективной информационной системы [7], способной автоматически выявлять области интереса [5] на рекламных изображениях в видеопотоке. Новизна исследования заключается в замене технологии айтрекинга алгоритмом машинного обучения [4] и агентным подходом [8] к восстановлению изображения с накоплением контекста между кадрами, что ранее не применялось в подобных информационных системах.

Объектом исследования является процесс анализа внимания пользователей к рекламным изображениям [6] с использованием технологий айтрекинга [3], машинного обучения [4] и агентного подхода [8]. Предметом исследования выступают методы автоматического распознавания областей интереса на визуальных материалах [6], а также их интеграция в информационную систему [9, 10] поддержки маркетинговых решений.

Целью работы является разработка информационной системы [9, 10], способной автоматически выявлять и анализировать области интереса [6] на

рекламных изображениях в видеопотоке. Для достижения этой цели были поставлены следующие задачи:

1. Провести анализ современных методов айтрекинга [3] и машинного обучения [4] для распознавания внимания пользователей;
2. Собрать данные о движении взгляда с использованием веб-камеры ноутбука [3];
3. Реализовать методы визуализации областей интереса в виде тепловых карт [5];
4. Реализовать агентный подход [8] для накопления контекста с предыдущих кадров видео;
5. Обучить модель машинного обучения для предсказания областей интереса на новых изображениях [4, 6];
6. Интегрировать разработанные алгоритмы в информационную систему [9, 10] и оценить её эффективность.

Патентные исследования [11] и обзор аналогов [12] показали, что существующие решения ориентированы на профессиональное оборудование и предсказывают внимание [5] по одному кадру, ориентируясь на цвет пикселей. В предлагаемом методе, напротив, применяется признаковое представление кадра [13] и агентный подход [9] для формирования контекста, что в совокупности повышает качество предсказания областей интереса [5] на рекламных изображениях. Метрологическое обеспечение работы включает использование стандартных метрик точности машинного обучения (MSE, BCE) [14, 15] и валидацию результатов на тестовых выборках рекламных изображений. Планируемый уровень разработки предполагает создание прототипа информационной системы [9, 10], способного работать на стандартном оборудовании, а полученные результаты должны легко визуально интерпретироваться [5] для последующей оптимизации рекламных материалов. Результаты работы могут быть использованы для оптимизации рекламных кампаний, улучшения пользовательского опыта и повышения эффективности визуальных коммуникаций.

1 ПРОБЛЕМАТИКА АВТОМАТИЗАЦИИ АНАЛИЗА ВНИМАНИЯ ПОЛЬЗОВАТЕЛЕЙ К РЕКЛАМНЫМ ИЗОБРАЖЕНИЯМ С ИСПОЛЬЗОВАНИЕМ ТЕХНОЛОГИЙ АЙТРЕКИНГА И МАШИННОГО ОБУЧЕНИЯ

В главе представлены основные термины и понятия по теме работы, предложена технология поиска информации [7], проведен поиск и анализ найденной информации, выполнен отбор аналогов, создан компилятивный прототип, выдвинута гипотеза о предлагаемом решении сформулированы цель и задачи исследования.

1.1 Ключевые термины

Ключевые слова для поиска: айтрекинг, автоматизация анализа внимания области интереса на изображениях, тепловые карты внимания, машинное обучение, информационная система поддержки маркетинга, предсказание областей интереса, агентный метод.

1.2 Технология поиска информации

Поиск информации для выпускной квалификационной работы осуществлялся по ключевым словам, приведённым в разделе 1.1, алгоритм поиска информации [7, 10] представлен на рисунке 1. В поиске использовались следующие источники информации:

1. бумажные носители: документы, книги (см. рисунок 2);
2. электронные носители: интернет, содержимое баз знаний, баз данных (см. рисунок 3);
3. естественный интеллект: знание экспертов, которые не были зафиксированы во внешних носителях (см. рисунок 4).

Поиск информации в сети Интернет осуществлялся с помощью следующих ресурсов:

1. Яндекс – поисковая система;
2. МПК – международная патентная классификация;

3. GitHub – платформа для размещения исходного кода приложений.

Отбор информации производился по следующим критериям:

1. по дате;
2. по релевантности – информация должна удовлетворять моей работе.

Отбор полезной информации производился по информационным ресурсам за последние 5 лет, так как данные должны быть актуальными на сегодняшний день.

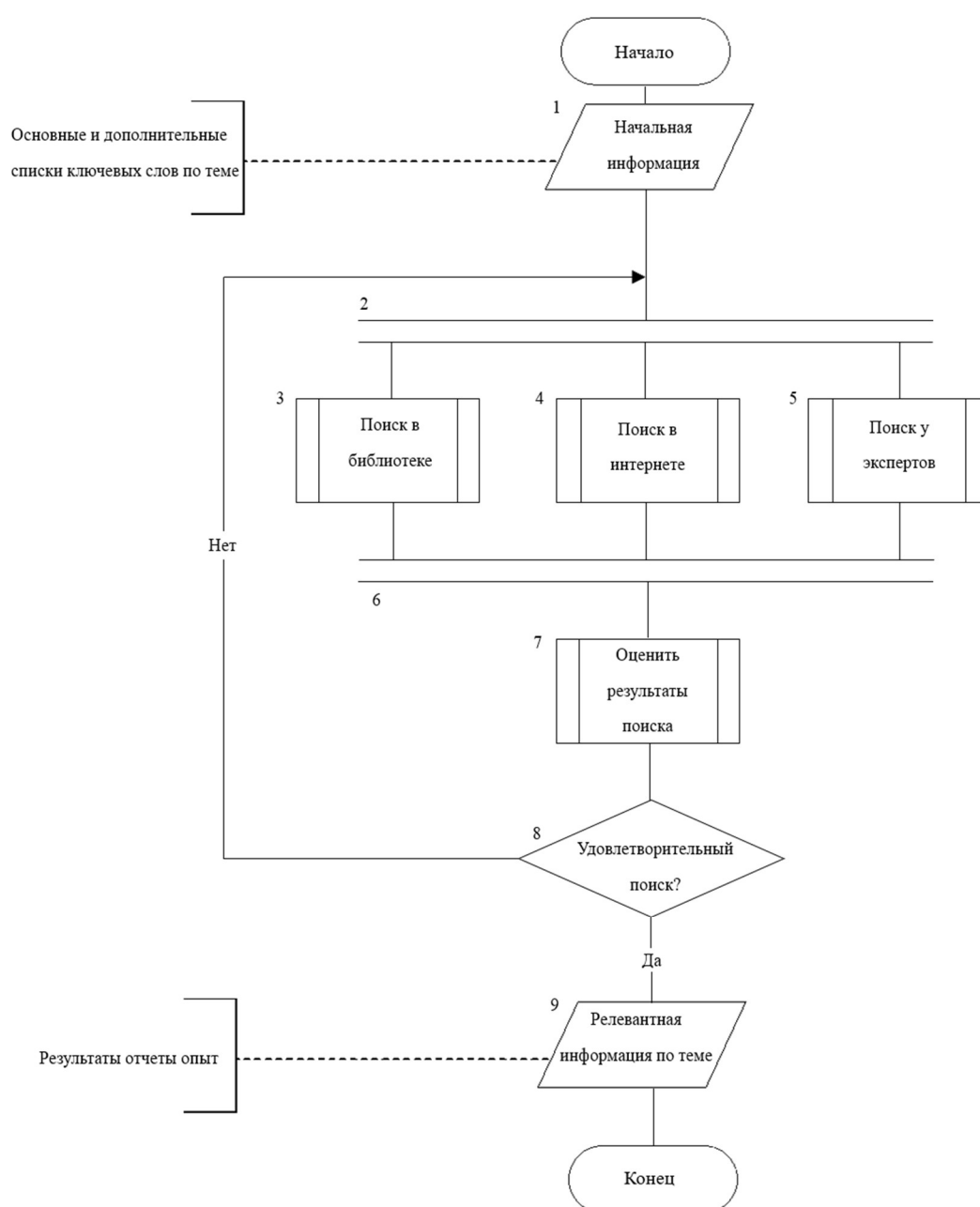


Рисунок 1 – Алгоритм поиска информации

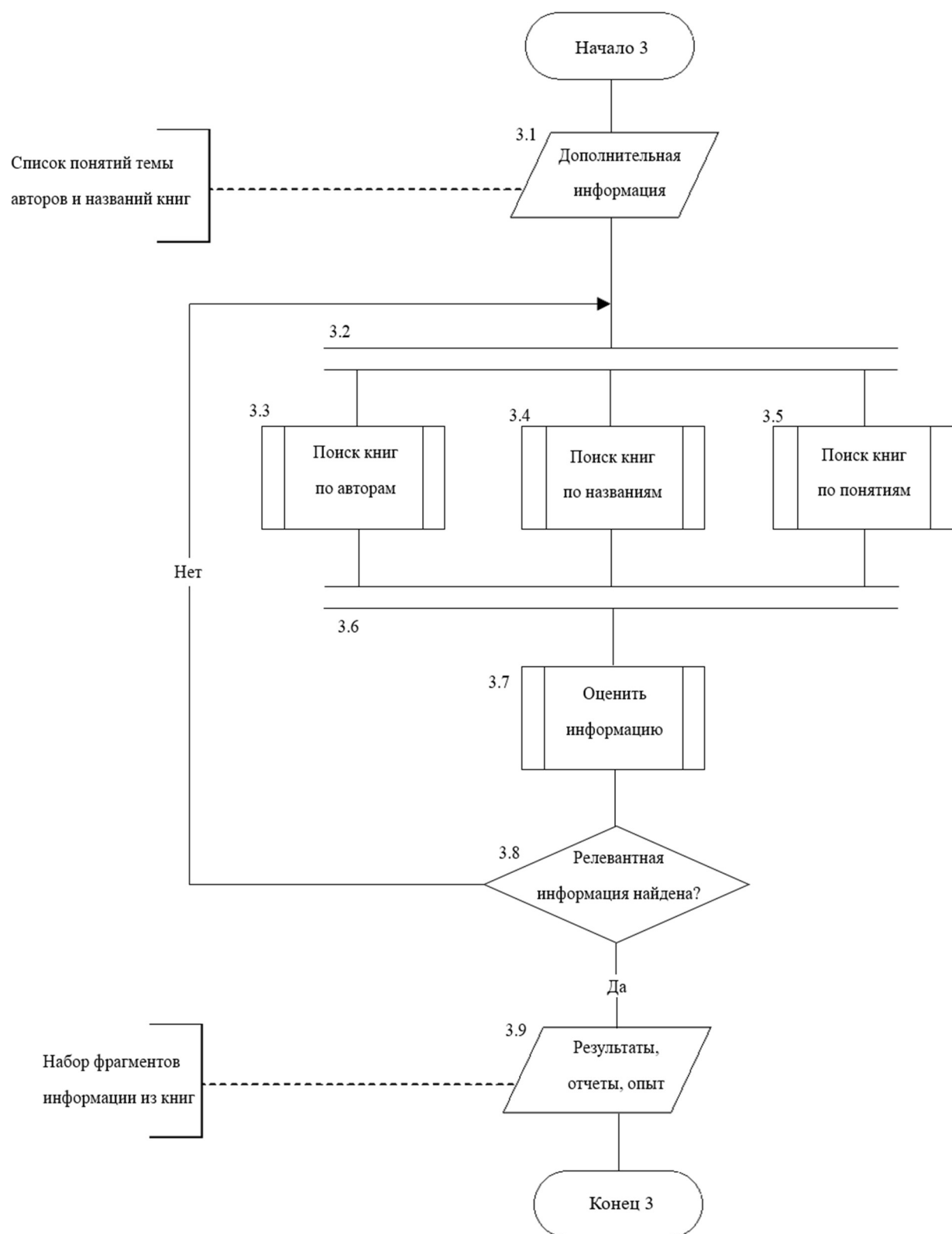


Рисунок 2 – Алгоритм поиска информации в библиотеке

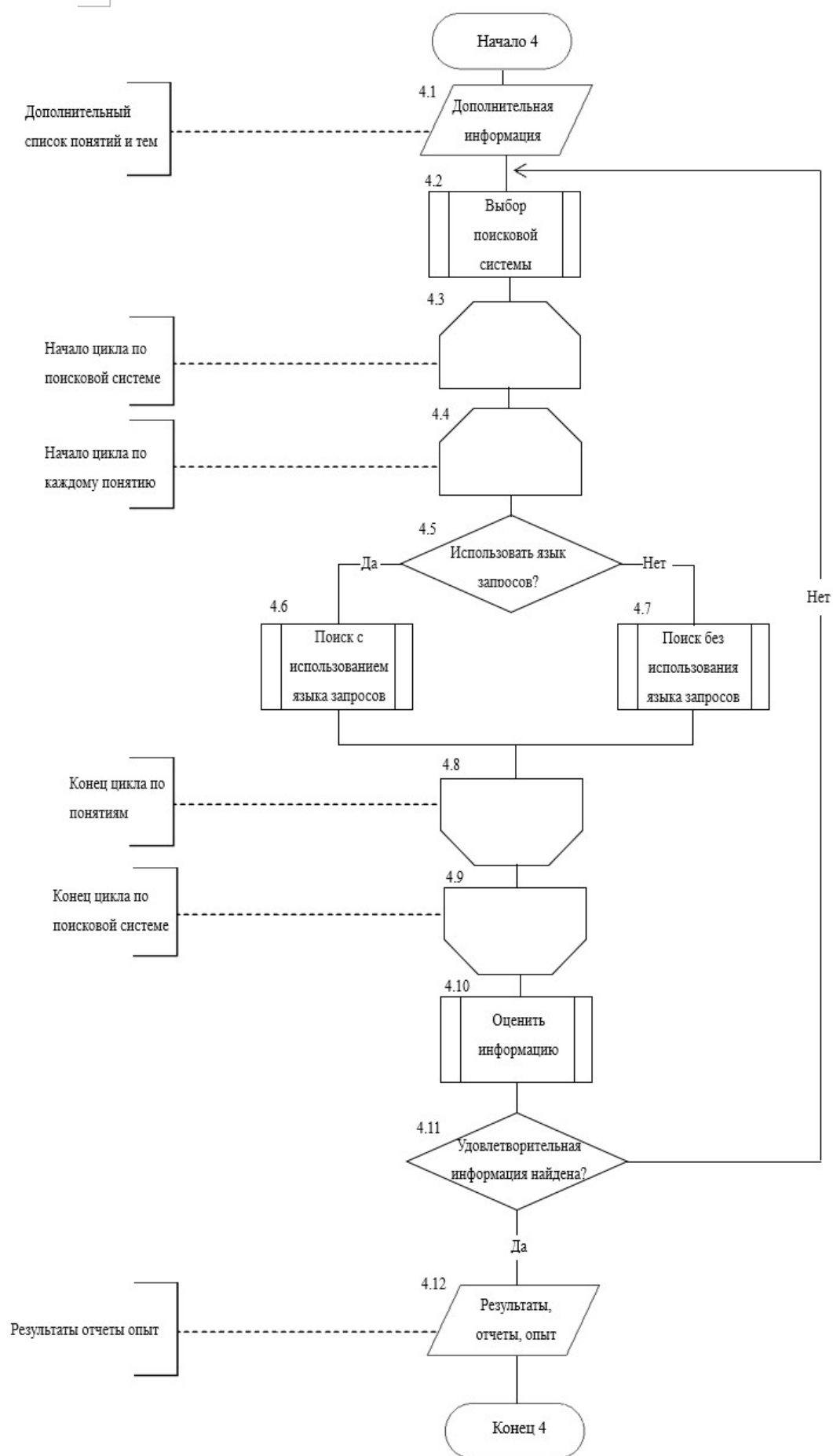


Рисунок 3 – Алгоритм поиска информации в Интернете

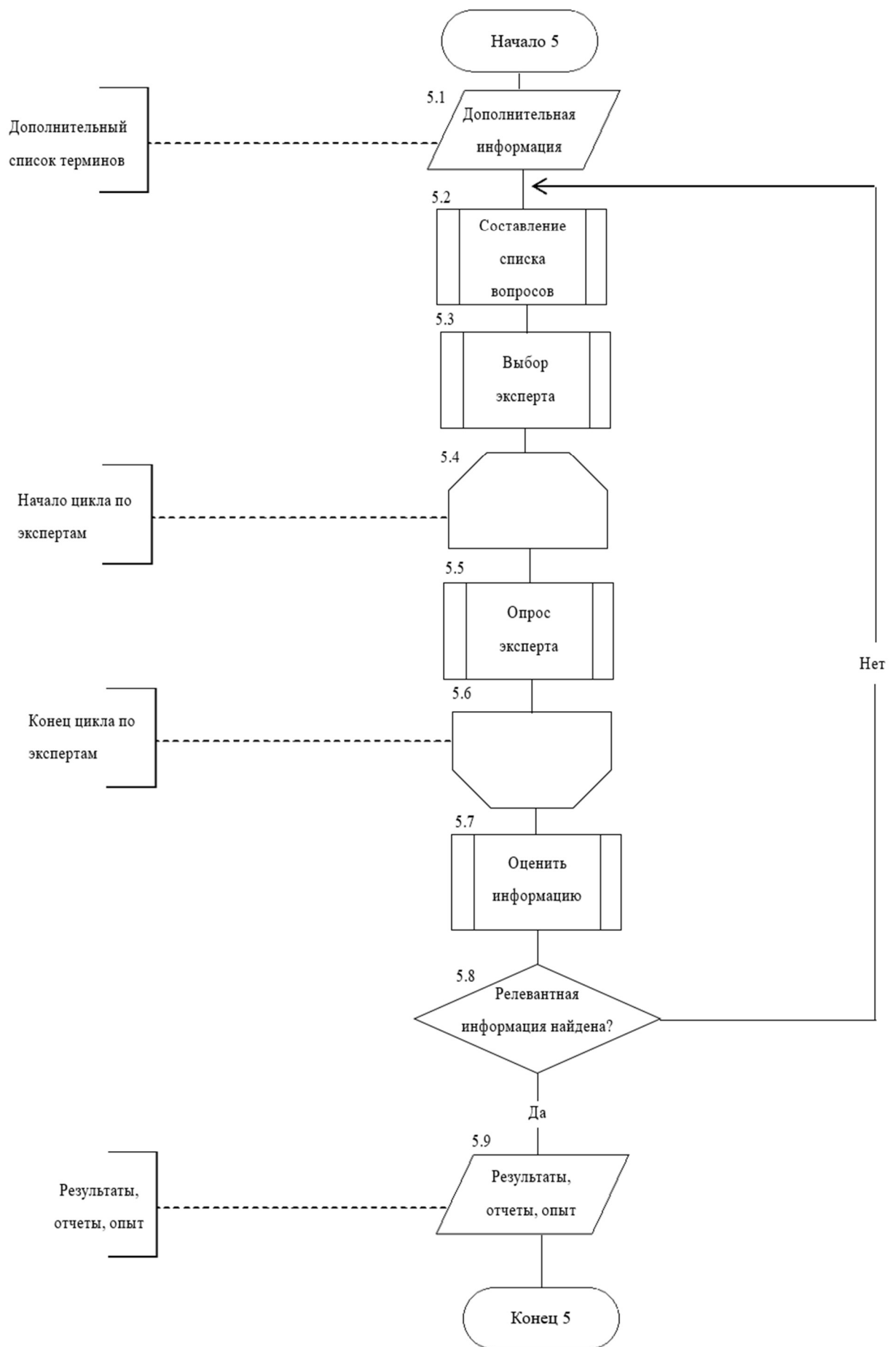


Рисунок 4 – Алгоритм поиска информации у экспертов

1.3 Литературно–аналитический обзор

Необходимость проведения литературно-аналитического обзора [7] в области анализа внимания пользователей к рекламным изображениям [2] обусловлена наличием ряда актуальных проблем, связанных с ограничениями существующих методов и технологий. Традиционные подходы к оценке эффективности рекламных материалов [2] зачастую основаны на субъективных методах, таких как опросы, фокус-группы или экспертные оценки. Эти методы требуют значительных временных и финансовых затрат, а их результаты не всегда объективно отражают реальное поведение целевой аудитории. Кроме того, традиционные методы не позволяют оперативно адаптировать рекламные материалы под изменяющиеся предпочтения пользователей, что снижает их эффективность в условиях динамичного цифрового рынка.

Одной из ключевых проблем является отсутствие доступных и точных инструментов для автоматизированного анализа внимания пользователей. Хотя технологии айтрекинга уже применяются в маркетинге, их использование часто ограничено высокой стоимостью оборудования и сложностью интеграции в существующие бизнес-процессы. Профессиональные айтрекеры [1, 3] обеспечивают высокую точность, но требуют значительных инвестиций и технической экспертизы. В то же время программные решения, основанные на стандартных веб-камерах [3], хотя и более доступны, зачастую не обеспечивают достаточной точности для надежного анализа.

Другой важной проблемой является отсутствие эффективных методов обработки и интерпретации данных [1], полученных с помощью айтрекинга [3]. Даже если удастся собрать данные о движении взгляда пользователей, их анализ часто остается ручным и трудоемким процессом. Это ограничивает возможности быстрой оптимизации рекламных материалов и снижает операционную эффективность маркетинговых кампаний. Кроме того, существующие алгоритмы машинного обучения [4], применяемые для анализа

внимания [1], не всегда адаптированы для работы с данными, полученными с помощью программного айтрекинга, что снижает их точность и надежность.

Еще одной значимой проблемой является отсутствие унифицированных подходов к визуализации и интерпретации результатов айтрекинга [3]. Тепловые карты [5], которые часто используются для отображения распределения внимания, зачастую строятся без учета специфики рекламных изображений и контекста их использования. Это может приводить к некорректной интерпретации данных и неверным выводам при оптимизации рекламных материалов [1].

Наконец, важной задачей является интеграция агентного подхода [8] предобработки кадров и методов машинного обучения [4] в единую информационную систему, которая могла бы автоматически анализировать и визуализировать данные о внимании пользователей [6]. Айтрекинг [3] при этом используется только для сбора обучающих данных. Такая система должна быть гибкой, масштабируемой и легко интегрируемой с существующими маркетинговыми платформами, что позволит компаниям оперативно адаптировать свои рекламные стратегии на основе объективных данных.

1.4 Группы аналогов

Задача автоматического распознавания областей интереса [6] на рекламных изображениях является комплексной и требует поэтапного решения сопутствующих проблем, описанных в разделе 1.2. Для этого необходим компилятивный прототип, объединяющий решения из смежных областей. В данном разделе рассматриваются группы аналогов, близких к решению выделенных проблем.

К таким проблемам относятся:

- Высокая стоимость и сложность интеграции профессиональных аппаратных систем айтрекинга в типовые бизнес-процессы информационного маркетинга [3];

- Низкая точность или ограниченная функциональность доступных программных решений на базе веб-камер;
- Отсутствие контекста между кадрами в видео;
- Отсутствие эффективных и адаптированных алгоритмов машинного обучения, способных на основе собранных данных предсказывать области интереса на новых рекламных изображениях без проведения повторных айтрекинг-исследований [4, 5];
- Разрозненность инструментов (сбор данных, визуализация, прогнозирование), что препятствует созданию целостной информационной системы поддержки принятия решений [9, 10].

В качестве первой группы аналогов будут рассмотрены библиотеки и технологии программного айтрекинга [3] на основе стандартной веб-камеры. Выбор конкретного инструмента определит точность и надёжность первичных данных (координат взгляда), что является фундаментом для всей последующей обработки и обучения моделей. В рамках обзора будут оценены библиотеки eyeGestures [16], OpenCV [17] в связке с готовыми моделями для детектирования лиц и dlib [18].

В рамках второй группы аналогов будут рассмотрены три агентных подхода к восстановлению изображения: Diffusion Agent [19], Ant Colony Optimization (ACO) [20] и агентный подход И.В. Кашина [8]. В отличие от классических методов визуализации данных внимания, данные алгоритмы демонстрируют механизмы накопления контекста и итеративного улучшения. Критерием отбора будет наличие явного агентного механизма, способность работать с нулевой начальной информацией и наличие накапливаемой памяти о предыдущих итерациях.

Третьей группой аналогов будут алгоритмы машинного обучения [4] для предсказания карт внимания и выделения областей интереса на изображениях [5]. Будут рассмотрены сверточная архитектура U-Net [21], свёрточная нейронная сеть CNN [22], а также архитектура на основе трансформеров Vision Transformer [23]. Отбор будет проводиться по

критериям точности предсказания, скорости обучения, скорости вывода, интерпретируемости и требовательности к вычислительным ресурсам.

1.4.1 Группа аналогов: системы программного айтрекинга

В данном разделе будет проведён анализ доступных библиотек и фреймворков для реализации айтрекинга [3] с использованием веб-камеры ноутбука с целью выбора оптимального инструмента для сбора первичных данных о координатах взгляда пользователя, что обеспечит точность входных данных для построения тепловых карт и обучения моделей машинного обучения [4]; процесс включает детекцию лица, локализацию ключевых точек глаз, калибровку и трекинг.

Все рассмотренные методы айтрекинга можно разделить на три подхода, основанных на способе детекции и трекинга взгляда:

1. Готовые программные решения предоставляют полный пайплайн айтрекинга с калибровкой и трекингом. Примером служит библиотека eyeGestures [16]. Такие решения удобны для быстрого прототипирования, но могут уступать в точности и гибкости. Такие решения часто ограничены фиксированными алгоритмами и не всегда поддерживают нестандартные сценарии использования.

2. Библиотеки компьютерного зрения требуют самостоятельной разработки алгоритмов детекции и трекинга взгляда. Примером является OpenCV [17]. Это даёт гибкость в настройке, но требует значительных временных затрат на разработку. Однако они позволяют точно настраивать алгоритмы под специфические задачи и аппаратные ограничения.

3. Специализированные библиотеки фокусируются на точной локализации ключевых точек лица и глаз. Примером служит библиотека dlib [18]. Такие решения обеспечивают высокую точность детекции, но требуют интеграции с дополнительными модулями для трекинга взгляда. Их основное преимущество — высокая точность детекции лицевых ориентиров, что критично для задач, требующих прецизионного анализа.

В ходе литературно-аналитического [7, 10] обзора были выявлены и рассмотрены следующие аналоги, представленные в Таблице 1.

Таблица 1 – Аналоги систем программного айтрекинга

№	Название библиотеки	Описание
1	eyeGestures	Исследовательское решение на Python [16, 24], разработанное для трекинга взгляда с использованием методов машинного обучения [4] и компьютерного зрения [25]. Предоставляет локальную библиотеку для калибровки по 9 точкам и получения потока координат (x, y) взгляда на экране в реальном времени с использованием стандартной веб-камеры.
2	OpenCV	Промышленный стандарт в компьютерном зрении [25]. Содержит предобученные классификаторы для детекции лица и глаз, а также алгоритмы отслеживания объектов [17]. Требуется самостоятельной разработки полного пайплайна для оценки направления взгляда и его калибровки относительно экрана.
3	dlib	Высокоточная библиотека для детекции 68 лицевых ориентиров, включая контуры глаз и рта [18]. Может быть использована в связке с алгоритмами оценки геометрии взгляда (например, методом углов вектора между зрачками и уголками глаз). Требуется глубокой доработки для создания end-to-end решения.

1.4.2 Группа аналогов: агентные методы восстановления изображения

Одна из ключевых задач после сбора данных айтрекинга [3] – предобработка изображения с восстановлением его структуры и накоплением агентами информации с предыдущих кадров. В отличие от прямого применения моделей машинного обучения [4] к изображению, агентные подходы позволяют сохранять контекст видео на уровне кадров [8].

Все рассмотренные агентные методы можно разделить на три подхода, основанных на способе организации агентов и механизме памяти:

1. Одноагентные диффузионные методы (Diffusion Agent) [19]. Единый агент (Гауссов шум) итеративно предсказывает и удаляет собственную зашумлённую компоненту. Память реализована через временной шаг t , убывающий от T до 0 , что даёт агенту информацию о прогрессе восстановления. Изображение формируется после полного прохода по шагам, где каждый уточняет структуру. Подход обеспечивает высокое качество, но требует больших вычислительных ресурсов.

2. Мультиагентные методы с феромонной памятью (Ant Colony Optimization) [20]. Множество агентов-муравьёв перемещается по графу пикселей, оставляя феромон – накапливаемую память о посещениях. Вероятность перехода в узел зависит от уровня феромона и эвристической привлекательности. Испарение феромона балансирует исследование новых областей и эксплуатацию найденных решений. Восстановленное изображение формируется на основе феромонной карты, где интенсивность феромона отражает значимость пикселя.

3. Мультиагентные методы с иерархическим сужением области поиска (агентный подход И.В. Кашина) [8]. Данный подход восстанавливает изображение, используя локальные перемещения агентов для достижения глобального результата. Ключевая идея заключается в том, что изначально не связанные между собой пиксели становятся глобально связанными после перевода интенсивностей в проценты (деления на общую сумму яркости), поэтому изменение одного пикселя влияет на восприятие всего изображения. Агенты перемещаются локально (по соседним клеткам холста), но выбирают направление на основе сравнения эталонного изображения и динамического распределения (накопленных посещений агентов) с учётом их норм. Если в текущей области агентов меньше, чем нужно для соответствия эталону, они стремятся туда; если больше – уходят в другое место. Таким образом,

локальными перемещениями агентов достигается глобальное восстановление исходного изображения.

В ходе обзора были рассмотрены конкретные реализации данных подходов, представленные в Таблице 2.

Таблица 2 – Аналоги агентных методов восстановления изображения

№	Название метода	Описание
1	Diffusion Agent	Одноагентный диффузионный подход [19]. Агент – гауссов шум, итеративно предсказывающий и удаляющий собственную зашумлённую компоненту. Память: временной шаг t .
2	Ant Colony Optimization (ACO)	Вероятность перехода агента из узла i в узел j : $p_{ij} \propto \tau_{ij}^\alpha \cdot \eta_{ij}^\beta$, где τ_{ij} – уровень феромона на ребре (i, j) , η_{ij} – эвристическая привлекательность узла j . α и β – параметры, определяющие влияние памяти и эвристики соответственно. Механизм испарения феромона предотвращает застревание в локальных оптимумах [20].
3	Агентный подход И.В. Кашина	Мультиагентный подход с глобальной связью через нормировку [8]. Коэффициент привлекательности: $K_p = \tilde{\rho} - \frac{\tilde{N}}{N_t} \rho_t$, где $\tilde{\rho}$ – эталонная яркость, ρ_t – накопленные посещения, $\tilde{N}$ – сумма эталонных яркостей, N_t – общее число посещений.

1.4.3 Группа аналогов: алгоритмы машинного обучения для распознавания областей интереса

Ключевой задачей, определяющей цель работы, является прогнозирование областей интереса на новых рекламных изображениях без необходимости проведения трудоёмких и дорогостоящих айтрекинг-исследований [3, 5]. Для решения этой задачи требуется построить модель [4], которая на основе анализа пикселей входного изображения будет

предсказывать тепловую карту внимания, что соответствует задачам регрессии или плотностной сегментации.

В ходе обзора были рассмотрены три класса алгоритмов, различающихся по архитектуре и принципу работы, для решения задачи предсказания тепловых карт внимания. Все рассмотренные методы можно разделить на:

1. Первый подход основан на применении глубоких свёрточных нейронных сетей и заключается в использовании модифицированной архитектуры U-Net [21], представляющей собой симметричную свёрточную нейросеть с кодирующе-декодирующей структурой (encoder-decoder). U-Net способна самостоятельно извлекать иерархические признаки из данных и предсказывать тепловые карты внимания [5] в режиме регрессии. Преимущество метода состоит в высокой точности благодаря сквозному обучению (end-to-end) и умеренной вычислительной стоимости обучения. Недостатками являются необходимость наличия размеченного набора данных и низкая интерпретируемость получаемых решений. Тем не менее, эта архитектура широко используется в задачах плотностной сегментации благодаря своей простоте и эффективности.

2. Второй подход реализует свёрточную нейронную сеть [22] с архитектурой encoder-decoder, в которой энкодер построен на базе сети VGG16, предварительно обученной на наборе данных ImageNet [26], включающем более 14 миллионов изображений с 1000 классами объектов. VGG16 последовательно уменьшает пространственное разрешение, увеличивая глубину карт признаков от 64 до 512 каналов, в то время как декодер восстанавливает детализацию до исходного размера изображения. Между энкодером и декодером размещается модуль ASPP (Atrous Spatial Pyramid Pooling) [27], обеспечивающий захват многомасштабного контекста. В работе реализован подход к обучению с учителем [28], при котором на вход модели подаются пары, состоящие из изображения и соответствующей карты внимания, а оптимизация ведётся с использованием комбинированной

функции потерь, объединяющей среднеквадратичную ошибку (MSE) и бинарную кросс-энтропию (BCE) [14,15]. Достоинствами метода являются автоматическое извлечение иерархических признаков, способность к обобщению сложных визуальных паттернов и высокая точность предсказаний [5]. К недостаткам следует отнести требование большого объёма размеченных данных для обучения ASPP-модуля и значительную вычислительную стоимость обучения. Однако высокая точность предсказаний оправдывает эти затраты для большинства практических применений.

3. Данный подход заключается в использовании архитектуры Transformer [23], адаптированной для предсказания карт внимания [5]. Изображение разбивается на последовательность патчей, которые преобразуются в токены с добавлением позиционных кодировок. Основу составляет энкодер трансформера с механизмом многоголового самовнимания (Multi-Head Self-Attention). Задача обучения сводится к подбору коэффициентов для голов внимания, позволяющих предсказывать области интереса человека на изображении. В работе реализован подход к обучению с учителем [28], при котором на вход модели подаются пары, состоящие из изображения и карты внимания, а оптимизация ведётся за счёт комбинированной функции потерь, объединяющей MSE и BCE [14, 15]. Данная архитектура демонстрирует высокую эффективность в задачах, требующих глобального понимания сцены, благодаря способности учитывать взаимосвязи между всеми элементами изображения независимо от их пространственного расположения. Преимущество – способность моделировать долгосрочные зависимости между удалёнными областями изображения. Недостаток – квадратичная вычислительная сложность и высокие требования к объёму обучающей выборки. Эти ограничения могут быть частично решены с помощью оптимизированных реализаций и уменьшения размера патчей.

Рассмотренные аналоги приведены в Таблице 3.

Таблица 3 – Аналоги алгоритмов машинного обучения

№	Название алгоритма	Описание
1	U-Net	Глубокая сверточная нейронная сеть симметричной U-образной архитектуры [21], изначально созданная для биомедицинской сегментации. Состоит из сжимающего пути энкодера для извлечения контекстных признаков и расширяющего пути декодера для точной пространственной локализации. Адаптируется для задачи регрессии путем замены выходного слоя на один канал с сигмоидальной или линейной активацией.
2	CNN	Сверточная нейронная сеть типа энкодер-декодер с ASPP-модулем для многомасштабного контекста [22]. Энкодер на базе VGG16 [29] извлекает иерархические признаки изображения, ASPP [27] анализирует эти признаки параллельно на нескольких масштабах, а декодер восстанавливает пространственное разрешение и преобразует признаки в тепловую карту значимости [5] со значениями от 0 до 1. Обучение с учителем [28] с комбинированной функцией потерь MSE+BCE [14, 15].
3	Vision Transformer	Трансформерная архитектура для компьютерного зрения [23, 25]: изображение 224×224 пикселя разбивается на 196 патчей 16×16 , каждый проецируется в вектор размерностью 768 с добавлением позиционных эмбеддингов. Обучение проводится методом самообучения [30] на основе дистилляции знаний между teacher и student. Для извлечения карт внимания применяется метод Attention Rollout [31] – рекурсивное перемножение матриц внимания всех блоков с добавлением остаточных связей. Полученная карта внимания 14×14 патчей [5] интерполируется до размера исходного изображения.

1.5 Оценка групп аналогов

В ходе литературно-аналитического обзора были отобраны аналоги для трёх групп [7, 10, 12]. В качестве первой группы аналогов были рассмотрены

системы программного айтрекинга [3]. Выбор системы влияет на точность первичных данных, о чём упоминалось в разделе 1.4.1, а как следствие, определяет качество входных данных и формирует требования к последующей обработке. В качестве второй группы аналогов рассмотрены агентные подходы восстановления изображения [8] с сохраняем контекста предыдущих кадров. Третьей группой аналогов выбраны алгоритмы машинного обучения [4] для распознавания областей интереса.

Для того чтобы перейти к выбору прототипа, в данном разделе будут сформулированы критерии оценки и выполнена оценка для каждой из групп аналогов [7, 10, 12].

1.5.1 Выбор критериев оценки для группы аналогов: системы программного айтрекинга

Для оценки аналогов предлагаются следующие критерии:

- Точность (Т) – способность системы корректно определять координату взгляда на экране.
- Надёжность/Стабильность (Н) – устойчивость работы при различных условиях (освещение, очки, движение головы).
- Простота интеграции (ПИ) – легкость внедрения в разрабатываемое приложение (качество API, документации, зависимости).
- Производительность (П) – потребление вычислительных ресурсов и возможность работы в реальном времени.
- Стоимость/Доступность (С) – наличие бесплатной лицензии для академических или коммерческих целей.

Общая модель оценки аналогов (ОА) задаётся формулой (1):

$$ОА_{\text{айтрек}} = \langle O(T), O(H), O(ПИ), O(П), O(С); R \rangle \quad (1)$$

Точность. Данный критерий оценивается по трехбалльной шкале и задаётся формулой (2):

$$I_T = \begin{cases} 1, \text{если точность высокая} \\ 0.5, \text{если точность средняя} \\ 0, \text{если точность низкая} \end{cases} \quad (2)$$

Надёжность. Данный критерий оценивается по двухбалльной шкале и задаётся формулой (3):

$$I_H = \begin{cases} 1, \text{если работа стабильна} \\ 0.5, \text{если работа средне стабильная} \\ 0, \text{если работа нестабильна} \end{cases} \quad (3)$$

Простота интеграции. Данный критерий оценивается по двухбалльной шкале и задаётся формулой (4):

$$I_{\text{пи}} = \begin{cases} 1, \text{если интеграция проста} \\ 0, \text{если интеграция сложна} \end{cases} \quad (4)$$

Производительность. Данный критерий оценивается по двухбалльной шкале и задаётся формулой (5):

$$I_{\text{п}} = \begin{cases} 1, \text{если производительность высокая} \\ 0.5, \text{если производительность средняя} \\ 0, \text{если производительность низкая} \end{cases} \quad (5)$$

Стоимость. Данный критерий оценивается по двухбалльной шкале и задаётся формулой (6):

$$I_c = \begin{cases} 1, \text{если решение бесплатное} \\ 0, \text{если требуется коммерческая лицензия} \end{cases} \quad (6)$$

Таким образом, оценка аналогов задаётся формулой (7):

$$OA_{\text{айтрек}} = O(T)\alpha(T) + O(H)\alpha(H) + O(\text{ПИ})\alpha(\text{ПИ}) + O(\text{П})\alpha(\text{П}) + O(C)*\alpha(C) \quad (7)$$

где $\sum \alpha_i = 1$;

α_i – весовой коэффициент i -го критерия.

1.5.2 Выбор критериев оценки для группы аналогов: агентных методов восстановления изображения

Для оценки аналогов предлагаются следующие критерии:

- Способность к накоплению контекста (СНК) – возможность метода сохранять и использовать информацию о предыдущих кадрах видеопотока.

- Сходимость и стабильность (СиС) – способность метода гарантированно достигать устойчивого состояния за конечное число итераций независимо от начальных условий.

- Вычислительная эффективность (ВЭ) – скорость обработки одного кадра и возможность работы в реальном времени.

- Масштабируемость (М) – способность метода эффективно работать при увеличении разрешения изображения или количества агентов без экспоненциального роста сложности.

- Интерпретируемость промежуточных состояний (ИПС) – наглядность и возможность анализа динамики работы агентов для понимания процесса восстановления.

Общая модель оценки аналогов (ОА) задаётся формулой (8):

$$OA_{\text{агент}} = \langle O(\text{СНК}), O(\text{СиС}), O(\text{ВЭ}), O(\text{М}), O(\text{ИПС}); R \rangle \quad (8)$$

Способность к накоплению контекста. Данный критерий оценивается по двухбалльной шкале и задаётся формулой (9):

$$I_{\text{СНК}} = \begin{cases} 1, & \text{если метод поддерживает накопление контекста} \\ 0, & \text{если каждый кадр обрабатывается независимо} \end{cases} \quad (9)$$

Сходимость и стабильность. Данный критерий оценивается по двухбалльной шкале и задаётся формулой (10):

$$I_{\text{сис}} = \begin{cases} 1, & \text{если гарантирована сходимость за конечное время} \\ 0, & \text{если сходимость не гарантирована} \end{cases} \quad (10)$$

Вычислительная эффективность. Данный критерий оценивается по двухбалльной шкале и задаётся формулой (11):

$$I_{\pi} = \begin{cases} 1, & \text{если метод требует маленьких вычислительных ресурсов} \\ 0.5, & \text{если метод требует средних вычислительных ресурсов} \\ 0, & \text{если метод требует значительных вычислительных ресурсов} \end{cases} \quad (11)$$

Масштабируемость. Данный критерий оценивается по двухбалльной шкале и задаётся формулой (12):

$$I_{\text{м}} = \begin{cases} 1, & \text{если вычислительная сложность растёт линейно} \\ 0, & \text{если сложность растёт квадратично или быстрее} \end{cases} \quad (12)$$

Интерпретируемость промежуточных состояний. Данный критерий оценивается по двухбалльной шкале и задаётся формулой (13):

$$I_{\text{ипс}} = \begin{cases} 1, & \text{возможно визуализировать положение агентов} \\ 0, & \text{если метод фиксирован} \end{cases} \quad (13)$$

Таким образом, оценка аналогов задаётся формулой (14):

$$OA_{\text{агент}} = O(\text{CHK})\alpha(\text{CHK}) + O(\text{СиС})\alpha(\text{СиС}) + O(\text{ВЭ})\alpha(\text{ВЭ}) + O(\text{М})\alpha(\text{М}) + O(\text{ИПС})\alpha(\text{ИПС}), \quad (14)$$

где $\sum \alpha_i = 1$;

α_i – весовой коэффициент i -го критерия.

1.5.3 Выбор критериев оценки для группы аналогов: алгоритмы машинного обучения

Для оценки аналогов предлагаются следующие критерии:

- Точность предсказания (ТП) – основная метрика качества модели (MSE, BCE).
- Скорость обучения (СО) – время, необходимое для обучения модели на подготовленном наборе данных.
- Скорость вывода (СВ) – время, затрачиваемое на предсказание областей интереса для нового изображения.
- Интерпретируемость (Ин) – возможность объяснить, на основе каких признаков модель принимает решение.
- Вычислительные ресурсы (ВР) – объём оперативной памяти, требования к GPU/CPU и дисковому пространству, необходимые для обучения и работы модели.

Общая модель оценки аналогов (ОА) задаётся формулой (15):

$$OA_{\text{мл}} = \langle O(\text{ТП}), O(\text{СО}), O(\text{СВ}), O(\text{Ин}), O(\text{ВР}); R \rangle \quad (15)$$

Точность предсказания. Данный критерий оценивается по трехбалльной шкале и задаётся формулой (16):

$$I_{\text{ТП}} = \begin{cases} 1, \text{если ожидается высокая точность} \\ 0.5, \text{если ожидается средняя точность} \\ 0, \text{если ожидается низкая точность} \end{cases} \quad (16)$$

Скорость обучения. Данный критерий оценивается по двухбалльной шкале и задаётся формулой (17):

$$I_{\text{СО}} = \begin{cases} 1, \text{если скорость обучения высокая} \\ 0.5, \text{если скорость обучения средняя} \\ 0, \text{если скорость обучения низкая} \end{cases} \quad (17)$$

Скорость вывода. Данный критерий оценивается по двухбалльной шкале и задаётся формулой (18):

$$I_{\text{св}} = \begin{cases} 1, \text{если скорость вывода высокая} \\ 0.5, \text{если скорость вывода средняя} \\ 0, \text{если скорость вывода низкая} \end{cases} \quad (18)$$

Интерпретируемость. Данный критерий оценивается по двухбалльной шкале и задаётся формулой (19):

$$I_{\text{ин}} = \begin{cases} 1, \text{если модель интерпретируема} \\ 0, \text{если модель является чёрным ящиком} \end{cases} \quad (19)$$

Вычислительные ресурсы. Данный критерий оценивается по двухбалльной шкале и задаётся формулой (20):

$$I_{\text{вр}} = \begin{cases} 1, \text{если требования к вычислительным ресурсам низкие} \\ 0.5, \text{если требования к вычислительным ресурсам средние} \\ 0, \text{если требования к вычислительным ресурсам высокие} \end{cases} \quad (20)$$

Таким образом, оценка аналогов задаётся формулой (21):

$$OA_{\text{мл}} = O(\text{ТП})\alpha(\text{ТП}) + O(\text{СО})\alpha(\text{СО}) + O(\text{СВ})\alpha(\text{СВ}) + O(\text{Ин})\alpha(\text{Ин}) + O(\text{ВР})\alpha(\text{ВР}), \quad (21)$$

где $\sum \alpha_i = 1$;

α_i – весовой коэффициент i -го критерия.

1.5.4 Определение весовых коэффициентов критериев оценки

В рамках проведения парных сравнений критериев широко применяется метод анализа иерархий (Analytic Hierarchy Process, АНП) [7, 10, 32], который обеспечивает формализацию субъективных экспертных оценок относительно значимости каждого из рассматриваемых критериев. Данный метод позволяет структурировать процесс принятия решений, преобразуя качественные

суждения в количественные показатели, что особенно важно в условиях многокритериального анализа. Для реализации метода АНР [32] используется специализированная шкала словесных определений уровня важности. Каждому словесному определению степени превосходства одного критерия над другим соответствует числовое значение, что стандартизирует оценку и снижает влияние субъективных факторов. Указанные уровни важности [7, 10] детализированы и представлены в Таблице 4. Данная шкала основывается на классической методике Томаса Саати [33] и адаптирована для решения задач сравнительного анализа критериев в контексте проектирования информационных систем. Числовые значения шкалы отражают качественное и количественное превосходство одного критерия над другим.

Таблица 4 – Шкала уровней важности

Уровень важности	Значение
Абсолютное превосходство	9
Промежуточное значение между значительным и абсолютным превосходством	8
Значительное (большое) превосходство	7
Промежуточное значение между сильным и значительным превосходством	6
Существенное или сильное превосходство	5
Промежуточное значение между слабым и сильным превосходством	4
Умеренное превосходство	3
Промежуточное значение между равенством и слабым превосходством	2
Равная важность	1

Матрицы попарного сравнения для определения весовых коэффициентов (α) представлены в Таблицах 5-7 [7, 10].

Таблица 5 – Оценка весовых коэффициентов критериев для систем айтрекинга

Критерий	Т	Н	ПИ	П	С	Сумма	α
Т	1	3	1/5	1/3	5	9.53	0.18
Н	1/3	1	5	1/7	3	9.48	0.18
ПИ	5	1/5	1	1/5	1/7	6.54	0.12
П	3	7	5	1	1/3	16.33	0.31
С	1/5	1/3	7	3	1	11.53	0.21
Общая сумма						53.42	1.00

Таблица 6 – Оценка весовых коэффициентов критериев для агентных методов восстановления изображения

Критерий	СНК	СиС	ВЭ	М	ИПС	Сумма	α
СНК	1	3	3	3	9	19	0.29
СиС	1/3	1	3	5	7	18.33	0.28
ВЭ	1/3	1/3	1	5	7	15.67	0.24
М	1/3	1/5	1/5	1	5	8.73	0.13
ИПС	1/9	1/7	1/7	1/5	1	3.6	0.06
Общая сумма						65.33	1

Таблица 7 – Оценка весовых коэффициентов критериев для алгоритмов машинного обучения

Критерий	ТП	СО	СВ	Ин	ВР	Сумма	α
ТП	1	9	7	7	5	29.00	0.54
СО	1/9	1	1/3	1/3	1/5	1.84	0.03
СВ	1/7	3	1	1	1/3	5.48	0.10
Ин	1/7	3	1	1	1/3	5.48	0.10
ВР	1/5	5	3	3	1	12.2	0.23
Общая сумма						54	1.00

1.5.5 Сравнение аналогов в рамках каждой из групп

Формула (22) для вычисления весовых коэффициентов:

$$\alpha = \frac{\sum_1^n o_i}{\text{Общая сумма}}, \quad (22)$$

где $\sum O_i$ – сумма оценок относительной важности одного критерия по сравнению с другими в строке матрицы парных сравнений;

Общая сумма – сумма всех значений в матрице парных сравнений;

n – количество критериев, по которым проводится сравнение.

В соответствии с найденной информацией об аналогах, в Таблицах 8-10 приведены показатели критериев для каждой группы [7, 10].

Таблица 8 – Показатели критериев группы аналогов: системы айтрекинга

Название	Т	Н	ПИ	П	С
eyeGestures	0.5	0.5	1	1	1
OpenCV	1	1	0	0	1
dlib	1	0.5	0	0.5	1

Таблица 9 – Показатели критериев группы аналогов: агентные методы восстановления изображения

Название	СНК	СиС	ВЭ	М	ИПС
Diffusion Agent	0	1	1	1	0
Ant Colony Optimization (ACO)	1	1	0.5	0	1
Агентный подход И.В. Кашина	1	1	1	1	1

Таблица 10 – Показатели критериев группы аналогов: алгоритмы машинного обучения

Название	ТП	СО	СВ	ИН	ВР
U-Net	0	0.5	1	0	1
CNN	1	0.5	0.5	1	0.5
Vision Transformer	0.5	0	1	1	0.5

Итоговая оценка аналогов по критериям [7, 10] представлена в Таблицах 11-13.

Таблица 11 – Оценка группы аналогов: системы айтрекинга

Название	Т	Н	ПИ	П	С	Интегральная оценка (α)
eyeGestures	0.09	0.09	0.12	0.31	0.21	0.82
OpenCV	0.18	0.18	0	0	0.21	0.57
dlib	0.18	0.09	0	0.155	0.21	0.635

Таблица 12 – Оценка группы аналогов: агентные методы восстановления изображения

Название	СНК	СнС	ВЭ	М	ИПС	Интегральная оценка (α)
Diffusion Agent	0	0.28	0.24	0.13	0	0.65
Ant Colony Optimization	0.29	0.28	0.12	0	0.06	0.75
Агентный подход И.В. Кашина	0.29	0.28	0.24	0.13	0.06	1

Таблица 13 – Оценка группы аналогов: алгоритмы машинного обучения

Название	ТП	СО	СВ	Ин	ВР	Интегральная оценка (α)
U-Net	0	0.015	0.1	0	0.23	0.345
CNN	0.54	0	0.05	0.1	0.115	0.805
Vision Transformer	0.27	0	0.1	0.1	0.115	0.585

1.5.6 Выбор прототипа для каждой из групп аналогов, критика

Прототипом нулевого ранга [7, 10] можно считать разрозненный исследовательский процесс, где для анализа внимания к рекламе используются отдельные, не интегрированные инструменты: профессиональный айттрекер [3] для сбора данных или стандартные библиотеки машинного обучения [4] для построения прогнозных моделей. Такой прототип обладает точностью, но исключительно дорог, сложен в воспроизведении и не представляет собой целостной системы.

В первой группе аналогов (системы программного айттрекинга [3]) наибольшую интегральную оценку получила библиотека eyeGestures (0.82) [16]. Критика выбранного прототипа заключается в его средней точности (0.5) и потенциальной нестабильности работы в неконтролируемых условиях (разное освещение, наличие очков), что отражено средним баллом по критерию «Надёжность» (0.5). Тем не менее, ключевое преимущество – простота интеграции (1.0) и нулевая стоимость (1.0) – является определяющим для создания рабочего прототипа системы.

Во второй группе аналогов (агентные методы восстановления изображения) лидером стал агентный подход И.В. Кашина [8] с интегральной оценкой 1.00. Данный метод превосходит аналоги по всем ключевым критериям: способность к накоплению контекста (1.0), гарантированная сходимость (1.0), высокая вычислительная эффективность (1.0), линейная масштабируемость (1.0) и полная интерпретируемость промежуточных

состояний (1.0). Критика прототипа минимальна в контексте поставленной задачи.

В третьей группе аналогов (алгоритмы машинного обучения [4]) наивысшую интегральную оценку получила свёрточная нейронная сеть CNN (0.805) [22], значительно опередив Vision Transformer (0.585) [23] и U-Net (0.345) [21]. Критика выбранного прототипа: CNN демонстрирует наивысшую ожидаемую точность предсказания (0.54) благодаря использованию предобученной на ImageNet [26] архитектуры VGG16 [29] и ASPP-модуля [27] для захвата многомасштабного контекста. Однако модель имеет низкую скорость обучения ASPP-модуля (0) и низкую скорость предсказания (0.05). Тем не менее, именно эта архитектура демонстрирует лучшее качество предсказания карт внимания и выбирается в качестве прототипа для дальнейшего развития системы. Vision Transformer [23] может рассматриваться как компромиссный вариант при ограниченном времени на вывод результата (0.1).

Таким образом, получен компилятивный прототип, состоящий из:

1. Системы сбора данных на базе eyeGestures [16];
2. Агентного метода И.В. Кашина восстановления изображения с накоплением контекста [8];
3. Прогнозной модели на основе архитектуры CNN (VGG16 + ASPP + декодер) [22, 27, 29].

Данный прототип требует доработки, направленной на повышение надёжности айтрекинга [3] в неконтролируемых условиях за счёт дополнительной предобработки видеопотока, подбора параметров агентного метода [8], обучения модуля ASPP [27] модели CNN [22] для качественного предсказания тепловых карт.

1.6 Гипотеза о предлагаемом решении

Гипотеза о предлагаемом решении заключается в том, что разработка целостной информационной системы, интегрирующей агентный метод

восстановления изображения с накоплением контекста, разработанный И.В. Кашиным [8], алгоритм машинного обучения на основе сверточной нейронной сети CNN [22] и программный айтрекинг [3] EyeGestures [16] для обучения ASPP-модуля [27], позволит эффективно и с достаточной точностью автоматически распознавать области интереса [5] на рекламных изображениях. Это в свою очередь обеспечит инструментальную поддержку для принятия маркетинговых решений по оптимизации визуального контента на основе объективных данных о внимании пользователей, а также снизит зависимость от дорогостоящего оборудования и субъективных экспертных оценок.

1.7 Цели и задачи исследования

Глобальной целью исследования является повышение эффективности современного информационного маркетинга за счёт предоставления автоматизированного, объективного и масштабируемого инструмента для анализа визуального внимания целевой аудитории.

Локальной целью работы является развитие информационной системы поддержки маркетинговых решений на основе создания и экспериментальной проверки работоспособности прототипа, способного автоматически распознавать области интереса [5] на рекламных изображениях.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Реализовать модуль сбора данных на основе библиотеки программного айтрекинга (eyeGestures) [16].
2. Сформировать набор данных пар «рекламное изображение – тепловая карта внимания».
3. Реализовать, обучить и сравнить эффективность прогнозных моделей на основе алгоритмов U-Net, CNN и Vision Transformer [21, 22, 23].
4. Интегрировать модули в прототип системы с интерфейсом для загрузки изображения и отображения области интереса [7].

5. Провести экспериментальную оценку точности работы системы и проанализировать результаты.

1.8 Результаты и выводы по первой главе

В ходе работы над первой главой были получены следующие результаты:

- Рассмотрены особенности анализа визуального внимания [5] пользователей к рекламным изображениям в контексте современного информационного маркетинга.
- Сформулирована и подтверждена литературными источниками проблема отсутствия доступных автоматизированных решений для объективной оценки рекламных изображений на основе данных о взгляде [5]. Показана актуальность предлагаемой разработки.
- Определены ключевые компоненты системы: модуль сбора данных, агентный модуль восстановления изображения и модуль прогнозирования [3, 4, 5].
- Проведён обзор трёх групп аналогов: систем программного айтрекинга [3], агентных методов восстановления изображения [8] и алгоритмов машинного обучения для распознавания областей интереса [4].
- Разработаны системы критериев для оценки аналогов [7, 10], проведено их сравнение с определением весовых коэффициентов и расчётом интегральных оценок.
- На основе оценки выбран компилятивный прототип (eyeGestures, метод Кашина, CNN) [8, 16, 22], дана его критика с выделением направлений улучшения.
- Сформулирована гипотеза решения, уточнены цели и задачи исследования.

Проведенный обзор и анализ позволяют сделать вывод, что получено достаточно информации для начала проектирования и разработки предлагаемой информационной системы [7].

2 МОДЕЛИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ АНАЛИЗА ВНИМАНИЯ НА ОСНОВЕ АЙТРЕКИНГА И МАШИННОГО ОБУЧЕНИЯ

2.1 Концептуальная модель

Описание концептуальной модели [7, 10] предложено реализовать в рамках следующего кортежного формализма (23):

$$KM = \langle \Phi, \Pi, C, H, \Psi \rangle, \quad (23)$$

где КМ – концептуальная модель;

Φ – функции;

Π – путь реализации функций;

С – структура;

Н – направленность;

Ψ – цель для определяемого понятия.

Общая концептуальная модель

Модель анализа внимания пользователей к рекламным изображениям – последовательность процедур, выполняющих такие функции как: сбор данных о движении взгляда с использованием программного айттрекинга [3]; создание тепловых карт областей внимания [5] и создание набора данных; обучение модели машинного обучения [4] на наборе данных; тест обученной модели; предобработка полученных данных с накоплением контекста между кадрами [8]; прогнозирование областей интереса на основе обученных алгоритмов машинного обучения; визуализация результатов в виде тепловых карт. Модель направлена на получение объективных сведений о распределении внимания целевой аудитории в виде интерпретированных признаков (координат фиксаций, зон повышенного интереса).

Базово-уровневая концептуальная модель

Модель анализа внимания пользователей к рекламным изображениям с использованием айттрекинга [3] и машинного обучения [4] – представляет

собой последовательность комбинированных и специфических процедур, выполняющих *функции*: сбора данных о движении взгляда с использованием программного айтрекинга [3], создания тепловых карт областей внимания [5] и создания набора данных, обучения модели машинного обучения [4] на наборе данных, теста обученной модели, предобработки полученных данных с накоплением контекста между кадрами, прогнозирования областей интереса на основе обученных алгоритмов машинного обучения и визуализации результатов в виде тепловых карт *путем*: захвата изображения экрана в реальном времени или использования сохраненной последовательности видеок кадров; получения координат взгляда пользователя через веб-камеру с использованием библиотеки eyeGestures [16]; построения тепловых карт на основе собранных координат взгляда с применением гауссова размытия [34]; формирования размеченного набора данных из пар «рекламное изображение – тепловая карта внимания»; извлечения признаков из изображений; обучения модуля ASPP [27] в модели CNN [22] на сформированном наборе данных; оценки качества предсказаний модели на отложенной тестовой выборке с использованием метрик MSE и BCE [14, 15]; преобразования цветного изображения в полутоновое [35]; разбиения на дискретную сетку ячеек; нормализации эталонного распределения яркости; применения агентного подхода [8] для накопления контекста предыдущих кадров и восстановления структуры изображения; подачи предобработанного изображения на вход обученной модели [4]; получения предсказанной карты внимания [5]; построения цветовой тепловой карты на основе предсказанных значений; наложения тепловой карты на исходное рекламное изображение; отображения результатов пользователю.

Основа структуры модели анализа внимания пользователей состоит из подсистем: блок сбора данных, блок создания тепловых карт и формирования набора данных, блок обучения модели машинного обучения [4], блок тестирования обученной модели, блок предобработки с накоплением

контекста [8], блок прогнозирования областей интереса [5], блок визуализации результатов, интерфейс пользователя.

Модель анализа внимания пользователей к рекламным изображениям *направленна на* удовлетворение потребностей маркетологов, веб-дизайнеров и UX-аналитиков в знании и определении областей повышенного визуального интереса [5] на рекламных изображениях, характеризующих эффективность восприятия ключевых элементов (товаров, брендов).

Модификационная модель

Модель анализа внимания пользователей [5] к рекламным изображениям на основе агентного подхода [8] накопления контекста с предыдущих кадров видео – представляет собой последовательность комбинированных и специфических процедур, выполняющих *функции*: 1 – сбора данных о движении взгляда с использованием программного айтрекинга [3]; 2 – создания тепловых карт областей внимания [5] и формирования набора данных: 2.1 – построение тепловых карт [5] на основе собранных координат взгляда, 2.2 – применение гауссова размытия для сглаживания [34], 2.3 – формирование размеченного набора данных из пар «рекламное изображение – тепловая карта внимания»; 3 – обучения модели машинного обучения [4] на наборе данных: 3.1 – извлечение признаков из изображений, 3.2 – обучение модуля ASPP [27] в модели CNN [22] на сформированном наборе данных, 3.3 – оптимизация весов модели с использованием функции потерь; 4 – теста обученной модели: 4.1 – оценка качества предсказаний на отложенной тестовой выборке, 4.2 – расчет метрик MSE и BCE [14, 15]; 5 – предобработки полученных данных с накоплением контекста между кадрами: 5.1 – преобразование цветного изображения в полутоновое [35], 5.2 – разбиение на дискретную сетку ячеек, 5.3 – нормализация эталонного распределения яркости, 5.4 – применение агентного подхода И.В. Кашина для накопления контекста предыдущих кадров [8], 5.5 – восстановление структуры изображения на основе динамического распределения посещений агентов; 6 – прогнозирования областей интереса на основе обученных алгоритмов

машинного обучения [4]: 6.1 – подача предобработанного изображения на вход обученной модели, 6.2 – получение предсказанной тепловой карты внимания [5], 6.3 – постобработка результатов; 7 – визуализации результатов в виде тепловых карт: 7.1 – построение цветовой тепловой карты на основе предсказанных значений интенсивности внимания, 7.2 – наложение тепловой карты на исходное рекламное изображение, 7.3 – отображение результатов пользователю.

Путем: захвата изображения экрана в реальном времени и получения координат взгляда пользователя через веб-камеру с использованием библиотеки eyeGestures [16] на языке Python [24] (для функции 1); построения тепловых карт [5] с применением гауссова размытия и формирования набора данных средствами Python с использованием библиотек NumPy [36] (для функции 2); выполнения обучения модели модуля ASPP [27] в модели CNN [22] на языке Python с использованием библиотеки TensorFlow/Keras [37, 38] (для функции 3); расчета метрик MSE и BCE [14, 15] на отложенной тестовой выборке средствами Python с использованием библиотек scikit-learn и NumPy [39, 40] (для функции 4); выполнения алгоритмов нормализации, разбиения на сетку ячеек и агентного восстановления [8] изображения на C++ [41] с использованием разработанной библиотеки agent_sim (для функции 5); выполнения прямого прохода (инференса) через обученную модель на языке Python [24] с использованием TensorFlow/Keras [37, 38] (для функции 6); построения и наложения тепловых карт на языке Python с использованием библиотек pygame, OpenCV и NumPy [17, 40, 42] (для функции 7).

Выше озвученные функции реализуются *на основе* структуры из маркетологов и UX-аналитиков, методической информации (знаний) о процессе оценки визуального внимания и критериях эффективности рекламных изображений, программного комплекса предобработки изображений с накоплением контекста на основе агентного подхода [8], реализованного на C++ [41], системы машинного обучения [4] для прогнозирования тепловых карт внимания [5] на базе архитектуры CNN [22]

на Python [24] и диалогово-логической системы для связи специалистов с программой через графический интерфейс пользователя. Модель *направлена* на удовлетворение потребностей маркетологов, веб-дизайнеров и UX-аналитиков в знании и определении областей повышенного визуального интереса на рекламных изображениях, характеризующих эффективность восприятия ключевых элементов (товаров, брендов), *с целью* обеспечения более высокого уровня конверсии рекламных кампаний и оптимизации визуального контента на основе объективных данных о распределении внимания целевой аудитории.

2.2 Структурная модель

Структурная схема [7, 10] проектируемой модели анализа нулевого ранга представлена на рисунке 5.

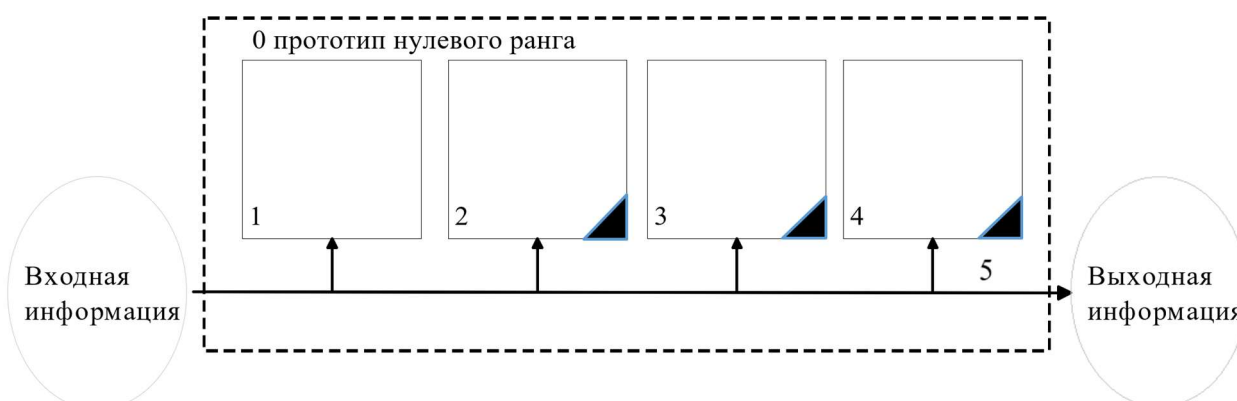


Рисунок 5 – Системно–структурная модель по предлагаемому решению:
новизна обозначена штриховкой; модификации – треугольник

На рисунке 5 используются следующие обозначения:

- 1 – подсистема загрузки данных;
- 2 – подсистема предобработки данных;
- 3 – подсистема применения модели машинного обучения [4];
- 4 – подсистема визуализации внимания (построение тепловой карты [5]);
- 5 – интерфейс пользователя.

Предлагаемое решение связано с модернизацией подсистемы 2, с обучением и внедрением модели машинного обучения [4] из подсистемы 3 и визуализацией областей интереса [5] в подсистеме 4. Развитие подсистемы 2 составляет основную новизну работы и связано с необходимостью предобработки изображений с добавлением контекста из предыдущих кадров для последующего анализа [8]. Модификация подсистемы 3 связана с обучением и внедрением модели машинного обучения для прогнозирования карты внимания [4, 5]. Модификация подсистемы 4 связана с автоматизацией преобразования прогнозов модели в интуитивно понятные тепловые карты [5] для визуального анализа, а также с реализацией наложения предсказанной тепловой карты поверх экрана компьютера или видеопотока.

На рисунке 6 представлены системно-структурные модели первого ранга для второй подсистемы проектируемой модели [7, 10].

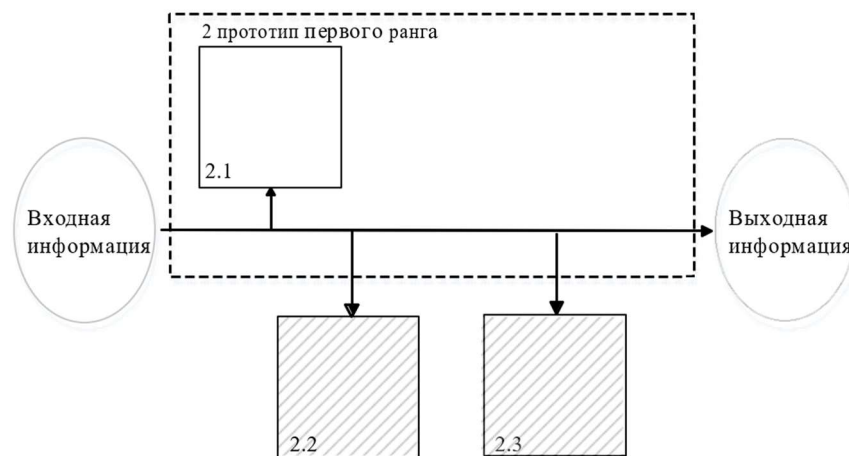


Рисунок 6 – Системно-структурная модель первого ранга второй подсистемы

На рисунке 6 используются следующие обозначения:

- 2.1 – подсистема предобработки данных (изображения);
- 2.2 – подсистема расчёта коэффициента привлекательности K_p (вероятность перемещения агента в данную область) [8];
- 2.3 – подсистема перемещения агентов [8] (обеспечивает выбор одной из восьми соседних клеток с максимальным значением коэффициента K_p и осуществляет перемещение агентов в выбранную область).

Предлагаемое решение связано с внедрением модулей 2.2, 2.3. Внедрение модуля 2.2 и 2.3 связана с необходимостью использования агентного подхода восстановления изображения с сохранением контекста предыдущего кадра [8].

На рисунке 7 представлены системно-структурные модели первого ранга для третьей подсистемы проектируемой модели [7, 10].

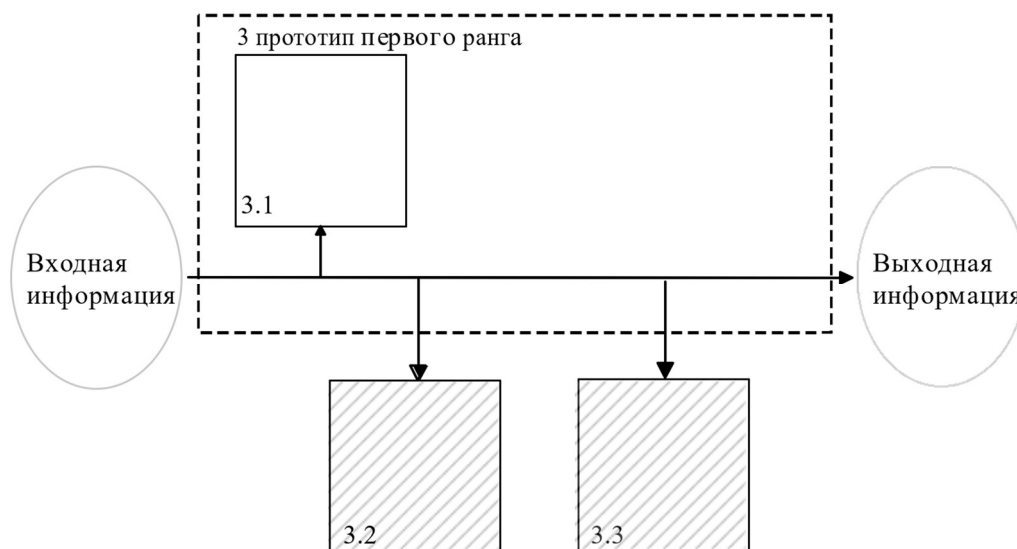


Рисунок 7 – Системно-структурная модель первого ранга третьей подсистемы

На рисунке 7 используются следующие обозначения:

- 3.1 – подсистема извлечения иерархических признаков на основе предобученной сверточной основы VGG16 [29];
- 3.2 – подсистема многомасштабного анализа контекста на основе ASPP-модуля [27];
- 3.3 – подсистема восстановления пространственного разрешения и формирования карты внимания на основе декодера [22].

Предлагаемое решение связано с внедрением модулей 3.2 и 3.3 [7]. В качестве основы для построения прогностической модели используются слои предобученной сверточной нейронной сети VGG16 [22, 29], которые

выполняют функцию извлечения иерархических признаков из входного изображения. На выходе модуля 3.1 формируются карты признаков [5], обобщающие визуальные характеристики рекламного изображения, такие как формы, текстуры и границы объектов. Эти карты признаков поступают на вход обучаемого ASPP [27] из модуля 3.2, который анализирует их параллельно с использованием свёрток с разными коэффициентами расширения. Благодаря этому модуль 3.2 способен захватывать контекстную информацию одновременно на нескольких масштабах. Полученные после ASPP-модуля [27] многомасштабные карты признаков затем передаются в декодер из модуля 3.3, который последовательно восстанавливает пространственное разрешение и формирует тепловую карту [5] внимания, где каждый пиксель соответствует прогнозируемой интенсивности взгляда пользователя. Таким образом, использование модуля 3.1 для извлечения признаков из изображения, обучение на их основе ASPP-модуля [27] и декодера из модуля 3.2 и 3.3 позволяет повысить точность предсказания карт внимания [5] за счёт учёта объектов разных масштабов по сравнению с использованием только базовой свёрточной сети [22].

На рисунке 8 представлены системно-структурные модели первого ранга для четвертой подсистемы проектируемой модели [7, 10].

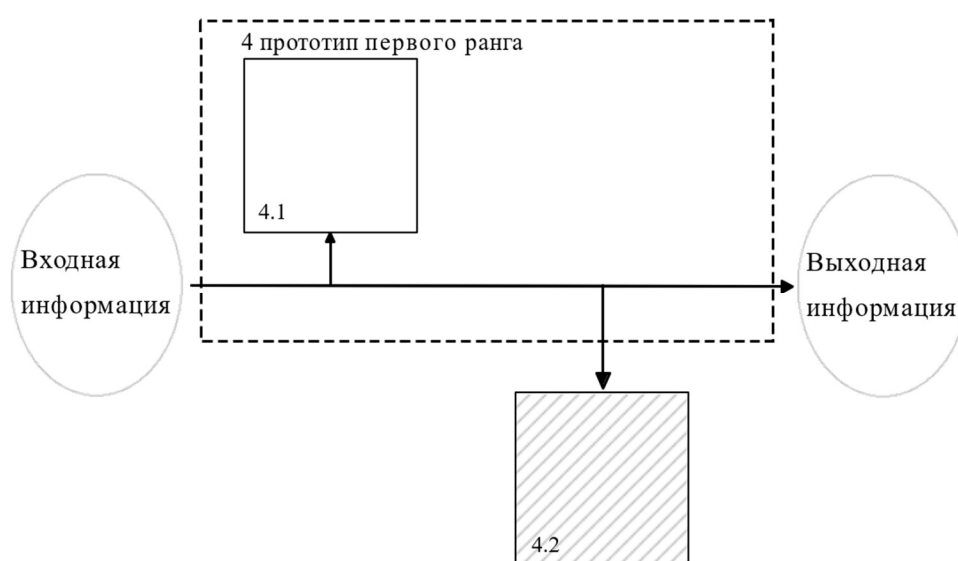


Рисунок 8 – Системно-структурная модель первого ранга четвертой подсистемы

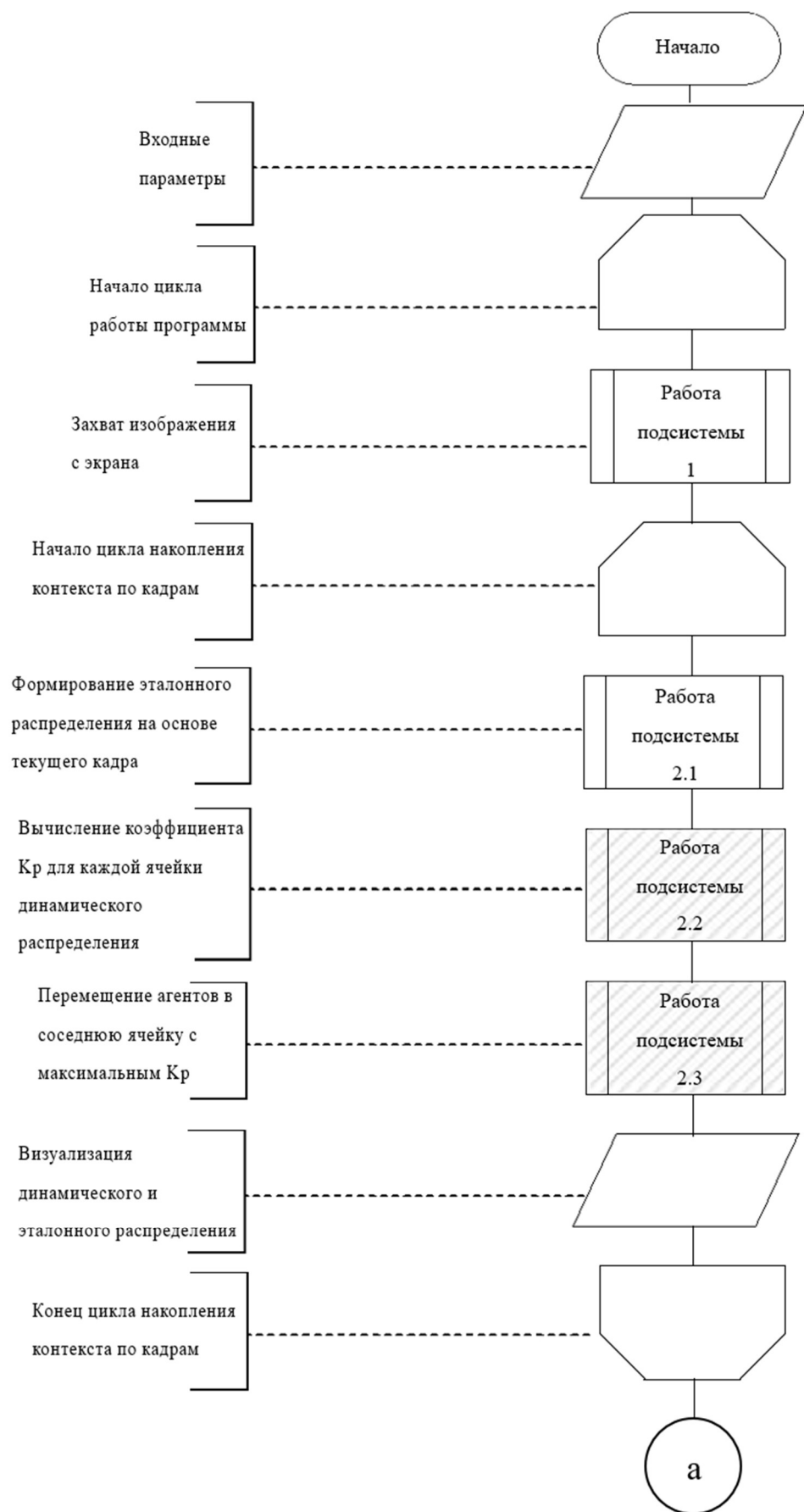
На рисунке 8 используются следующие обозначения:

- 4.1 – подсистема преобразования предсказанной моделью карты областей внимания [5] из оттенков серого в цветовую тепловую карту;
- 4.2 – подсистема наложения предсказанной тепловой карты поверх изображения.

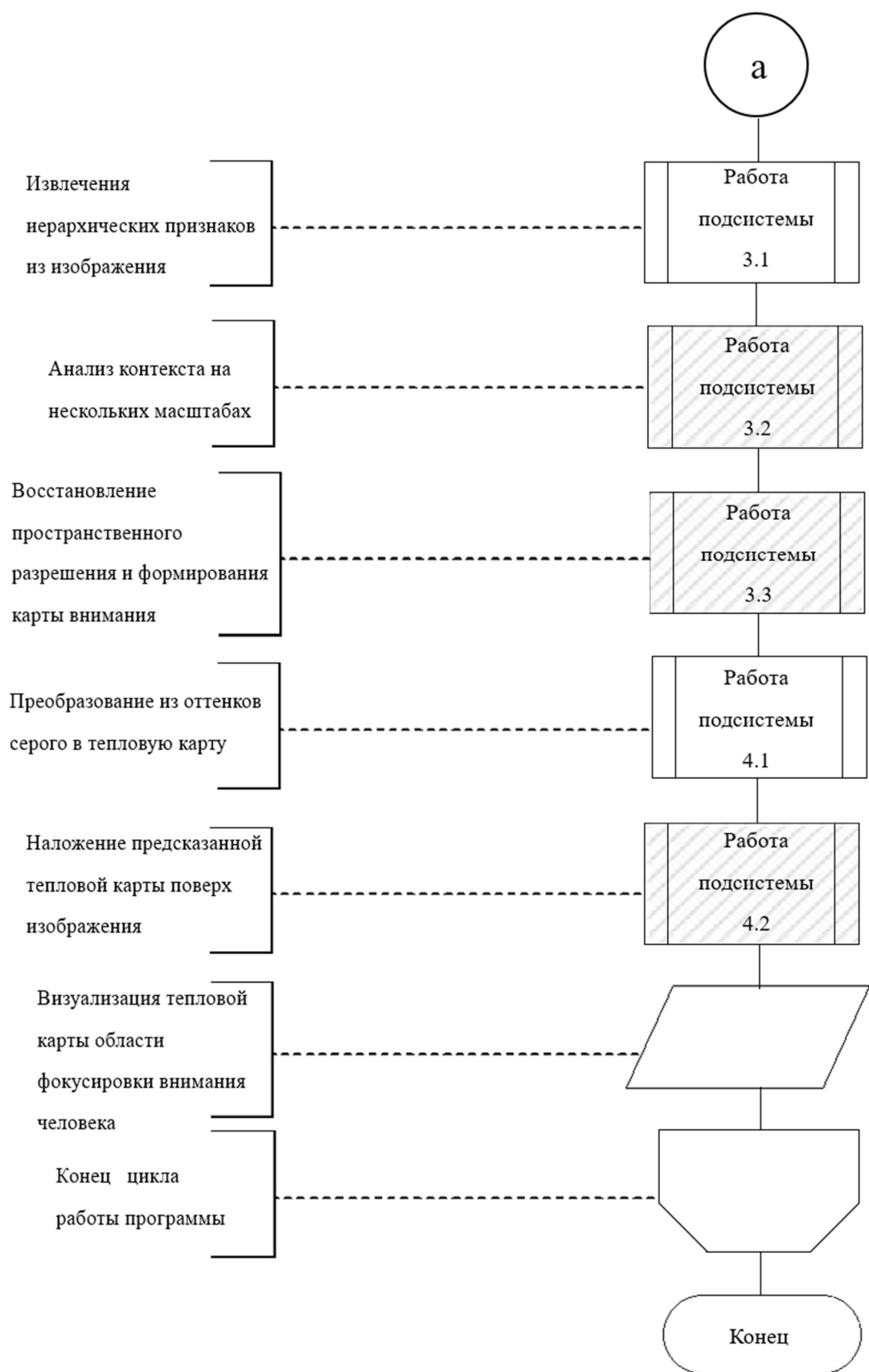
Предлагаемое решение связано с внедрением модуля 4.2, что обусловлено необходимостью гибкой визуализации результатов прогнозирования: в режиме захвата экрана тепловая карта накладывается поверх текущего изображения экрана компьютера пользователя с регулируемой прозрачностью, а в режиме обработки видео – поверх кадров исходного видеопотока с последующим сохранением результирующего видео с наложенной тепловой картой [5].

2.3 Алгоритмическая модель

В данном разделе представлено формализованное описание алгоритмов [7, 10], лежащих в основе разрабатываемой информационной системы для анализа внимания пользователей к рекламным изображениям. Алгоритмическая модель включает в себя последовательность шагов, необходимых для предобработки входного изображения с накоплением контекста между кадрами на основе агентного подхода [8], применения обученной модели машинного обучения [4] для прогнозирования областей интереса и визуализации полученных результатов в виде тепловой карты [5]. Каждый этап алгоритма направлен на обеспечение точности, надёжности и автоматизации процесса анализа, а также на минимизацию участия человека в интерпретации данных. Особое внимание уделено этапу агентной [8] предобработки, который позволяет учитывать историю предыдущих кадров видеопотока и повышает качество последующего прогнозирования. Схема алгоритма отражает последовательность действий от загрузки рекламного изображения до визуализации областей интереса (см. рисунок 9 а, б).



а)



б)

Рисунок 9 – Алгоритмическая схема по ГОСТ 19.701-90 работы разрабатываемого решения

2.4 Функционально–структурная модель

Для наиболее полного понимания процесса функционирования разрабатываемого решения была создана функционально–структурная модель в соответствии с Р 50.1.028–2001 [7, 10]. Для построения моделей использовано программное средство AllFusionProcessModeler 7.3 [43].

На рисунке 10 приведена функционально–структурная модель проектируемого решения нулевого уровня [7, 10], отображающая входящие и исходящие данные, а также механизмы управления и функционирования в целом.

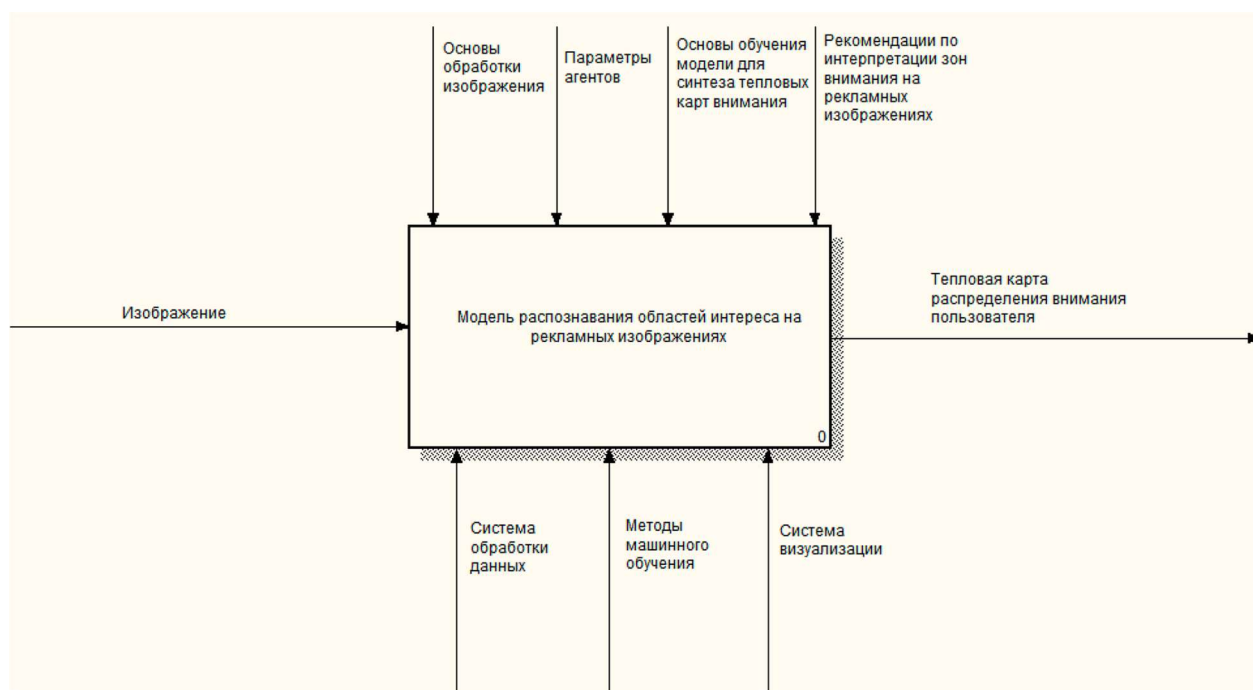


Рисунок 10 – IDEF0 модель TO-BE нулевого уровня

На рисунке 11 приведена функционально–структурная модель проектируемого решения первого уровня [7, 10], где детализированы основные направления деятельности системы, представляющие собой ключевые подсистемы: сбор данных, их предобработка, машинное обучение [4] для прогнозирования областей интереса [5], визуализация результатов, что обеспечивает целостное представление о взаимодействии компонентов системы.

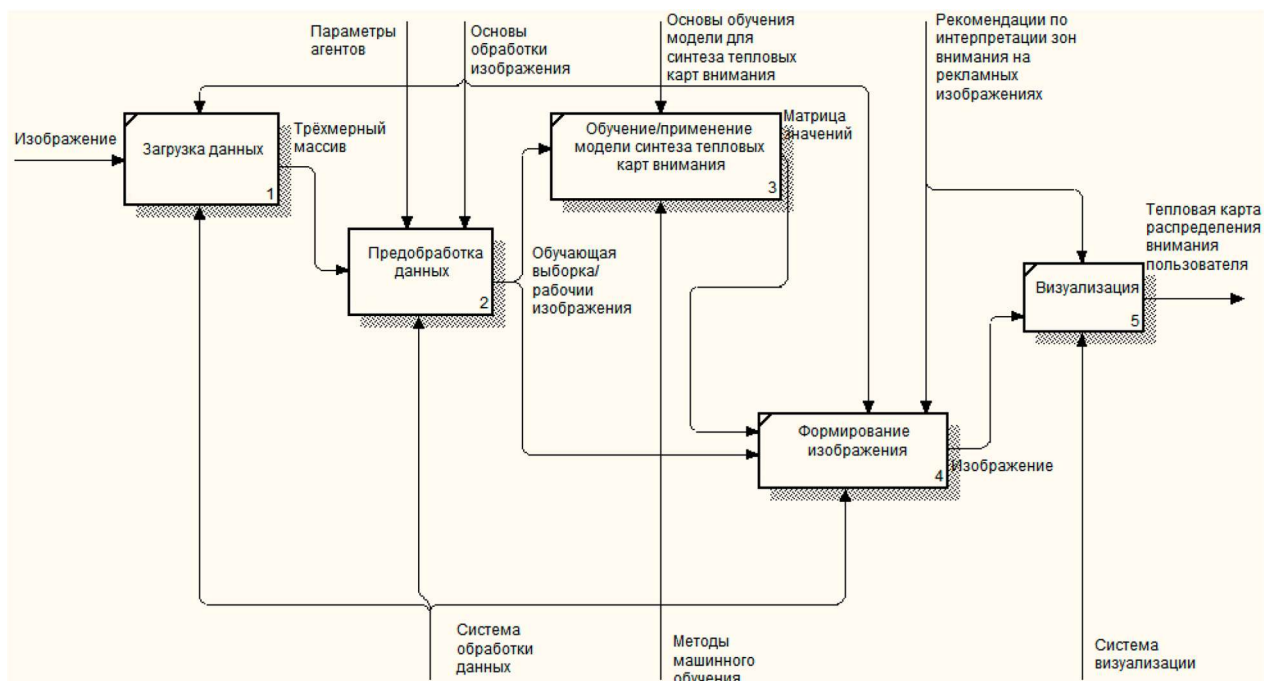


Рисунок 11 – IDEF0 модель ТО-ВЕ первого уровня

На рисунке 12 приведена функционально-структурная модель второго уровня [7, 10].

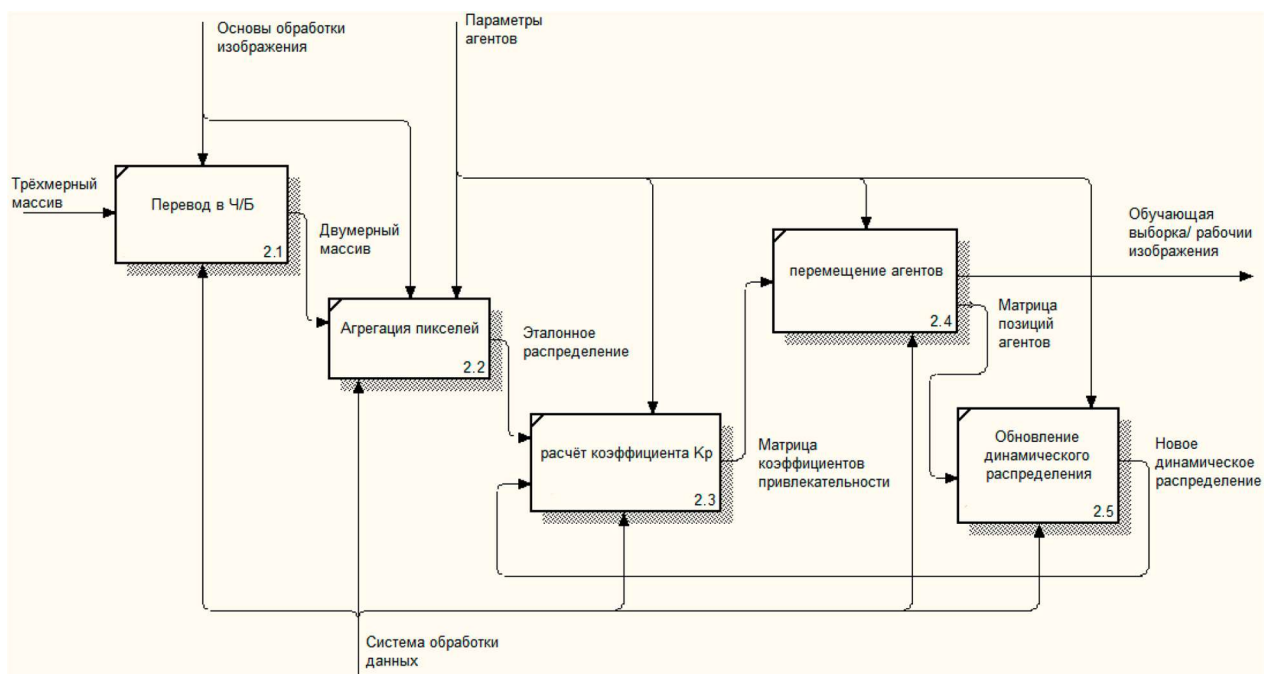


Рисунок 12 – IDEF0 модель «предобработка» ТО-ВЕ второго уровня

2.5 Иерархическая модель

На рисунке 13 представлен фрагмент онтологии оценки качества [7, 10] работы модели анализа внимания пользователей к рекламным изображениям на основе агентного подхода накопления контекста [8] и машинного обучения.

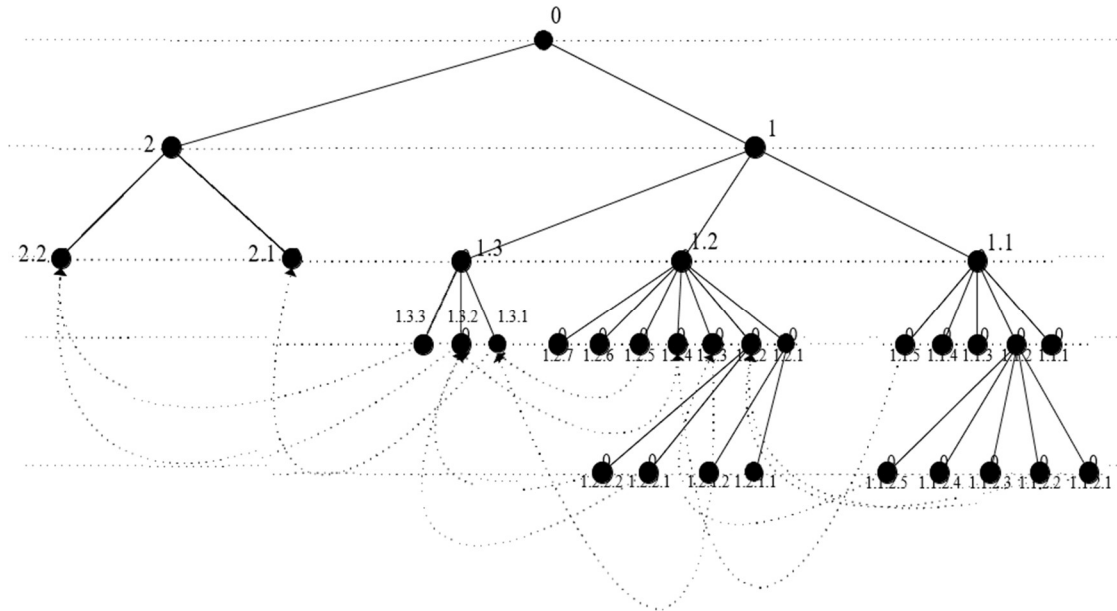


Рисунок 13 – Фрагмент онтологии

На рисунке используются следующие обозначения: 0 – Анализ внимания пользователей к рекламным изображениям; 1 – модель распознавания областей интереса [4]; 2 – качество оценки; 2.1 – воспроизводимость; 2.2 – точность предсказания (MSE, BCE) [14, 15]; 1.1 – структура; 1.2 – функции; 1.3 – результаты; 1.1.1 – загрузка данных; 1.1.2 – предобработка данных; 1.1.3 – извлечение признаков; 1.1.4 – применение модели машинного обучения [4]; 1.1.5 – визуализация внимания [5]; 1.1.2.1 – преобразование изображения в полутоновое [35]; 1.1.2.2 – разбиение на дискретную сетку ячеек; 1.1.2.3 – нормализация эталонного распределения яркости; 1.1.2.4 – агентное накопление контекста с предыдущих кадров [8]; 1.1.2.5 – восстановление структуры изображения; 1.2.1 – проверка корректности входных данных; 1.2.1.1 – получение кадра с экрана пользователя; 1.2.1.2 – получение кадра из видеофайла в формате MP4; 1.2.2 – нормализация; 1.2.2.1 – создание сетки; 1.2.2.2 – нормализация интенсивности пикселей; 1.2.3 – расчет коэффициента

привлекательности K_p ; 1.2.4 – прогнозирование карты внимания; 1.2.5 – построение цветовой тепловой карты [5]; 1.2.6 – наложение тепловой карты на исходное изображение; 1.2.7 – оценка и интерпретация результатов (MSE, BCE) [14, 15]; 1.3.1 – обработанный кадр с рекламным изображением; 1.3.2 – прогнозируемая тепловая карта внимания; 1.3.3 – предсказанная тепловая карта внимания, наложенная на исходный кадр.

2.6 Математическая модель

Идея – возможность восстановления визуально значимых областей видеопотока путём адаптивного перемещения агентов по дискретной сетке [8], построенной на основе яркости пикселей экрана, с сохранением памяти о предыдущих кадрах.

Дано: видеопоток с экрана (последовательность цветных изображений в формате RGB), размером $H \times W$ пикселей.

Требуется: синтезировать финальное распределение посещений агентов $\rho_{\text{final}}(i, j)$, которое создаёт контекст и накапливает значимые детали с учётом текущего кадра и истории предыдущих кадров[8].

Порядок проведения эксперимента. Были получены серии изображений экрана в реальном времени. Каждое изображение представлено множеством значений интенсивности пикселей. Из-за того, что прямая связь между яркостью пикселя и его долговременной значимостью отсутствует, для формирования адаптивного распределения агентов предлагается использовать итеративный статистический подход, опирающийся на балансировку между эталонным распределением (текущий кадр) и динамическим распределением (накопленная история посещений). При этом на изображении не выделяются отдельные анатомические структуры, а сетка равномерно покрывает всю область экрана.

Изображение экрана представляет собой множество значений интенсивности пикселя в цветовом пространстве RGB [10], которые можно представить формулой (24):

$$Img_{RGB} = (R(x, y), G(x, y), B(x, y)), \quad (24)$$

где Img_{RGB} – исходное цветное изображение;

$R(x, y)$, $G(x, y)$, $B(x, y)$ – интенсивности красного, зелёного и синего каналов в точке (x, y) соответственно.

Из-за того, что прямая связь между яркостью пикселя и его долговременной значимостью отсутствует, для формирования адаптивного распределения агентов предлагается использовать итеративный статистический подход, опирающийся на балансировку между эталонным распределением (текущий кадр) и динамическим распределением (накопленная история посещений). При этом на изображении не выделяются отдельные структуры, а сетка равномерно покрывает всю область экрана.

2.6.1 Определения и обозначения

Пространство (G) – дискретная двумерная сетка [10], представленная ячейками размером $c \times c$ пикселей. Задаётся формулой (25):

$$G = \{ (i, j) \mid i = 0, 1, \dots, M - 1, j = 0, 1, \dots, N - 1 \}, \quad (25)$$

где G – множество всех ячеек в симуляционном пространстве;

(i, j) – целочисленные координаты ячейки;

$M = \left\lfloor \frac{H}{c} \right\rfloor$, $N = \left\lfloor \frac{W}{c} \right\rfloor$ – размеры сетки по вертикали и горизонтали;

H, W – высота и ширина исходного изображения в пикселях.

Эталонное распределение – функция, задающая целевое значение каждой ячейки, получаемую из текущего кадра видеопотока.

Динамическое распределение – функция, отражающая количество «посещений» агентами каждой ячейки пространства к моменту времени t . $\rho_t(i, j)$ аппроксимирует накопленную значимость ячейки.

Агент (A_k) – k -й элементарный носитель, где $k \in \{1, \dots, N_{\text{agents}}\}$. N_{agents} – общее число агентов [8, 10]. Состояние агента в момент времени t описывается кортежем, представленным в формуле (26):

$$\sigma_k(t) = (p_k(t), D_k(t)), \quad (26)$$

где $\sigma_k(t)$ – состояние k -го агента в момент времени t ;

$p_k(t)$ – позиция (координаты ячейки) k -го агента в момент t ;

$D_k(t)$ – текущий порядок проверки направлений (множество доступных векторов смещения в определённой последовательности) для k -го агента в момент t .

Направления перемещения – множество векторов смещения $d = (d_i, d_j)$, доступных агенту для перемещения [8, 10]. Восемь возможных направлений задаются множеством (10.3):

$$D_{\text{default}} = \{(0,1), (0,-1), (1,0), (-1,0), (1,1), (1,-1), (-1,1), (-1,-1)\}. \quad (10.3)$$

2.6.2 Предобработка

Для перехода от RGB к полутоновому изображению используется взвешенное суммирование каналов с учётом физиологии человеческого зрения [10, 35]. Задаётся формулой (27):

$$I_{\text{gray}}(x, y) = 0.299 \cdot R(x, y) + 0.587 \cdot G(x, y) + 0.114 \cdot B(x, y). \quad (27)$$

Полученное полутоновое изображение разбивается на ячейки размером $c \times c$ пикселей. Для каждой ячейки с координатами (i, j) вычисляется среднее арифметическое яркостей всех пикселей [10], попадающих в её область (28):

$$\tilde{\rho}_{\text{raw}}(i, j) = \frac{1}{c^2} \sum_{x=i \cdot c}^{(i+1)c-1} \sum_{y=j \cdot c}^{(j+1)c-1} I_{\text{gray}}(x, y). \quad (28)$$

После этого полученная матрица нормализуется к диапазону $[0, 255]$ путём деления на максимальное значение и умножения на 255 [10] по формуле (29):

$$\tilde{\rho}(i, j)_{\text{norm}} = \frac{\tilde{\rho}_{\text{raw}}(i, j)}{\max_{p, q} \tilde{\rho}_{\text{raw}}(p, q)} \cdot 255. \quad (29)$$

В результате формируется матрица эталонного распределения $\tilde{\rho}$ размером $M \times N$, где $M = \left\lfloor \frac{H}{c} \right\rfloor$, $N = \left\lfloor \frac{W}{c} \right\rfloor$. Создаётся динамическое распределение ρ_t – матрица тех же размеров, изначально заполненная нулями. В ячейки сетки случайным образом размещается N_{agents} агентов [8, 10]. Каждому агенту сопоставляется индивидуальная случайная перестановка чисел от 0 до 7, задающая порядок проверки восьми возможных направлений перемещения (вверх, вниз, влево, вправо и четыре диагонали). Такая рандомизация предотвращает детерминированное поведение и способствует равномерному исследованию пространства.

2.6.3 Обновление динамического распределения

Процесс перемещения агентов направлен на максимизацию привлекательности целевых ячеек [8, 10]. Для всех ячеек сетки одновременно вычисляется коэффициент привлекательности $K_p(i, j)$ по формуле (30):

$$K_p(i, j) = \tilde{\rho}(i, j)_{\text{norm}} - \frac{S_{\tilde{\rho}}}{S_{\rho_t} + \varepsilon} \cdot \rho_t(i, j), \quad (30)$$

где $\tilde{\rho}(i, j)_{\text{norm}}$ – нормализованное эталонное распределение;

$S_{\tilde{\rho}}$ – полная сумма эталонного распределения;

S_{ρ_t} – полная сумма динамического распределения на шаге t ;

ε – малое положительное число, предотвращающее деление на ноль;

$\rho_t(i, j)$ – значение динамического распределения в ячейке (i, j) .

Полная сумма динамического распределения обновляется на каждом шаге по формуле (31):

$$S_{\rho_{t+1}} = S_{\rho_t} + N_{\text{agents}} \cdot \alpha, \quad (31)$$

где α – вес шага.

Полная сумма эталонного распределения вычисляется по формуле (32):

$$S_{\tilde{\rho}} = \sum_{i=0}^{M-1} \sum_{j=0}^{N-1} \tilde{\rho}(i, j)_{\text{norm}}. \quad (32)$$

После вычисления коэффициента привлекательности для каждого агента с использованием его индивидуальной перестановки $D_k(t)$ проверяются восемь возможных направлений. Агент выбирает направление, ведущее в ячейку (i', j') , для которой значение $K_p(i', j')$ максимально, при условии нахождения (i', j') в пределах сетки [8, 10]. Затем производится обновление динамического распределения по формуле (33):

$$\rho_{t+1}(i, j) = \rho_t(i, j) + \alpha \cdot \text{count}_{t+1}(i, j), \quad (33)$$

где $\text{count}_{t+1}(i, j)$ – количество агентов, выбравших ячейку (i, j) на текущем шаге $t+1$.

2.6.4 Метрики оценки качества

Для количественной оценки качества предсказания тепловых карт внимания разрабатываемой системой используются две взаимодополняющие

метрики: Mean Squared Error (MSE) и Binary Cross-Entropy (BCE) [14, 15]. Выбор данных метрик обусловлен спецификой задачи регрессии плотности внимания, где целевое значение для каждого пикселя тепловой карты [5] находится в диапазоне от 0 до 1. Пусть y_i – истинное значение интенсивности внимания в i -м пикселе (эталонная тепловая карта, полученная на основе данных айтрекинга), $\hat{y}_i$ – предсказанное значение модели в i -м пикселе, а N – общее количество пикселей в тепловой карте ($N = H \times W$, где H и W – высота и ширина изображения).

Mean Squared Error (MSE) – метрика, вычисляющая средний квадрат разности между истинными и предсказанными значениями [10, 14]. Математически MSE определяется следующей формулой (34):

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2, \quad (34)$$

MSE [14] чувствительна к крупным ошибкам за счёт возведения разности в квадрат, что делает её эффективной для обнаружения значительных отклонений предсказаний от эталонных значений. Для нормированных тепловых карт метрика принимает значения от 0 до 1, где $MSE = 0$ соответствует идеальному предсказанию. Основным недостатком MSE является то, что она одинаково штрафует ошибки во всех диапазонах значений, что не всегда согласуется с физиологией восприятия внимания, где ошибки в высокоинтенсивных областях (фиксациях взгляда) могут быть критичнее ошибок в фоновых зонах.

Binary Cross-Entropy (BCE) – метрика, заимствованная из задач бинарной классификации и адаптированная для оценки качества вероятностных предсказаний [10, 15]. Несмотря на то, что целевые значения являются непрерывными в диапазоне от 0 до 1, BCE интерпретирует каждый пиксель как независимую вероятность принадлежности к «области интереса». Математически BCE определяется формулой(35):

$$BCE = -\frac{1}{N} \sum_{i=1}^N [y_i \cdot \log(\hat{y}_i + \varepsilon) + (1 - y_i) \cdot \log(1 - \hat{y}_i + \varepsilon)], \quad (35)$$

где ε – малое положительное число ($\varepsilon = 10^{-7}$), вводимое для избежания логарифмирования нуля и обеспечения численной стабильности.

В отличие от MSE, BCE [14, 15] не ограничена сверху и может принимать значения от 0 до $+\infty$, причём значение $BCE = 0$ достигается только при идеальном совпадении $y_i = \hat{y}_i$ для всех пикселей. Чем хуже предсказание, тем больше значение BCE. Важным свойством BCE является то, что она более строго штрафует ошибки в областях с экстремальными значениями, то есть там, где истинные значения y_i близки к 0 или 1. Это соответствует физиологии визуального внимания: ошибка в области, где пользователь гарантированно смотрел ($y_i \approx 1$), является более критичной, чем ошибка в области с неопределённым вниманием ($y_i \approx 0.5$). Основным недостатком BCE является предположение о независимости пикселей, что не вполне корректно для тепловых карт с пространственной корреляцией, однако на практике это ограничение не приводит к существенному снижению качества оценки.

В рамках данной работы для обучения модели и оценки её качества используется комбинированный подход, объединяющий MSE и BCE [10, 14, 15]. Общая функция потерь L задаётся формулой(36):

$$L = \lambda \cdot MSE + (1 - \lambda) \cdot BCE, \quad (36)$$

где $\lambda \in [0,1]$ – весовой коэффициент, определяющий вклад каждой из метрик.

В исследовании [10] эмпирически установлено, что оптимальное значение составляет $\lambda = 0.3$. Данный выбор позволяет сочетать устойчивость MSE к малым ошибкам и чувствительность BCE к экстремальным значениям,

что обеспечивает наилучшее качество предсказания тепловых карт внимания. Результирующая оценка качества модели вычисляется как среднее значение комбинированной функции потерь на тестовой выборке, при этом отдельно фиксируются и анализируются значения MSE и BCE для диагностики характера ошибок модели.

Для количественной оценки близости динамического распределения ρ_t к эталонному распределению $\tilde{\rho}_{\text{norm}}$ используется нормированный коэффициент корреляции Пирсона [10, 44], преобразованный в проценты сходства. Оба распределения представляют собой двумерные матрицы размером $M \times N$, которые преобразуются в одномерные векторы длины $n = M \times N$. Коэффициент корреляции между векторами $\tilde{\rho}_{\text{vec}}$ и ρ_{vec} вычисляется по формуле (37):

$$r = \frac{\sum_{k=1}^n (\tilde{\rho}_k - \tilde{\bar{\rho}})(\rho_k - \bar{\rho})}{\sqrt{\sum_{k=1}^n (\tilde{\rho}_k - \tilde{\bar{\rho}})^2 \sum_{k=1}^n (\rho_k - \bar{\rho})^2}}, \quad (37)$$

где $\tilde{\bar{\rho}} = \frac{1}{n} \sum_{k=1}^n \tilde{\rho}_k$ – среднее значение элементов эталонного распределения

$\bar{\rho} = \frac{1}{n} \sum_{k=1}^n \rho_k$ – среднее значение элементов динамического распределения.

Значение r лежит в интервале $[-1, 1]$. Для перевода в проценты сходства используется преобразование (38):

$$S = \frac{r + 1}{2} \times 100\%. \quad (38)$$

Таким образом, $S = 100\%$ соответствует полной положительной корреляции, $S = 0\%$ – отсутствию корреляции, а отрицательные значения корреляции не учитываются в метрике сходства.

2.7 Результаты и выводы по второй главе

В ходе выполнения работы были достигнуты следующие результаты [7, 10]:

- Разработана общая схема предлагаемого решения, отражающая ключевые этапы и взаимодействие компонентов системы;
- Построена системно-структурная модель, определяющая архитектуру и взаимосвязи подсистем информационной системы;
- Представлена общая концептуальная модель, формализующая цели, функции и направленность системы;
- Создана базово-уровневая концептуальная модель, описывающая основные процессы и элементы анализа внимания;
- Разработана модификационная концептуальная модель, учитывающая особенности и улучшения предлагаемого решения;
- Построена функционально-структурная модель (IDEF0), визуализирующая потоки данных и функциональные блоки системы;
- Предложена алгоритмическая модель;
- Создана иерархическая модель, отражающая логические связи и уровни абстракции компонентов системы;
- Разработана математическая модель, формализующая процессы предобработки данных, извлечения признаков и прогнозирования областей интереса.

Полученные модели обеспечивают глубокое понимание структуры и функционирования будущей системы.

3 ПРОЕКТИРОВАНИЕ

3.1 Внешнее проектирование

На данном этапе проектирования было разработано техническое задание на разработку системы в соответствии с ГОСТ 34.602–89 [45]. Техническое задание приведено в приложении А.

3.2 Внутреннее проектирование

В ходе системно-структурного моделирования, приведенного в главе 2 [7, 10], были выделены следующие подсистемы разрабатываемой системы:

- подсистема сбора данных и формирования набора данных
- подсистема загрузки данных;
- подсистема предобработки;
- подсистема применения модели машинного обучения [4];
- подсистема визуализации внимания [5];
- интерфейс пользователя.

Подсистема сбора данных и формирования набора данных

При загрузке рекламного изображения для последующего прогнозирования областей интереса необходимо, чтобы данные айтрекинга [3] и изображение были пространственно согласованы между собой. Из-за возможного разного разрешения и смещения координат взгляда относительно экрана необходимо обеспечить, чтобы координаты фиксации взгляда, полученные от библиотеки eyeGestures [16], корректно отображались на пиксельную сетку рекламного изображения. Необходимо выполнить калибровку системы координат путем преобразования координат взгляда из экранных в координаты изображения с учётом масштабирования и положения окна просмотра. Для проверки корректности согласования использовать визуальный контроль наложения точек фиксации на исходное изображение. В случае невозможности получения заданного уровня соответствия (среднеквадратичное отклонение превышает 20 пикселей) необходимо

уведомить пользователя и прекратить работу подсистемы. На вход подсистема должна получать рекламное изображение и txt-файл с координатами взгляда (x, y), полученными от eyeGestures [16]. На выходе подсистема должна быть способна сохранять преобразованное полутоновое изображение [35] стандартного размера (224×224 пикселя), а также соответствующую ему тепловую карту внимания [5], построенную методом гауссова размытия ($\sigma = 25$ пикселей) [34].

Подсистема загрузки данных

Подсистема должна принимать на вход рекламные изображения в видеопотоке в формате MP4, а также обеспечивать захват текущего изображения с экрана пользователя. В разработанной системе захват экрана реализован с использованием библиотеки mss [46], которая позволяет получать скриншоты выбранного монитора с высоким быстродействием. Загружаемые файлы могут иметь произвольное разрешение; при необходимости подсистема должна преобразовывать входную информацию (изменять размер, конвертировать цветовые пространства) для дальнейшей работы других подсистем.

Подсистема предобработки данных

В разработанной системе предобработка данных для работы в потоковом режиме реализована в виде отдельной библиотеки на языке C++ [41] (agent_sim), которая выполняет следующие функции: обновление эталонного распределения на основе полученного изображения реализовано в функции update_reference, перемещение агентов по коэффициенту привлекательности K_p , а также вычисление динамического распределения, реализовано в функции multi_step_agents. Данная подсистема обеспечивает накопление контекста с предыдущих кадров видео и позволяет восстанавливать структуру изображения для последующего прогнозирования областей интереса.

Подсистема применения модели машинного обучения

Общая схема [7, 10] обучения и применения модели синтеза тепловых карт областей внимания пользователя приведена на рисунке 14.

Подсистема должна фиксировать визуальные признаки рекламного изображения в соответствии с рекомендациями по анализу привлекательности рекламных материалов [47]. Входными данными для подсистемы являются нормализованное чёрно-белое изображение размером 224×224 пикселя на выходе подсистемы предобработки. На основе этих данных подсистема с помощью обученной сверточной нейронной сети [22] с архитектурой энкодер-декодер и ASPP-модулем [27] должна предсказывать тепловую карту внимания [5], где значение каждого пикселя соответствует прогнозируемой интенсивности взгляда пользователя в диапазоне от 0 до 1.

В разработанной системе подсистема применения модели машинного обучения [4] реализована на языке Python [24] с использованием библиотеки TensorFlow [37]. Модель в формате TensorFlow SavedModel [48] загружается из папки models в фоновом потоке при запуске приложения, что позволяет не блокировать основной интерфейс. Для предсказания тепловой карты [5] выполняется следующая последовательность действий: изображение конвертируется из цветowego в чёрно-белое изображение, приводится к размеру 224×224 пикселя, затем выполняется нормализация значений пикселей путем деления на 255 для приведения к диапазону от 0 до 1. К полученному изображению применяется агентный модуль [8] для восстановления изображения с сохранением контекстных связей. Применяется модель машинного обучения [4]. Модель возвращает предсказанную тепловую карту [5], которая затем масштабируется к исходному разрешению кадра и визуализируется.

На вход в систему обучения приходит множество пар: рекламное изображение и соответствующая тепловая карта внимания [5]. При обучении модели выполняется оптимизация весов ASPP-модуля [27] сверточной нейронной сети [22] с использованием комбинированной функции потерь MSE и BCE [14, 15], описанной в разделе 2.6. Энкодер на базе VGG16 [29], предварительно обученный на наборе данных ImageNet, [26] остается замороженным и не обучается. Условием для прекращения обучения является

заданное количество эпох или достижение порога качества на валидационной выборке. В результате обученные веса модели сохраняются в формате TensorFlow SavedModel [48]. В папке models с хэш суммой модели содержатся файлы saved_model.pb и keras_metadata.pb, а также директория variables с файлами variables.data-00000-of-00001 и variables.index, которые используются для прогнозирования тепловых карт внимания [5] новых рекламных изображений. При прогнозировании тепловой карты внимания от одного нового рекламного изображения выполняется прямой проход через обученную сверточную нейронную сеть [22]. На выход из подсистемы поступают данные, необходимые для визуализации в виде двумерного массива значений интенсивности внимания.

Для проведения исследований с целью обучения модели и её последующего использования была выбрана серия изображений, а также собранные с помощью eyeGestures [16] данные айтрекинга 14 пользователей. После получения обученной модели становится возможным прогнозирование тепловых карт внимания [5] для новых изображений без необходимости в повторных айтрекинг-исследованиях. Точность полученных прогнозов зависит от объёма и качества размеченных данных на этапе обучения.

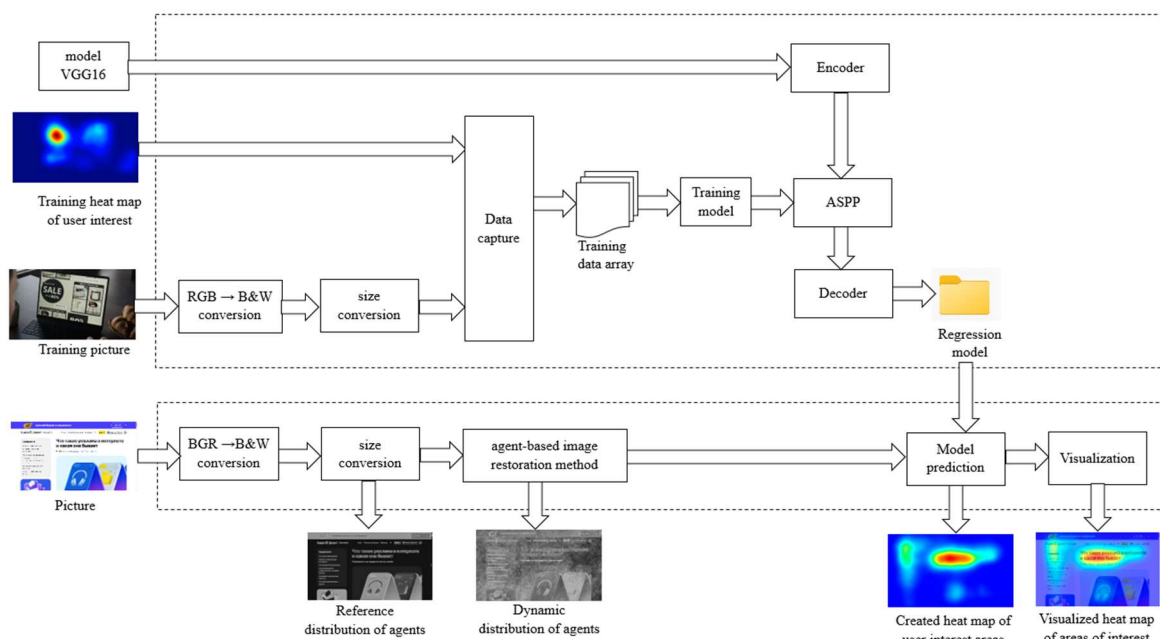


Рисунок 14 – Общая схема обучения и применения модели синтеза тепловых карт областей внимания пользователя

3.2.1 Выбор программной платформы и среды разработки

На рынке информационных технологий существует большое количество различных средств разработки приложений. Выбор среды разработки должен основываться на особенностях разрабатываемой системы [7, 10]. Предлагаемая система автоматического распознавания областей интереса на рекламных изображениях должна быть реализована в виде отдельного независимого модуля, выполняющего свои функции, и иметь возможность быть встроенной в уже существующие маркетинговые платформы. Основными требованиями к среде разработки относятся наличие встроенного интерактивного отладчика, удобство редактирования кода, наличие средств визуального программирования и наличие необходимых библиотек. В соответствии с описанными требованиями в качестве языка программирования для реализации системы был выбран Python 3.12.6 [24]. Для высокопроизводительной подсистемы предобработки данных использован язык C++ [41] с компиляцией в динамическую библиотеку `agent_sim.pyd`, что позволило достичь необходимой скорости обработки потока данных в реальном времени. Оптимальной средой разработки была выбрана PyCharm 3.0 [49], обеспечивающая удобную интеграцию Python и C++-кода и мощный отладчик.

Зависимости: `mss` [46] – библиотека для высокопроизводительного захвата экрана, применялась для получения скриншотов монитора в реальном времени; `pygame` [42] – библиотека для создания графического окна и отображения наложения поверх всех окон, применялась для реализации интерфейса пользователя; `OpenCV` [17] – библиотека компьютерного зрения, применялась для обработки изображений, применения цветовых карт и изменения размера изображений; `NumPy` [36] – библиотека для работы с многомерными массивами, применялась для хранения координат взгляда, матричных операций и работы с распределениями внимания; `scikit-learn` [39] – библиотека для расчета метрик качества модели; `TensorFlow/Keras` [37, 38] – библиотека глубокого обучения, применялась для загрузки обученной модели

и предсказания областей внимания человека; keyboard [50] – библиотека для глобального перехвата нажатий клавиш, применялась для управления режимами отображения, прозрачностью и настройками симуляции; threading [51] – библиотека для работы с потоками, применялась для асинхронной загрузки модели, фонового захвата экрана и удержания окна поверх всех; ctypes [52] – библиотека для вызова функций Windows API, применялась для управления прозрачностью окна, режимом click-through и удержания окна поверх всех; agent_sim – пользовательская C++ библиотека, применялась для выполнения высокопроизводительных расчётов перемещения агентов, обновления эталонного распределения и multi-step симуляции.

3.3 Результаты и выводы по третьей главе

В ходе выполнения работы были получены следующие результаты [10]:

- Создано техническое задание на разработку системы в соответствии с ГОСТ 34.602–89 [45] (приложение А);
- Описаны все подсистемы разрабатываемой системы: загрузки данных, предобработки данных, применения модели машинного обучения (CNN [22]), визуализации внимания [5] и интерфейса пользователя;
- Определена программная платформа: Python 3.12.6 [24] для основных модулей, C++ [41] для подсистемы предобработки;
- Выбрана среда разработки PyCharm 3 [24];
- Установлен перечень необходимых библиотек и зависимостей.

Выводы:

Проведённая работа на стадии проектирования позволяет перейти к следующему этапу разработки – инженерной реализации.

4 ИНЖЕНЕРНАЯ РЕАЛИЗАЦИЯ

По результатам проектирования, была выполнена инженерная реализация, разработан программный прототип предлагаемого решения, выполнено тестирование, получены и оценены результаты работы прототипа.

4.1 Сбор набора данных

В качестве данных для айтрекинга [3] был использован открытый набор данных Open Images v7 от Google LLC [26], содержащий около 9 миллионов разнообразных фотографий. На рисунке 15 приведен код для скачивания из набора данных 11000 изображений на локальное устройство. Примеры скачанных изображений представлены на рисунке 16.

```
os.makedirs( name: "dataset", exist_ok=True)
dataset = load_dataset("bitmind/open-images-v7", split="train", streaming=True)
examples = list(dataset.take(11000))
random.shuffle(examples)

for i, example in enumerate(examples[:11000]):
    img = Image.open(BytesIO(requests.get(example["url"], timeout=10).content))
    img.save(f"dataset/image_{i:04d}.jpg")
```

Рисунок 15 – Скачивание набора данных

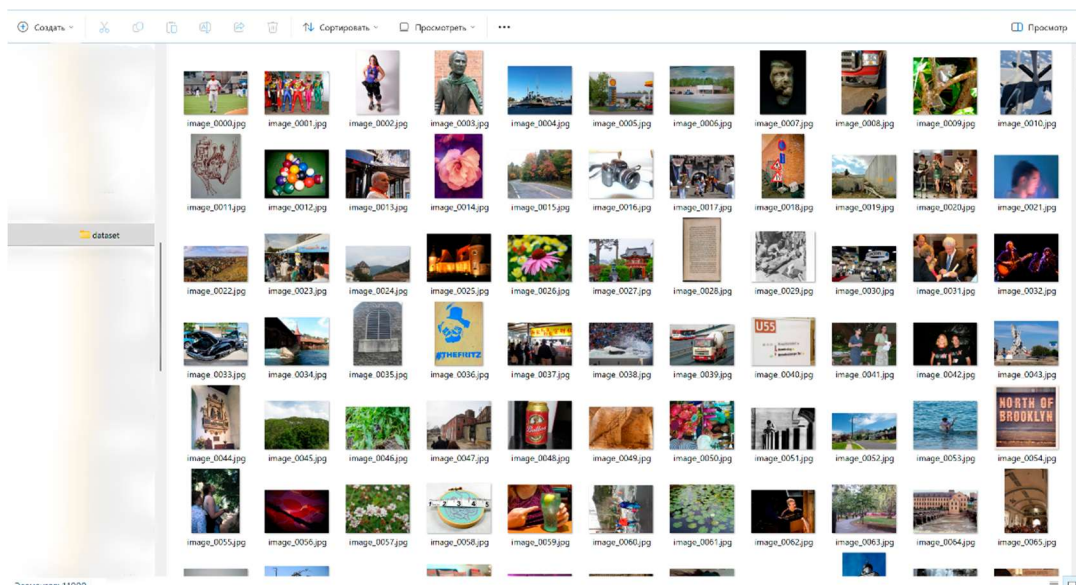


Рисунок 16 – Примеры изображений для айтрекинга

Была реализована калибровка системы айтрекинга [3]. Код калибровки представлен на рисунке 17. Для этого на экран равномерно выводятся 25 калибровочных точек (см. рисунок 18). Пользователь последовательно фиксирует взгляд на каждой точке, а система с помощью библиотеки EyeGestures_v3 [16] запоминает соответствие между положением зрачка и координатами на экране.

```
# Функция для создания равномерной сетки точек для калибровки
def calibration_points(width, height, n_points=25):
    grid_size = int(np.sqrt(n_points))
    x_ratio = np.linspace(0.1, 0.9, grid_size)
    y_ratio = np.linspace(0.1, 0.9, grid_size)
    xx, yy = np.meshgrid(x_ratio, y_ratio)
    calibration_points = []
    for i in range(grid_size):
        for j in range(grid_size):
            x_pos = int(x_ratio[j] * width)
            y_pos = int(y_ratio[i] * height)
            point = clamp_point(point=[x_pos, y_pos], width, height)
            calibration_points.append(point)
    return np.array(calibration_points)

# Функция для ограничения точек
def clamp_point(point, width, height):
    x = np.clip(point[0], 0, width - 1)
    y = np.clip(point[1], 0, height - 1)
    return [int(x), int(y)]

# Инициализация калибровочных точек
calibration_points = calibration_points(screen_width, screen_height, n_points=25)
n_points = len(calibration_points)

# Создание карты калибровки в относительных координатах
calibration_map_relative = calibration_points / [screen_width, screen_height]
# Загрузка калибровочной карты в систему жестов
gestures.uploadCalibrationMap(calibration_map_relative, context="my_context")
gestures.setFixation(1.0)
```

Рисунок 17 – Калибровка айтрекера

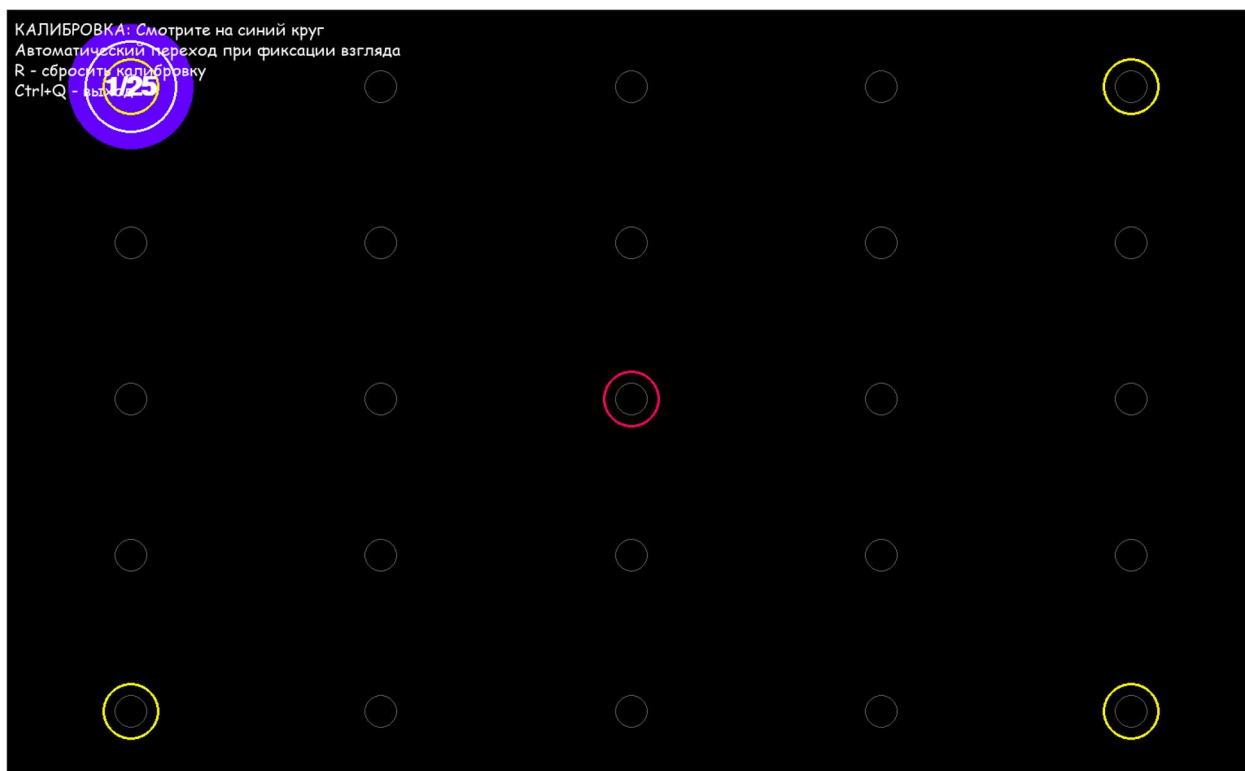


Рисунок 18 –Интерфейс калибровки айтрекера

После успешной калибровки система начинает регистрировать координаты взгляда пользователя. Код получения координат взгляда представлен на рисунке 19. Для корректной обработки тестовые изображения предварительно приводятся к единому разрешению: изображение вписывается в квадрат, а пустые области заполняются черным цветом (рисунок 20).

```
def get_gaze_coordinates():
    ret, frame = cap.read()
    if not ret:
        return None
    # Получение координат взгляда
    event, _ = gestures.step(np.flip(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB), axis=1),
                             False, screen_width, screen_height, "my_context")
    if event and event.point:
        # Ограничиваем координаты в пределах экрана
        return (int(np.clip(event.point[0], 0, screen_width - 1)),
                int(np.clip(event.point[1], 0, screen_height - 1)))
    return None
```

Рисунок 19 – Айтрекинг



Рисунок 20 – Интерфейс айтрекера

На рисунке 21 показан результат работы айтрекера – поверх тестового изображения отображаются полученные координаты взгляда пользователя [3].



Рисунок 21 – Координаты взгляда поверх тестового изображения

Была реализована функция построения тепловой карты визуального внимания [5] на основе собранных координат взгляда. Для каждой точки создается ядро Гаусса [34] с заданным радиусом влияния. Пример построенной тепловой карты представлен на рисунке 22.

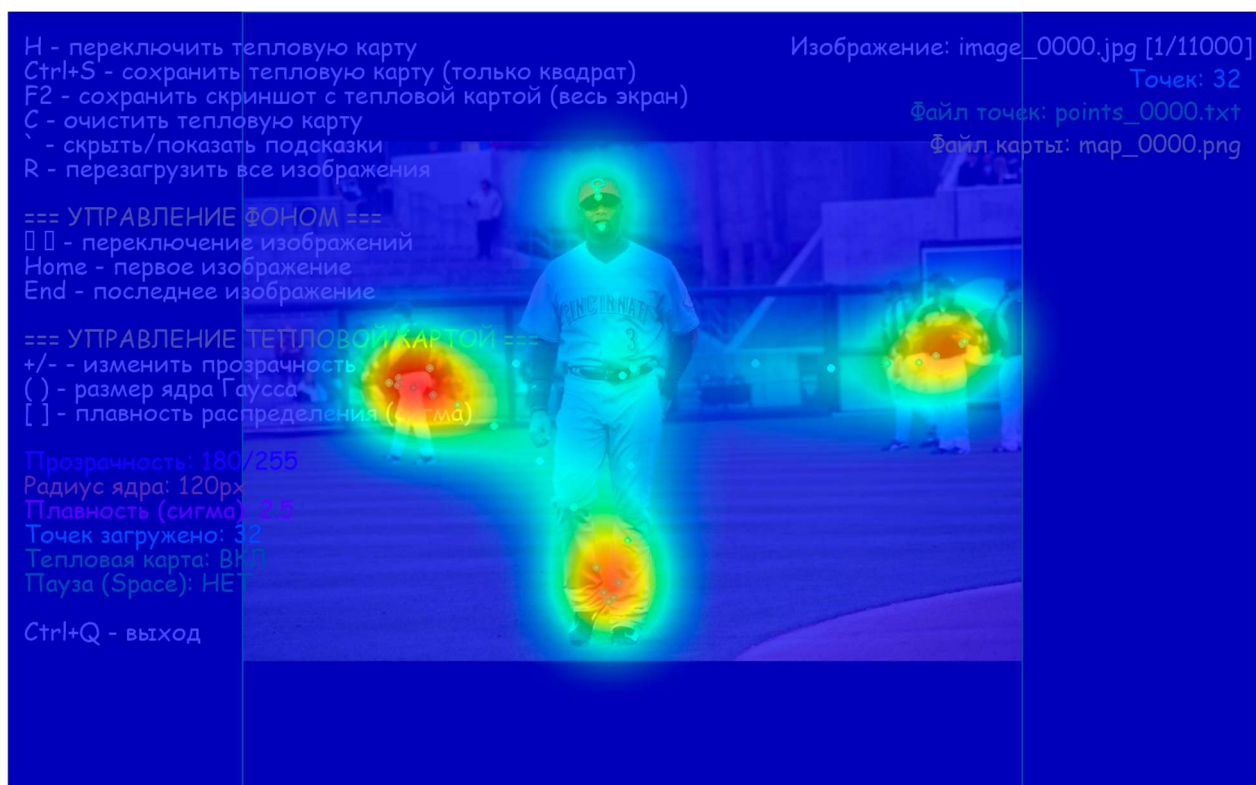


Рисунок 22 – Тепловая карта распределения визуального внимания

Данным способом был собран набор данных, состоящий из 11000 элементов. Разметка выполнялась 12 людьми для получения субъективного результата. Все скачанные изображения были приведены к черно-белому формату по формуле (27) из пункта 2.6.2, что позволило позже применить агентный метод [8] восстановления изображения с сохранением контекста предыдущих кадров. Пример преобразованных изображений в папке набора данных представлен на рисунке 23. Для каждого изображения были сохранены координаты взгляда пользователя в отдельные текстовые файлы, где координаты разделяются между собой точкой с запятой, а значения X и Y внутри пары – запятой (рисунок 24). Для каждого тестового изображения были сохранены тепловые карты в разрешении 224 на 224 пикселя (рисунок 25).

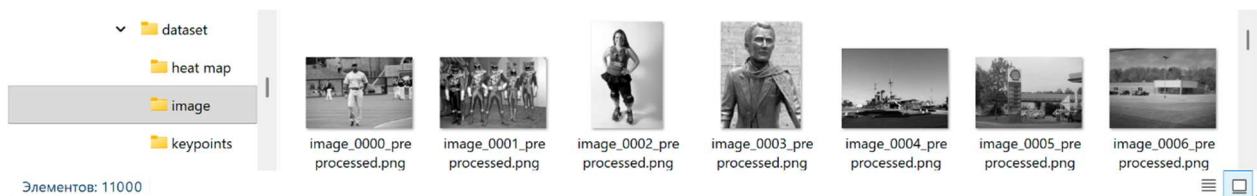


Рисунок 23 – Папка с изображениями

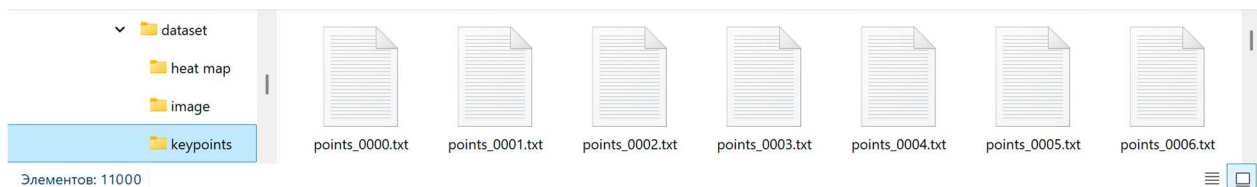


Рисунок 24 – Папка с текстовыми файлами координат взгляда

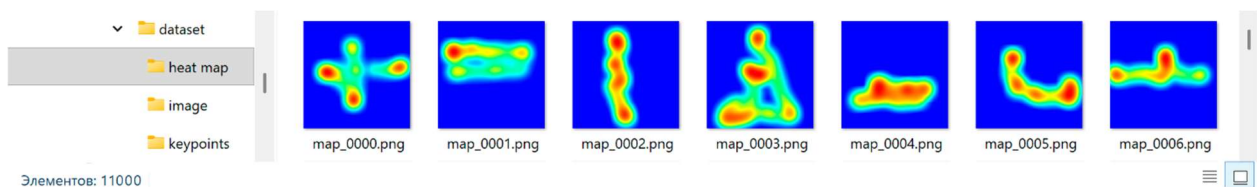


Рисунок 25 – Папка с сохраненными тепловыми картами

4.2 Обучение модели

Для решения задачи предсказания областей визуального внимания пользователя на изображении была выбрана архитектура CNN [22]. В качестве энкодера используются сверточные слои предобученной модели VGG16 [22, 29] для извлечения иерархических признаков из входного изображения. Полученные с энкодера признаки передаются в ASPP-модуль [27], который агрегирует контекстную информацию на различных масштабах за счет использования сверток [22] с разными размерами ядер (1, 3, 5 и 7 пикселей). Это позволяет модели учитывать как локальные детали, так и глобальный контекст изображения. Далее применяется декодер, который последовательно увеличивает пространственное разрешение карт признаков с использованием апсемплинга и объединения со skip-соединениями [22] от соответствующих слоев энкодера. Такая архитектура обеспечивает восстановление выходной карты значимости в исходном разрешении 224×224 пикселя. Код реализации архитектуры представлен на рисунке 26.


```

def build_model(input_shape=(224, 224, 3)):
    inputs = layers.Input(shape=input_shape, name='input_image')
    # Encoder: VGG16
    vgg = VGG16(include_top=False, weights='imagenet', input_tensor=inputs)
    vgg.trainable = False

    # 5 skip connections
    skip1 = vgg.get_layer('block1_conv2').output # 224x224x64
    skip2 = vgg.get_layer('block2_conv2').output # 112x112x128
    skip3 = vgg.get_layer('block3_conv3').output # 56x56x256
    skip4 = vgg.get_layer('block4_conv3').output # 28x28x512
    skip5 = vgg.get_layer('block5_conv3').output # 14x14x512 (bottleneck)

    # ASPP (на основе bottleneck)
    aspp1 = layers.Conv2D(256, 1, padding='same', activation='relu')(skip5)
    aspp2 = layers.Conv2D(256, 3, padding='same', activation='relu')(skip5)
    aspp3 = layers.Conv2D(256, 5, padding='same', activation='relu')(skip5)
    aspp4 = layers.Conv2D(256, 7, padding='same', activation='relu')(skip5)

    gap = layers.GlobalAveragePooling2D()(skip5)
    gap = layers.Reshape((1, 1, -1))(gap)
    gap = layers.Conv2D(256, 1, activation='relu')(gap)
    gap = layers.UpSampling2D(size=(14, 14), interpolation='bilinear')(gap)

    aspp_concat = layers.Concatenate()([aspp1, aspp2, aspp3, aspp4, gap])
    aspp_out = layers.Conv2D(256, 1, activation='relu')(aspp_concat)
    aspp_out = layers.Dropout(Config.DROPOUT_RATE)(aspp_out)

    # Decoder
    # Block 1: 14x14 -> 28x28 (co skip5)
    up1 = layers.UpSampling2D(size=(2, 2), interpolation='bilinear')(aspp_out)
    concat1 = layers.Concatenate()([up1, skip5])
    conv1 = layers.Conv2D(256, 3, padding='same', activation='relu')(concat1)
    conv1 = layers.BatchNormalization()(conv1)

    # Block 2: 28x28 -> 56x56 (co skip4)
    up2 = layers.UpSampling2D(size=(2, 2), interpolation='bilinear')(conv1)
    concat2 = layers.Concatenate()([up2, skip4])
    conv2 = layers.Conv2D(128, 3, padding='same', activation='relu')(concat2)
    conv2 = layers.BatchNormalization()(conv2)

    # Block 3: 56x56 -> 112x112 (co skip3)
    up3 = layers.UpSampling2D(size=(2, 2), interpolation='bilinear')(conv2)
    concat3 = layers.Concatenate()([up3, skip3])
    conv3 = layers.Conv2D(64, 3, padding='same', activation='relu')(concat3)
    conv3 = layers.BatchNormalization()(conv3)

    # Block 4: 112x112 -> 224x224 (co skip2)
    up4 = layers.UpSampling2D(size=(2, 2), interpolation='bilinear')(conv3)
    concat4 = layers.Concatenate()([up4, skip2])
    conv4 = layers.Conv2D(32, 3, padding='same', activation='relu')(concat4)
    conv4 = layers.BatchNormalization()(conv4)

    # Block 5: финальный выход (co skip1)
    up5 = layers.UpSampling2D(size=(2, 2), interpolation='bilinear')(conv4)
    concat5 = layers.Concatenate()([up5, skip1])
    conv5 = layers.Conv2D(16, 3, padding='same', activation='relu')(concat5)
    conv5 = layers.BatchNormalization()(conv5)

    output = layers.Conv2D(1, 1, padding='same', activation='sigmoid', name='saliency')(conv5)

    return Model(inputs, output, name='CNN')

```

Рисунок 26 – Реализация архитектуры модели

Обучение модели выполнялось на собранном наборе данных [26] из 11000 изображений с соответствующими тепловыми картами. Набор данных был разделен на обучающую (80%) и валидационную (20%) выборки. В качестве функции потерь использовалась комбинация среднеквадратичной ошибки (MSE) и бинарной кросс-энтропии (BCE) [4, 10], что позволяет модели учитывать как общую форму распределения внимания, так и точность значений пикселей. Оптимизация весов производилась с помощью алгоритма Adam [4, 53]. Этот алгоритм адаптивно подбирает скорость обучения для каждого параметра модели, что позволяет ускорить сходимость и достичь более высокой точности. Начальная скорость обучения была установлена равной 0,0001. Код обучения модели представлен на рисунке 27.

```
def combined_loss(y_true, y_pred): 1 usage
    return (tf.keras.losses.MeanSquaredError()(y_true, y_pred) +
            tf.keras.losses.BinaryCrossentropy()(y_true, y_pred))

model = build_model()
model.compile(
    optimizer=Adam(Config.LEARNING_RATE),
    loss=combined_loss,
    metrics=[correlation_score, 'mae'])

history = model.fit(
    train,
    validation_data=valdat,
    epochs=Config.EPOCHS,
    callbacks=[
        ModelCheckpoint(
            filepath='best_model.keras',
            monitor='validat_correlation_score',
            mode='max',
            save_best_only=True),
        EarlyStopping(monitor='validat_loss', patience=15),
        ReduceLRonPlateau(monitor='validat_loss', factor=0.5, patience=5)])

tf.saved_model.save(model, export_dir="model")
```

Рисунок 27 – Обучения модели

Для предотвращения переобучения применялся механизм ранней остановки (EarlyStopping) [4]: если в течение 15 эпох качество модели на валидационной выборке не улучшалось, обучение прекращалось досрочно. Также использовалось снижение скорости обучения при плато (ReduceLROnPlateau) [4]: если в течение 5 эпох ошибка не уменьшалась, скорость обучения снижалась вдвое, что позволяло точнее настраивать веса модели. После завершения обучения модель была сохранена в формате TensorFlow SavedModel [48]. Данный формат сохраняет полную информацию о модели: архитектуру, обученные веса и вычислительный граф. Как показано на рисунке 28, сохраненная модель представляет собой директорию, содержащую файл `saved_model.pb` с описанием графа вычислений, папку `variables` с бинарными файлами весов (`variables.data-00000-of-00001` и `variables.index`), а также служебные файлы `fingerprint.pb` и `assets`. Такая структура позволяет загружать модель для последующего использования без необходимости повторного определения архитектуры.

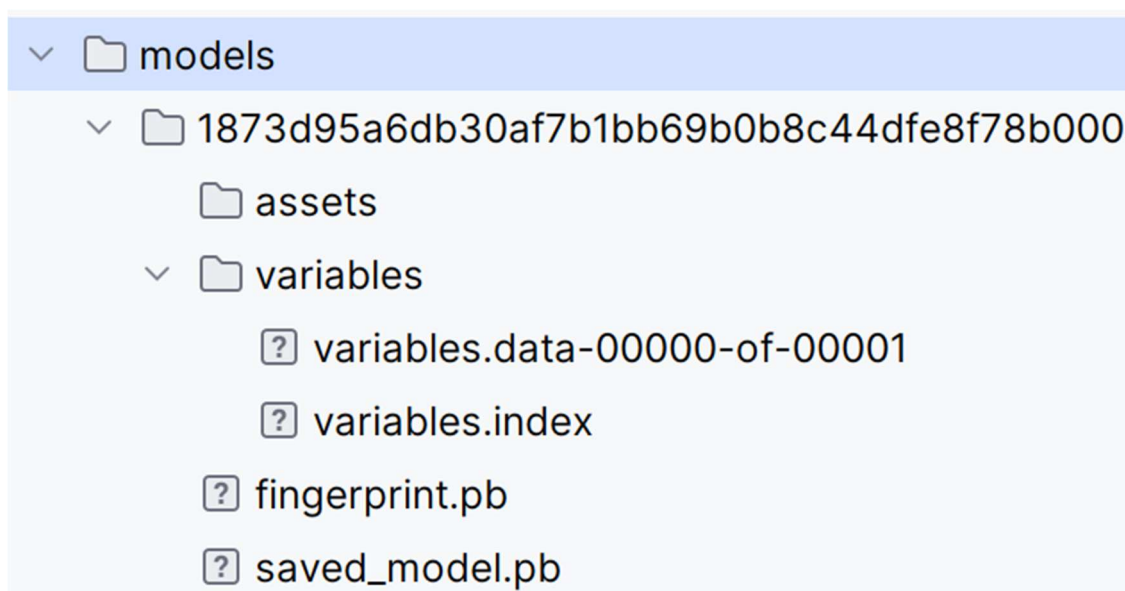


Рисунок 28 – Сохраненная модель машинного обучения

4.3 Агентный подход

Для создания контекста между последовательными кадрами видеопотока и повышения качества последующего прогнозирования областей

интереса был использован агентный подход [8] к восстановлению структуры изображения с сохранением информации о предыдущих кадрах. Данный подход способствует формированию более точной и плавно адаптируемой карты областей интереса.

Для обеспечения высокопроизводительных вычислений предобработка изображений агентным подходом была реализована в виде специализированной библиотеки на языке C++ [41], скомпилированной в динамический модуль и интегрированной с Python через интерфейс pybind11 [54].

Весь код реализованной библиотеки agent_sim представлен в приложении Б.

Такой подход позволил перенести все ресурсоёмкие операции на уровень скомпилированного кода, исключив накладные расходы интерпретатора и задействовав низкоуровневые оптимизации компилятора. На рисунке 29 представлен листинг заголовочной части библиотеки с подключением необходимых зависимостей и объявлением глобальных структур.

```
#include <pybind11/pybind11.h> // Связывание C++ с Python
#include <pybind11/numpy.h>    // Работа с NumPy-массивами
#include <cstdint>              // Использование uint8_t
#include <algorithm>            // Стандартные алгоритмы
#include <vector>               // динамический массив
#include <cmath>                // Математические функции
#include <mutex>                // для многопоточной работы
#include <stdexcept>           // Исключения (для обработки ошибок)

namespace py = pybind11; // Создание псевдонима для удобства
```

Рисунок 29 – Заголовочная часть

К числу операций, вынесенных в C++-библиотеку, относятся: преобразование захваченного изображения в оттенки серого [35]; разбиение на ячейки и усреднение яркости с последующей нормализацией эталонного распределения; вычисление матрицы коэффициентов привлекательности K_p для всех клеток сетки; выполнение многократных шагов перемещения агентов [8]; нормализация динамического распределения. Ниже подробно рассмотрена каждая из этих операций.

Первым этапом обработки является перевод RGB-изображения, захваченного с экрана, в полутоновое [35]. Преобразование выполняется в соответствии с формулой (27) взвешенного суммирования трёх цветовых каналов, коэффициенты которой подобраны с учётом спектральной чувствительности человеческого зрения. Как показано на рисунке 30, для каждого пикселя из буфера `img_ptr` извлекаются значения красного, зелёного и синего каналов, после чего вычисляется итоговая яркость `gray`. Доступ к пикселю с координатами (y, x) осуществляется по формуле `img_ptr + (y * W + x) * 3`, где W – ширина изображения в пикселях, а умножение на 3 обусловлено тем, что каждый пиксель в формате RGB представлен тремя последовательными байтами. Данная формула позволяет эффективно вычислять смещение в линейном буфере без использования многомерных массивов.

```
uint8_t* pixel = img_ptr + (y * W + x) * 3; // указатель на текущий пиксель
float gray = 0.299f * pixel[0] +           // взвешенный вклад красного канала
             0.587f * pixel[1] +           // взвешенный вклад зелёного канала
             0.114f * pixel[2];            // взвешенный вклад синего канала
```

Рисунок 30 – Преобразование RGB-пикселя в оттенки серого

Данная операция интегрирована непосредственно в цикл усреднения по ячейкам, что позволяет избежать дополнительного прохода по изображению. Размер ячейки задаётся параметром `cell_size` равным 10 пикселям. Для каждой ячейки с координатами (i, j) вычисляется среднее арифметическое яркостей

всех входящих в неё пикселей. Как показано на рисунке 31, внешние циклы перебирают все ячейки выходной сетки размером $out_h \times out_w$, а внутренние циклы по dy и dx осуществляют обход пикселей внутри текущей ячейки. Внутри внутренних циклов производится проверка границ ($y < H$, $x < W$) для корректной обработки краевых ячеек. Накопленная сумма (sum) после завершения внутренних циклов делится на $cell_size * cell_size$, что даёт среднее значение яркости для ячейки. Для ускорения вычислений применена директива `#pragma omp parallel for collapse(2)` [41], которая распараллеливает обработку ячеек по всем доступным ядрам процессора. Число 2 в директиве означает объединение двух уровней вложенных циклов (по i и j), что позволяет рассматривать все итерации как единое пространство задач и обеспечивает их равномерное распределение между потоками, предотвращая простои вычислительных ядер при малом количестве строк сетки out_h .

```
#pragma omp parallel for collapse(2)           // параллельное выполнение по ячейкам
for (int i = 0; i < out_h; ++i) {
    for (int j = 0; j < out_w; ++j) {
        float sum = 0.0f;                     // сумма яркостей пикселей в ячейке
        for (int dy = 0; dy < cell_size; ++dy) {
            for (int dx = 0; dx < cell_size; ++dx) {
                int y = i * cell_size + dy;    // координата пикселя по вертикали
                int x = j * cell_size + dx;    // координата пикселя по горизонтали
                if (y < H && x < W) {           // проверка выхода за границы
                    uint8_t* pixel = img_ptr + (y * W + x) * 3;
                    float gray = 0.299f * pixel[0] + 0.587f * pixel[1] + 0.114f * pixel[2];
                    sum += gray;                // накопление суммы яркостей
                }
            }
        }
        cell_means[i * out_w + j] = sum / (cell_size * cell_size); // усреднение по ячейкам
    }
}
```

Рисунок 31 – Вычисление средних значений яркости по ячейкам

Матрица `cell_means`, полученная после усреднения, содержит значения яркости. Как показано на рисунке 32, для приведения этих значений к стандартному диапазону $[0, 255]$ выполняется нормализация. Сначала с помощью функции `std::max_element` [41] определяется максимальное значение

max_val среди всех элементов cell_means. Затем каждый элемент матрицы последовательно делится на max_val и умножается на 255. Результат сохраняется в массив new_ref, который в дальнейшем используется как эталонное распределение $\tilde{\rho}$ [8]. Такая нормализация обеспечивает инвариантность алгоритма к общим изменениям яркости и контраста.

```
float max_val = *std::max_element(cell_means.begin(), cell_means.end()); // поиск максимума
for (int idx = 0; idx < out_h * out_w; ++idx) {
    new_ref[idx] = (cell_means[idx] / max_val) * 255.0f; // линейное масштабирование
}
```

Рисунок 32 – Нормализация эталонного распределения

На каждом шаге симуляции для всех ячеек сетки рассчитывается коэффициент привлекательности K_p , определяющий степень интереса агентов к каждой ячейке [8]. Вычисление коэффициента включает два этапа. На этапе инициализации после обновления кадра однократно вычисляется сумма эталонного распределения total_reference путём суммирования всех элементов current_ref, как показано на рисунке 33. На каждом шаге симуляции аналогично вычисляется сумма динамического распределения total_dynamic по массиву dyn_out_ptr, как показано на рисунке 34. Нормировочный коэффициент coeff рассчитывается как отношение суммы значений эталонного распределения (total_reference) к сумме значений динамического распределения (total_dynamic) с добавлением малой величины 1e-10f для предотвращения деления на ноль. Непосредственно для каждой целевой ячейки с линейным индексом target_linear коэффициент K_p вычисляется по формуле: значение эталонного распределения минус произведение нормировочного коэффициента на значение динамического распределения.

```
float total_reference = 0.0f;
for (int i = 0; i < rows * cols; ++i) {
    total_reference += current_ref[i]; // однократное суммирование после обновления кадра
}
```

Рисунок 33 – Вычисление суммы эталонного распределения

```

float total_dynamic = 0.0f;
for (int i = 0; i < rows * cols; ++i) {
    total_dynamic += dyn_out_ptr[i]; // суммирование на каждом шаге
}
float coeff = total_reference / (total_dynamic + 1e-10f); // нормировочный коэффициент

// Расчёт коэффициента привлекательности для целевой ячейки
float Kp = ref_ptr[target_linear] - coeff * dyn_ptr[target_linear]; // баланс эталона и истории

```

Рисунок 34 – Вычисление суммы динамического распределения

Ядром алгоритма является функция `single_step`, реализующая однократное перемещение всех агентов в симуляционном пространстве (см. рисунок 35). Для каждого агента [8], характеризуемого текущей позицией с координатами (r , c), выполняется последовательный перебор восьми возможных направлений перемещения. Порядок перебора направлений определяется индивидуальной последовательностью `seq_ptr`, которая генерируется для каждого агента случайным образом на этапе инициализации, что предотвращает детерминированное поведение и способствует равномерному исследованию пространства. Для каждого потенциального направления в первую очередь проверяется, не выходит ли целевая ячейка за границы дискретной сетки. Если целевая ячейка находится в допустимых пределах, для неё рассчитывается коэффициент привлекательности K_p , значение которого определяется на основе эталонного и динамического распределений. После вычисления K_p для всех допустимых направлений агент выбирает то направление, которое обеспечивает максимальное значение коэффициента привлекательности. В случае, если несколько направлений имеют одинаковое максимальное значение, агент выбирает первое из них в соответствии с порядком перебора. Для существенного ускорения вычислительного процесса цикл, осуществляющий перебор и перемещение всех агентов, выполняется параллельно с использованием директивы `#pragma omp parallel for`, что позволяет эффективно задействовать все доступные ядра многопроцессорной системы.

```

void single_step(
    int* pos_ptr,           // массив позиций агентов (N, 2)
    float* ref_ptr,        // эталонное распределение (rows, cols)
    float* dyn_ptr,        // динамическое распределение (rows, cols)
    int* seq_ptr,          // порядок направлений для текущего агента (8)
    float coeff,           // нормировочный коэффициент K_p
    int rows, int cols,    // размеры сетки
    int N,                 // количество агентов
    int dynamic_step       // вес добавления к динамическому распределению
) {
    // Массив направлений перемещения (8 соседних клеток)
    const int directions[8][2] = {
        {0, 1}, {1, 0}, {0, -1}, {-1, 0}, // вертикальные и горизонтальные
        {1, 1}, {1, -1}, {-1, 1}, {-1, -1} // диагональные
    };

    // Параллельное выполнение цикла по всем агентам
    #pragma omp parallel for
    for (int idx = 0; idx < N; ++idx) {
        int r = pos_ptr[idx * 2];           // текущая строка агента
        int c = pos_ptr[idx * 2 + 1];       // текущий столбец агента

        int best_r = r;                     // наилучшая строка (по умолчанию текущая)
        int best_c = c;                     // наилучший столбец (по умолчанию текущий)
        float best_Kp = -1e9f;              // максимальное значение K_p

        // Перебор направлений в порядке, заданном seq_ptr
        for (int d = 0; d < 8; ++d) {
            int dir = seq_ptr[d];           // индекс направления из последовательности
            int nr = r + directions[dir][0]; // координата строки целевой ячейки
            int nc = c + directions[dir][1]; // координата столбца целевой ячейки

            // Проверка выхода за границы сетки
            if (nr >= 0 && nr < rows && nc >= 0 && nc < cols) {
                int target = nr * cols + nc; // линейный индекс целевой ячейки

                // Расчёт коэффициента привлекательности целевой ячейки
                float Kp = ref_ptr[target] - coeff * dyn_ptr[target];

                if (Kp > best_Kp) {           // выбор направления с максимальным K_p
                    best_Kp = Kp;
                    best_r = nr;
                    best_c = nc;
                }
            }
        }

        // Обновление позиции агента
        pos_ptr[idx * 2] = best_r;
        pos_ptr[idx * 2 + 1] = best_c;
    }
}

```

Рисунок 35 – Функция выполнения одного шага перемещения всех агентов

Для минимизации накладных расходов на межъязыковые вызовы между Python [24] и C++ [41] была разработана функция `multi_step_agents`, выполняющая 200 шагов (steps) симуляции за один вызов из Python.

```
void multi_step_agents(
    int steps,                      // количество шагов (steps = 200)
    int dynamic_step,              // вес шага
) {
    // Получение указателей на входные массивы
    int* pos_ptr = static_cast<int*>(buf_pos.ptr);
    float* dyn_in_ptr = static_cast<float*>(buf_dyn_in.ptr);
    float* dyn_out_ptr = static_cast<float*>(buf_dyn_out.ptr);
    float* ref_out_ptr = static_cast<float*>(buf_ref_out.ptr);

    // Копирование входного динамического распределения в выходной массив
    for (int i = 0; i < rows * cols; ++i) {
        dyn_out_ptr[i] = dyn_in_ptr[i];
    }

    // Основной цикл симуляции
    for (int step = 0; step < steps; ++step) {
        // Вычисление суммы динамического распределения
        float total_dynamic = 0.0f;
        for (int i = 0; i < rows * cols; ++i) {
            total_dynamic += dyn_out_ptr[i];
        }

        // Расчёт нормировочного коэффициента
        float coeff = total_reference / (total_dynamic + 1e-10f);

        // Выполнение одного шага перемещения агентов
        single_step(pos_ptr, current_ref.data(), dyn_out_ptr, seq_ptr,
                    allow_multiple, coeff, rows, cols, N, dynamic_step);
    }

    // Копирование эталонного распределения в выходной массив
    for (int i = 0; i < rows * cols; ++i) {
        ref_out_ptr[i] = current_ref[i];
    }
}
```

Рисунок 36 – Функция выполнения нескольких шагов симуляции за один

ВЫЗОВ

Как показано на рисунке 36, внутри функции организован цикл, в котором на каждой итерации вычисляется актуальное значение суммы динамического распределения и коэффициент `coeff`, после чего вызывается функция `single_step`, осуществляющая перемещение всех агентов. Параметр `dynamic_step` определяет вес шага при посещении ячейки агентом. Передача массивов `positions`, `dynamic_out` и `reference_out` осуществляется по ссылке, что исключает создание промежуточных копий.

После выполнения всех шагов симуляции выполняется нормализация динамического распределения для его передачи в модель машинного обучения. Как показано на рисунке 37, сначала находится максимальное значение `max_val` среди всех ячеек массива `dyn_out_ptr`, после чего каждый элемент делится на этот максимум и умножается на 255. В результате динамическое распределение приводится к диапазону [0,255].

```
float max_val = *std::max_element(cell_means.begin(), cell_means.end());
for (int idx = 0; idx < out_h * out_w; ++idx) {
    new_ref[idx] = (cell_means[idx] / max_val) * 255.0f;
}
```

Рисунок 37 – Нормализация динамического распределения

В результате реализованная библиотека на C++ [41] `agent_sim` возвращает в Python [24] два массива: динамическое распределение (`dynamic_out`) и эталонное распределение (`reference_out`) [8]. Передача массивов по ссылке позволяет обращаться к данным в памяти без создания дополнительных копий [41].

Перенос ресурсоёмкой предобработки изображений агентным методом на уровень скомпилированного кода позволил существенно оптимизировать вычисления. Для количественной оценки достигнутого ускорения было проведено сравнение исходной (Python) и оптимизированной (Python, C++) реализаций [24, 41]. Сравнение выполнялось в идентичных условиях

(процессор 8 ядер, 16 потоков, 32 ГБ ОЗУ) с фиксированными параметрами симуляции: 1000 агентов, размер ячейки 10×10 пикселей, вес шага 50.

В качестве метрики сходимости использовался нормированный коэффициент корреляции Пирсона между эталонным распределением $\tilde{\rho}$ и динамическим распределением ρ_t по формуле 12.8 из подраздела 2.6.4, преобразованный в проценты сходства по формуле 12.9 из подраздела 2.6.4.

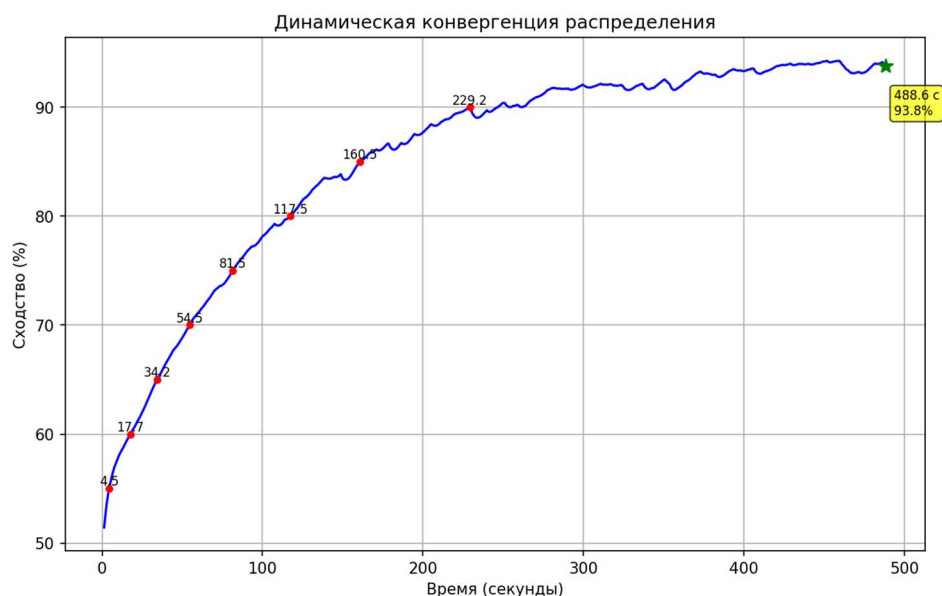


Рисунок 38 – Динамическая конвергенция исходного алгоритма (Python)

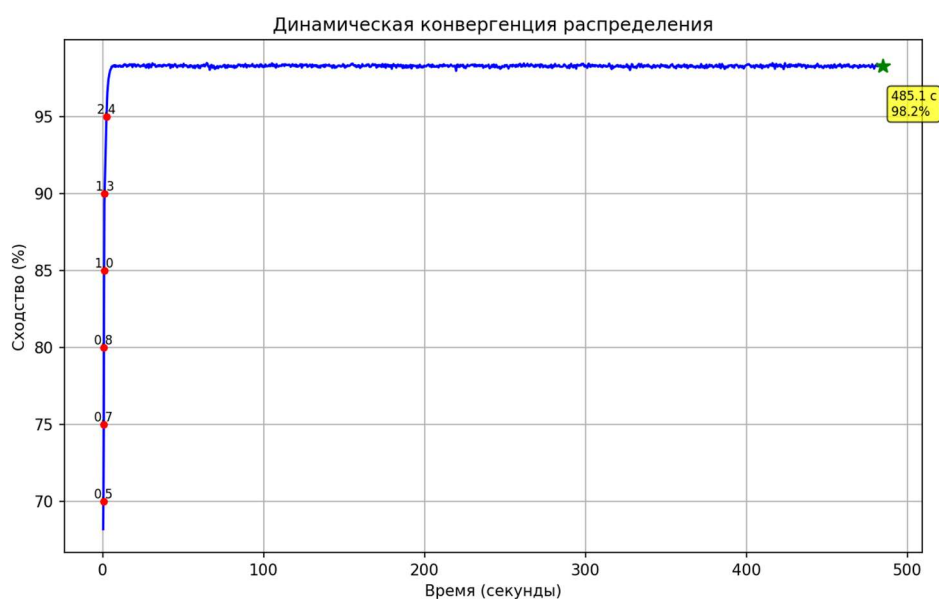


Рисунок 39 – Динамическая конвергенция оптимизированного алгоритма (C++, python)

Как показано на рисунках 38 и 39, оптимизированная версия достигает высоких уровней сходства с эталоном в 60–200 раз быстрее исходной реализации (см. таблицу 14).

Таблица 14 – Сравнение времени достижения заданных уровней сходства

Сходство, %	Время (исходный), с	Время (оптимизированный), с	Прирост, раз
55	4,5	-	-
60	17,7	0,3	59
65	34,2	0,4	85,5
70	54,5	0,5	109
75	81,5	0,7	116,4
80	117,5	0,8	146,9
85	160,5	1	160,5
90	229,2	1,3	176,3
93,8	488,6	2,3	212,4
95	-	2,4	-
98,2	-	2,8	-

Анализ данных таблицы 14 показывает, что оптимизированная реализация достигает уровня сходства 98,2 % за 2,8 секунды, тогда как исходный алгоритм не достиг уровня 95 % за 8 минут. Это означает, что оптимизированная версия работает в 172 раза быстрее при более высоком уровне точности. Такое ускорение критично для практического применения в реальном времени, где требуется обработка видеопотока без задержек.

Далее была исследована зависимость производительности от количества агентов. На рисунке 40 представлена тепловая карта среднего времени достижения различных уровней сходства для конфигураций с числом агентов от 500 до 10000, а рисунке 41 приведена тепловая карта средней загрузки процессора для тех же конфигураций агентов.

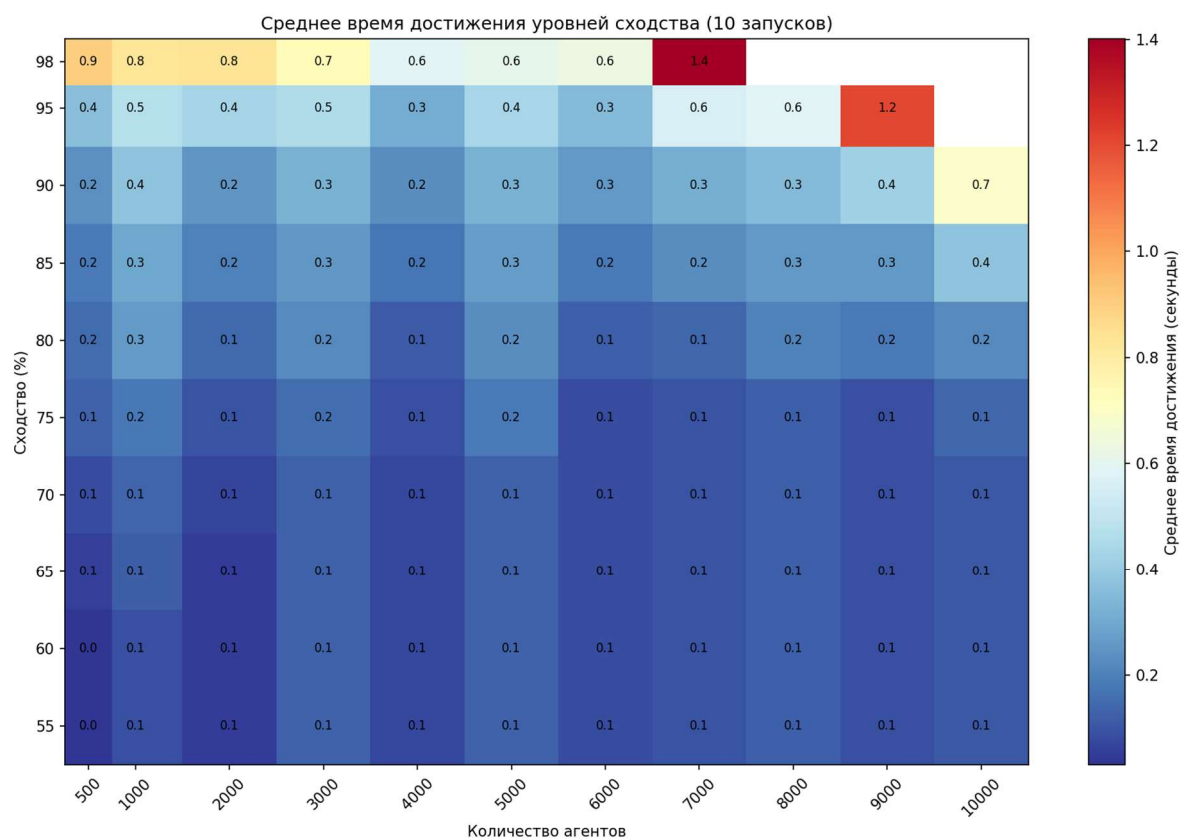


Рисунок 40 – Среднее время достижения уровней сходства

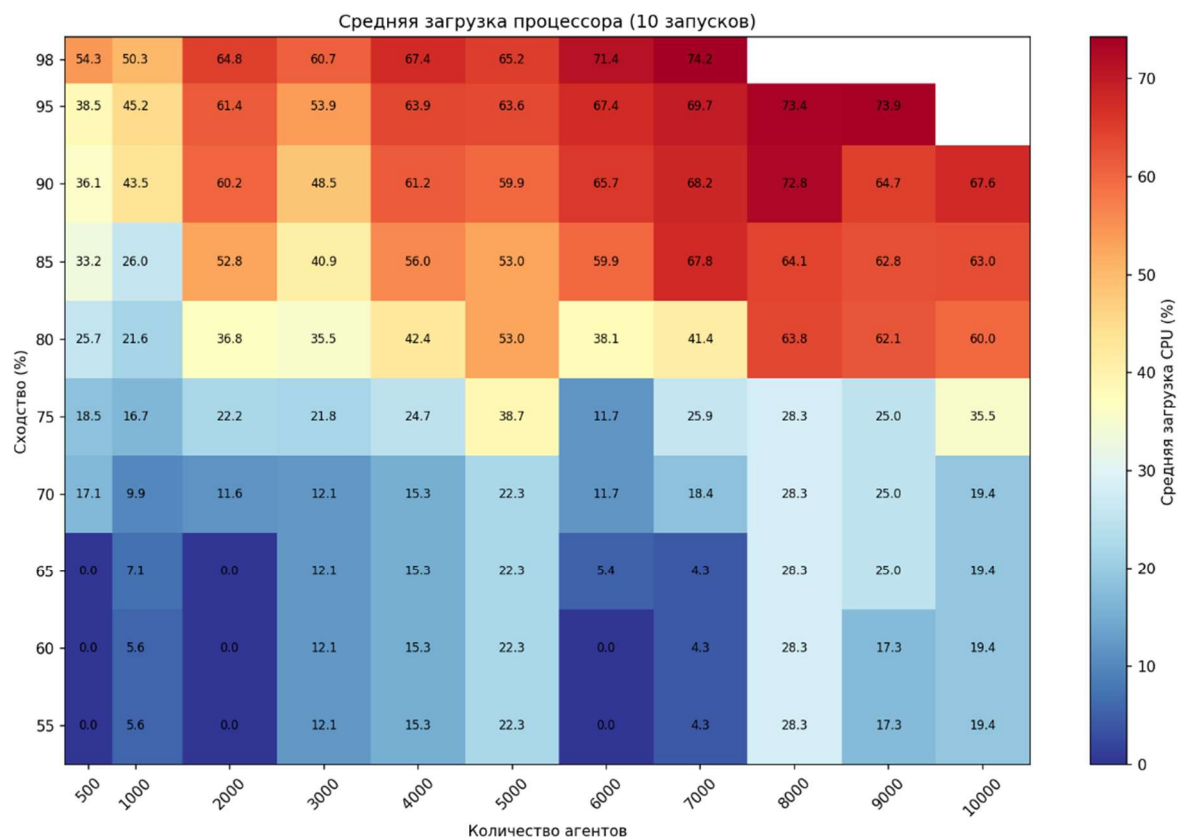


Рисунок 41 – Средняя загрузка процессора

На основе анализа тепловых карт на рисунках 40 и 41 установлено, что наиболее сбалансированной конфигурацией является набор из 1000 агентов. Данная конфигурация обеспечивает минимальную нагрузку на центральный процессор при незначительном (1,1 %) проигрыше во времени сходимости по сравнению с более ресурсоёмкими наборами.

Таким образом, перенос агентной предобработки в высокопроизводительную библиотеку на языке C++ [41] позволил сократить время достижения высоких уровней сходимости с минут до секунд, снизить нагрузку на центральный процессор в 1,6–2,0 раза по сравнению с исходной реализацией и обеспечить обработку видеопотока в реальном времени. Полученные результаты подтверждают эффективность предложенного подхода к оптимизации и обосновывают возможность интеграции разработанного модуля предобработки в информационную систему анализа внимания пользователей.

4.4 Интерфейс

Графический интерфейс приложения реализован на языке Python с использованием библиотеки Pygame [24, 42]. Весь код программы представлен в приложении В. Приложение представляет собой окно с тремя основными кнопками: «Захват экрана», «Обработка видео» и «Выход» (рисунок 42).

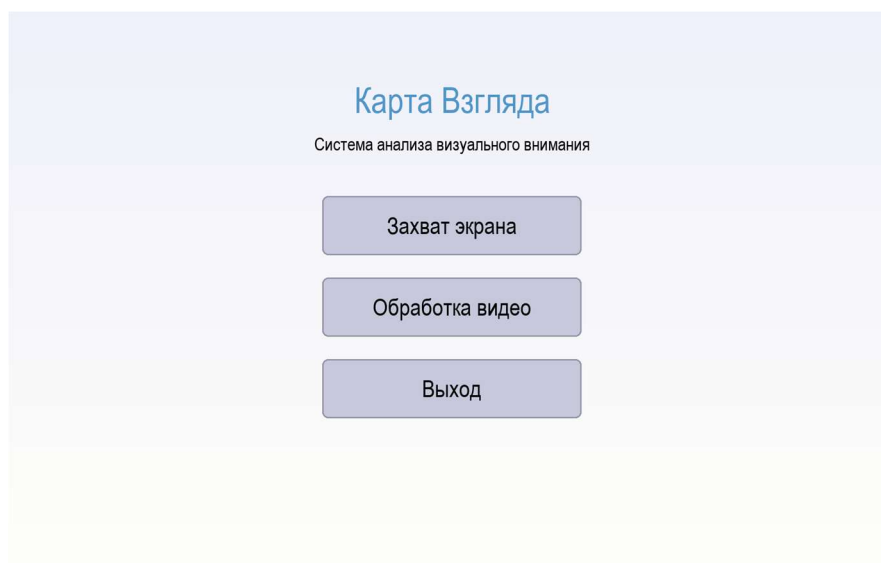


Рисунок 42 – Главное окно

При нажатии на кнопку «Захват экрана» запускается режим, при котором модель анализирует захваченное изображение и определяет области, наиболее привлекающие внимание пользователя, после чего результат отображается в виде тепловой карты [5], наложенной поверх экрана. Каждую секунду окно приложения автоматически становится прозрачным, чтобы выполнить захват изображения с экрана и визуализировать новое предсказание на основе текущего содержимого (рисунок 43).

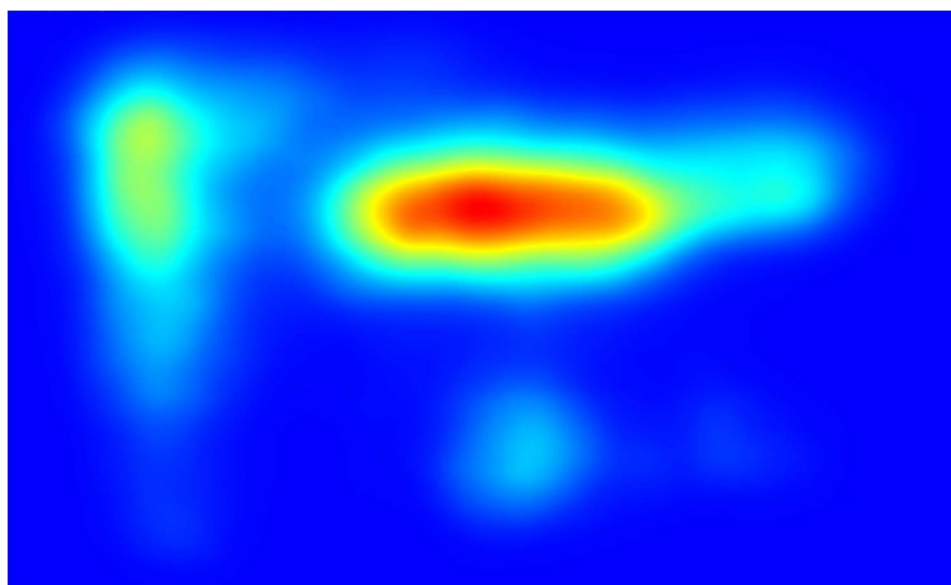


Рисунок 43 – Тепловая карта

Управление визуальной прозрачностью накладываемой тепловой карты реализовано через глобальные горячие клавиши: сочетания «Ctrl» + «+» и «Ctrl» + «-» позволяют дискретно увеличивать или уменьшать уровень прозрачности окна приложения. Данный механизм основан на динамическом изменении альфа-канала наложения (от 0 до 255) с шагом 10 единиц, что обеспечивает плавную регулировку в реальном времени без перезапуска визуализационного модуля. На рисунке 44 показано окно, сделанное прозрачным на 50 процентов, где хорошо видна наложенная тепловая карта областей внимания пользователя поверх исходного изображения экрана.

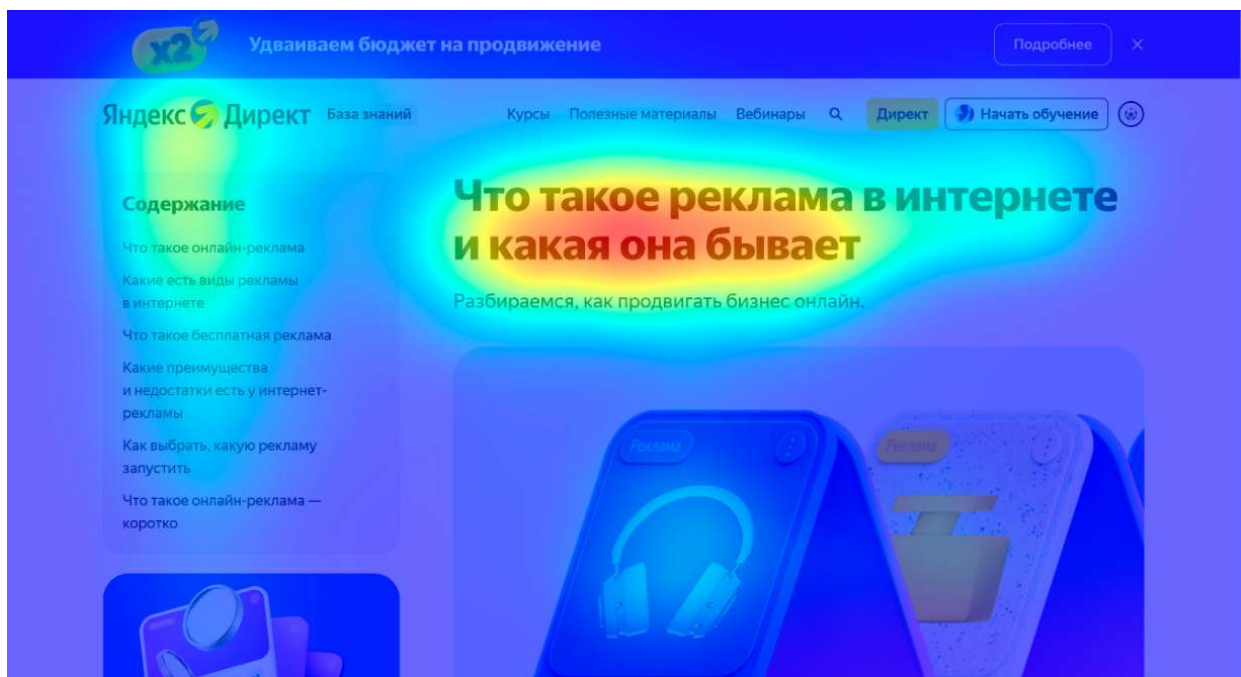


Рисунок 44 – Наложение тепловой карты с прозрачностью 50%

Переключение между режимами отображения осуществляется с помощью горячих клавиш: «Ctrl» + «1» – предсказанная карта взгляда, «Ctrl» + «2» – динамическое распределение агентов, «Ctrl» + «3» – эталонное распределение. Режим «Динамическое распределение агентов» показывает результат работы агентного моделирования, где 1000 виртуальных агентов [8] итеративно перемещаются по сетке экрана, руководствуясь коэффициентом привлекательности K_p и формируя карту накопленного внимания (рисунок 45). Данный режим демонстрирует процесс накопления контекста между кадрами: агенты покидают насыщенные области и исследуют новые зоны, что позволяет адаптивно отслеживать изменения визуального интереса во времени. Режим «Эталонное распределение» отображает изображение экрана после предобработки: преобразование в полутоновое [35], разбиение на ячейки 10×10 пикселей и нормировку яркости в диапазон 0–255 (рисунок 46). Переключение между тремя режимами позволяет проследить полный пайплайн обработки – от исходных данных через агентное моделирование к финальному предсказанию нейросети.

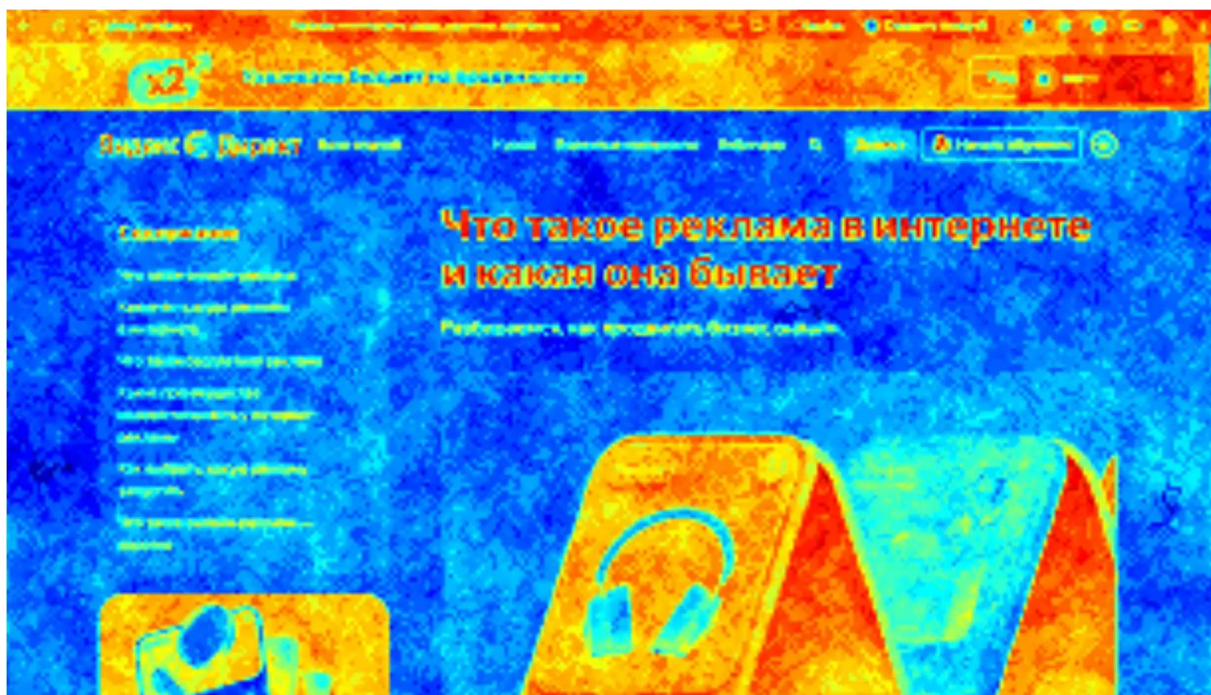


Рисунок 45 – Динамическое распределение агентов



Рисунок 46 – Эталонное распределение

Все три режима используют специальную цветовую схему [5]: изображения визуализируются из оттенков серого в оттенки от синего через голубой и желтый к красному. Это обеспечивает наглядное представление

областей привлечения внимания на экране, позволяя пользователю быстро идентифицировать наиболее важные зоны.

Если в главном меню выбрать пункт «Обработка видео», то откроется окно, в которое можно вставить путь к видеофайлу, хранящемуся на компьютере, которое необходимо обработать (см. рисунок 47).

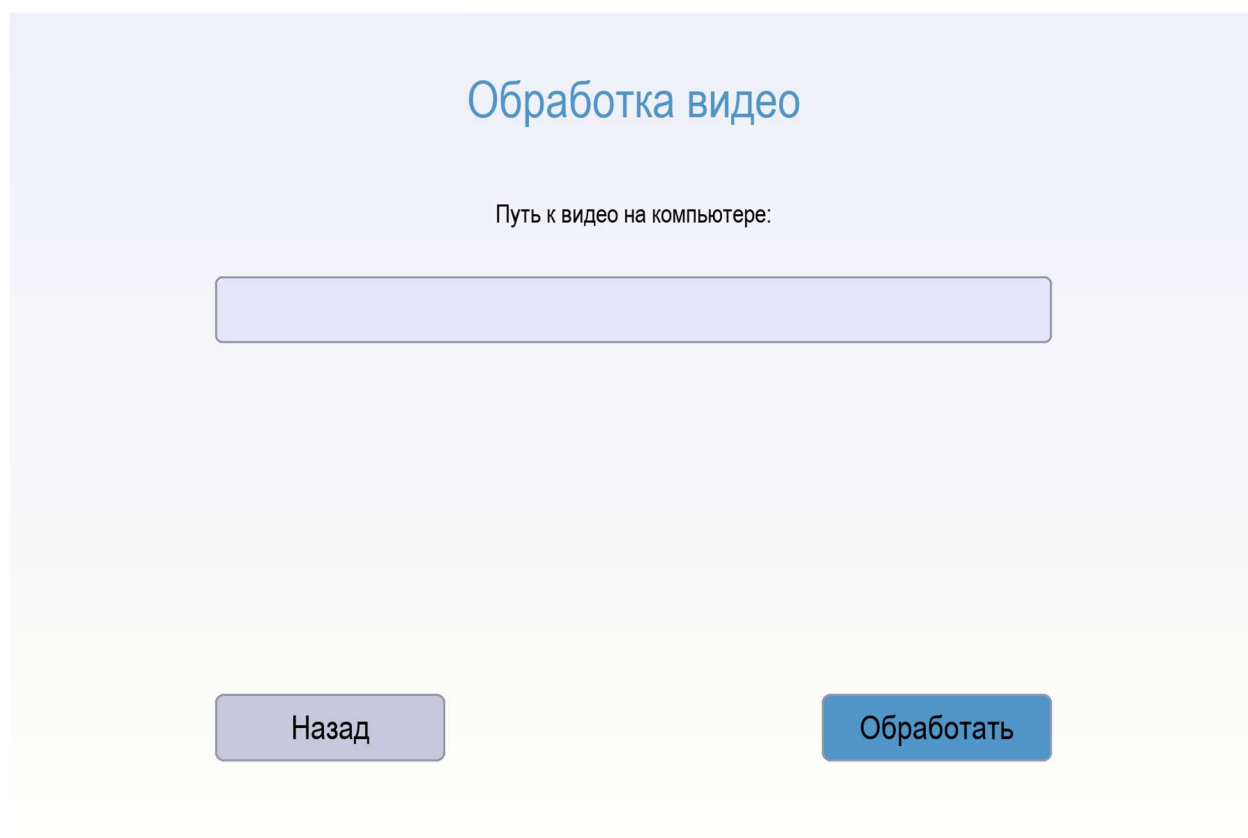


Рисунок 47 – Интерфейс выбора видеофайла для последующей обработки

Разработанный графический интерфейс обеспечивает удобное и интуитивно понятное взаимодействие пользователя с системой анализа визуального внимания, предоставляя возможность как в реальном времени наблюдать за предсказанными областями интереса на экране, так и обрабатывать заранее записанные видеоматериалы с наложением тепловых карт.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы было проведено исследование предметной области, связанной с автоматизацией анализа внимания пользователей к рекламным изображениям с использованием технологий айтрекинга и машинного обучения. Уточнена проблема отсутствия доступных, интегрированных и объективных инструментов для оценки эффективности рекламных визуальных материалов. Выявлены ключевые теоретические наработки и научные публикации в области программного айтрекинга, агентных методов восстановления изображения и алгоритмов машинного обучения для предсказания тепловых карт внимания. Определена глобальная цель исследования – повышение эффективности современного информационного маркетинга. Сформулирована гипотеза о предлагаемом решении, заключающаяся в том, что интеграция агентного подхода И.В. Кашина с CNN-моделью и библиотекой eyeGestures позволит автоматически распознавать области интереса с достаточной точностью. Определена локальная цель и сформулированы задачи исследования.

На этапе моделирования выполнена формализация путем построения концептуальных, структурной, алгоритмической, функционально-структурной, иерархической и математической моделей разрабатываемой системы. Математическая модель описывает процессы предобработки, извлечения признаков HOG, регрессии и оценки качества.

На этапе проектирования определены подсистемы разрабатываемой системы (загрузки данных, предобработки данных, применения модели машинного обучения, визуализации внимания, интерфейс пользователя) и подробно описаны их технология и функции. Выбраны программные средства и среда разработки: Python 3.12.6 для основных модулей, C++ для высокопроизводительной подсистемы предобработки, среда разработки

PyCharm 3. Выполнен синтез технического задания на разработку системы в соответствии с ГОСТ 34.602-89.

На стадии инженерной реализации произведена разработка демонстрационных программ для реализации предложенных алгоритмов. Собран набор данных из 11 000 пар «рекламное изображение – тепловая карта внимания» с использованием библиотеки eyeGestures. Обучена сверточная нейронная сеть с архитектурой энкодер-декодер и ASPP-модулем. Реализована высокопроизводительная C++ библиотека agent_sim для агентной предобработки с накоплением контекста между кадрами. Разработаны экранные формы пользовательского интерфейса системы на базе библиотеки Pygame, позволяющие в реальном времени отображать предсказанные тепловые карты, динамическое и эталонное распределения. Экспериментально подтверждено, что оптимизированная C++ реализация агентного подхода достигает уровня сходства 98,2 % за 2,8 секунды, что в 212 раз быстрее исходной реализации на Python, а выбранная конфигурация из 1000 агентов обеспечивает сбалансированную нагрузку на процессор.

Направлением дальнейшего развития является увеличение качества предсказания областей интереса путём расширения обучающей выборки за счёт большего числа пользователей и разнообразных типов рекламных изображений. Представленное решение подходит для интеграции в существующие маркетинговые платформы с целью автоматического A/B-тестирования рекламных креативов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Кравченко О. Н. Рекламная концепция и рекламный образ. Разработка эффективной рекламы / О. Н. Кравченко. – М. : Перо, 2017. – 134 с. – ISBN 978-5-44856-378-2
- 2 Белановский С. А. Глубокое интервью и фокус-группы : учебник по качественным методам социологического опроса / С. А. Белановский. – 3-е изд., перераб. – М. : Директ-Медиа, 2022. – 456 с. – ISBN 978-5-4499-3109-2.
- 3 Барабанщиков В. А. Айттрекинг: методы регистрации движений глаз в психологических исследованиях и практике : учебное пособие / В. А. Барабанщиков, А. В. Жегалло. – М. : Когито-Центр, 2014. – 128 с. – ISBN 978-5-89353-415-3.
- 4 Шакла Н. Машинное обучение и TensorFlow / Н. Шакла ; пер. с англ. А. И. Демьяникова. – СПб.: Питер, 2019. – 331 с. – (Серия «Библиотека программиста»). – ISBN 978-5-4461-0826-8.
- 5 Яковлев А. А. Визуализация данных в веб-аналитике / А. А. Яковлев. – СПб.: Питер, 2021. – 192 с. – ISBN 978-5-4461-1520-4.
- 6 Wedel M. Eye Tracking for Visual Marketing / M. Wedel, R. Pieters. – Hanover: Now Publishers Inc, 2008. – 93 p. – ISBN 978-1-60198-142-2.
- 7 Гольдштейн С. Л. Введение в системологию и системотехнику / С. Л. Гольдштейн, Т. Я. Ткаченко. – Екатеринбург: ИРРО, 1994. – 198 с.
- 8 Математическое моделирование. – 2025. – Т. 37, № 6. – С. 88–102. – URL: https://www.mathnet.ru/php/archive.phtml?wshow=paper&jrnid=mm&paperid=4650&option_lang=rus (дата обращения: 07.02.2026).
- 9 Исаченко О. В. Информационные технологии и системы: учебник / О. В. Исаченко. – М. : ИНФРА-М, 2025. – 234 с. – ISBN 978-5-16-019153-0.
- 10 ГОСТ 19.701-90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Условные обозначения и

- правила выполнения. – Взамен ГОСТ 19.002–80, ГОСТ 19.003–80; введ. 01.01.1992. – М. : Издательство стандартов, 1992. – 24 с.
- 11 Будович Л. С. Патентные исследования : учебное пособие / Л. С. Будович. – М. : РТУ МИРЭА, 2024. – 96 с. – ISBN 978-5-7339-2238-6.
 - 12 Caron M. Emerging Properties in Self-Supervised Vision Transformers / M. Caron, H. Touvron, I. Misra et al. [Электронный ресурс] // GitHub. – URL: <https://github.com/facebookresearch/dino.git> (дата обращения: 24.01.2026)
 - 13 Гонсалес Р. Цифровая обработка изображений / Р. Гонсалес, Р. Вудс. – М. : Техносфера, 2012. – 1104 с. – ISBN 978-5-94836-331-8.
 - 14 Bishop C. M. Pattern Recognition and Machine Learning / C. M. Bishop. – New York : Springer, 2006. – 738 p. – ISBN 978-0-387-31073-2.
 - 15 Goodfellow I. Deep Learning / I. Goodfellow, Y. Bengio, A. Courville. – Cambridge : MIT Press, 2016. – 800 p. – ISBN 978-0-262-03561-3.
 - 16 Bulling A. It's in Your Eyes: Towards Context-Awareness and Mobile HCI Using Wearable EOG Goggles / A. Bulling, D. Roggen, G. Tröster // Proceedings of the 10th International Conference on Ubiquitous Computing (UbiComp). – 2008. – P. 84–93.
 - 17 Bradski G. The OpenCV Library / G. Bradski // Dr. Dobb's Journal of Software Tools. – 2000. – Vol. 25, No. 11. – P. 120–123.
 - 18 King D. E. Dlib-ml: A Machine Learning Toolkit / D. E. King // Journal of Machine Learning Research. – 2009. – Vol. 10. – P. 1755–1758.
 - 19 Zhu Z. MADiff: Offline Multi-agent Learning with Diffusion Models / Z. Zhu, M. Liu, L. Mao et al. // arXiv preprint arXiv:2305.17330. – 2023.
 - 20 Dorigo M. Ant Colony Optimization / M. Dorigo, T. Stützle. – Cambridge: MIT Press, 2004. – 305 p. – ISBN 978-0-262-04219-2.
 - 21 Ronneberger O. U-Net: Convolutional Networks for Biomedical Image Segmentation / O. Ronneberger, P. Fischer, T. Brox // Medical Image Computing and Computer-Assisted Intervention (MICCAI). – 2015. – P. 234–241.

- 22 LeCun Y. Gradient-Based Learning Applied to Document Recognition / Y. LeCun, L. Bottou, Y. Bengio, P. Haffner // Proceedings of the IEEE. – 1998. – Vol. 86, No. 11. – P. 2278–2324.
- 23 Dosovitskiy A. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale / A. Dosovitskiy, L. Beyer, A. Kolesnikov et al. // International Conference on Learning Representations (ICLR). – 2021.
- 24 Van Rossum G. Python Tutorial / G. Van Rossum, F. L. Drake. – Python Software Foundation, 2019 [Электронный ресурс] // [Python.org](https://docs.python.org/3/tutorial/). – URL: <https://docs.python.org/3/tutorial/> (дата обращения: 12.02.2026).
- 25 Шапиро Л. Компьютерное зрение / Л. Шапиро, Дж. Стокман. – М. : Бином. Лаборатория знаний, 2013. – 752 с. – ISBN 978-5-9963-0991-7.
- 26 Deng J. ImageNet: A Large-Scale Hierarchical Image Database / J. Deng, W. Dong, R. Socher et al. // IEEE Conference on Computer Vision and Pattern Recognition (CVPR). – 2009. – P. 248–255.
- 27 Chen L. C. DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs / L. C. Chen, G. Papandreou, I. Kokkinos et al. // IEEE Transactions on Pattern Analysis and Machine Intelligence. – 2018. – Vol. 40, No. 4. – P. 834–848.
- 28 Hastie T. The Elements of Statistical Learning: Data Mining, Inference, and Prediction / T. Hastie, R. Tibshirani, J. Friedman. – 2nd ed. – New York: Springer, 2009. – 746 p. – ISBN 978-0-387-84857-0.
- 29 Simonyan K. Very Deep Convolutional Networks for Large-Scale Image Recognition / K. Simonyan, A. Zisserman // International Conference on Learning Representations (ICLR). – 2015.
- 30 Jing L. Self-Supervised Visual Feature Learning with Deep Neural Networks: A Survey / L. Jing, Y. Tian // IEEE Transactions on Pattern Analysis and Machine Intelligence. – 2021. – Vol. 43, No. 11. – P. 4037–4058.
- 31 Abnar S. Quantifying Attention Flow in Transformers / S. Abnar, W. Zuidema // Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL). – 2020. – P. 85–95.

- 32 Саати Т. Л. Принятие решений. Метод анализа иерархий / Т. Л. Саати. – М. : Радио и связь, 1993. – 278 с. – ISBN 5-256-00343-8.
- 33 Saaty T. L. Deriving the AHP 1–9 Scale from First Principles / T. L. Saaty // ISAHp 2001, Berne, Switzerland, August 2–4, 2001. – P. 1–4. [Электронный ресурс] // ISAHp. – URL: <http://isahp-org.creativedecisions.net/uploads/125-p.pdf> (дата обращения: 24.03.2026).
- 34 Duchowski A. T. Aggregate Gaze Visualization with Real-time Heatmaps / A. T. Duchowski, K. Krejtz, I. Krejtz // Proceedings of the Symposium on Eye Tracking Research and Applications (ETRA). – 2012. – P. 13–20. – DOI: 10.1145/2168556.2168558.
- 35 ITU-R BT.601-7. Studio encoding parameters of digital television. – Geneva : ITU, 2011. – 34 p.
- 36 Harris C. R. Array Programming with NumPy / C. R. Harris, K. J. Millman, S. J. van der Walt et al. // Nature. – 2020. – Vol. 585. – P. 357–362.
- 37 Abadi M. TensorFlow: A System for Large-Scale Machine Learning / M. Abadi, P. Barham, J. Chen et al. // 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI). – 2016. – P. 265–283.
- 38 Chollet F. Deep Learning with Python / F. Chollet. – Shelter Island: Manning Publications, 2017. – 384 p. – ISBN 978-1-61729-443-3.
- 39 Pedregosa F. Scikit-learn: Machine Learning in Python / F. Pedregosa, G. Varoquaux, A. Gramfort et al. // Journal of Machine Learning Research. – 2011. – Vol. 12. – P. 2825–2830.
- 40 Oliphant T. E. Guide to NumPy. – 2nd ed. – Austin : CreateSpace, 2015. – 382 p. – ISBN 978-1-517-30007-4.
- 41 Страуструп Б. Язык программирования C++ : специальное издание / Б. Страуструп. – М. : Бином-Пресс, 2011. – 1136 с. – ISBN 978-5-9518-0324-1.
- 42 McGugan W. Beginning Game Development with Python and Pygame: From Novice to Professional / W. McGugan. – Apress, 2007. – 338 p. – ISBN 978-1-59059-872-6.

- 43 AllFusion Process Modeler 7.3. Руководство пользователя. – Computer Associates, 2006. – 500 с.
- 44 Pearson K. Notes on Regression and Inheritance in the Case of Two Parents / K. Pearson // Proceedings of the Royal Society of London. – 1895. – Vol. 58. – P. 240–242.
- 45 ГОСТ 34.602–89. Информационная технология. Техническое задание на создание автоматизированной системы. – Введ. 01.01.1990. – М. : Издательство стандартов, 1990. – 11 с.
- 46 mss: Fast Screenshot Module for Python [Электронный ресурс] // GitHub. – URL: <https://github.com/BoboTiG/python-mss> (дата обращения: 21.0.2026).
- 47 Огилви Д. Огилви о рекламе / Д. Огилви. – М. : Манн, Иванов и Фербер, 2013. – 240 с. – ISBN 978-5-00057-931-2.
- 48 TensorFlow Documentation. SavedModel format [Электронный ресурс] // TensorFlow. – URL: https://www.tensorflow.org/guide/saved_model (дата обращения: 11.01.2026).
- 49 JetBrains PyCharm 3.0. Руководство разработчика. – JetBrains, 2013.
- 50 Python keyboard library [Электронный ресурс] // GitHub. – URL: <https://github.com/boppreh/keyboard> (дата обращения: 19.01.2026).
- 51 Python threading documentation [Электронный ресурс] // Python.org. – URL: <https://docs.python.org/3/library/threading.html> (дата обращения: 14.01.2026).
- 52 Python ctypes documentation [Электронный ресурс] // Python.org. – URL: <https://docs.python.org/3/library/ctypes.html> (дата обращения: 25.12.2026).
- 53 Kingma D. P. Adam: A Method for Stochastic Optimization / D. P. Kingma, J. Ba // International Conference on Learning Representations (ICLR). – 2015.
- 54 pybind11 documentation [Электронный ресурс] // pybind11 documentation. – URL: <https://pybind11.readthedocs.io> (дата обращения: 24.01.2026).

ПРИЛОЖЕНИЕ А

(Обязательное)

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

УРАЛЬСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ ИМЕНИ ПЕРВОГО
ПРЕЗИДЕНТА РОССИИ Б.Н. ЕЛЬЦИНА
(УрФУ)

Физико-технологический институт
Кафедра технической физики

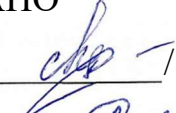
**РАЗВИТИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ
ПОДДЕРЖКИ СОВРЕМЕННОГО ИНТЕРНЕТ-МАРКЕТИНГА
НА ОСНОВЕ АВТОМАТИЧЕСКОГО РАСПОЗНАВАНИЯ
ОБЛАСТЕЙ ИНТЕРЕСА НА РЕКЛАМНЫХ
ИЗОБРАЖЕНИЯХ В ВИДЕОПОТОКЕ**

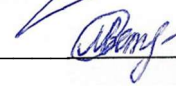
ТЕХНИЧЕСКОЕ ЗАДАНИЕ

На 9 листах

Действует с «22» 01.2026г.

СОГЛАСОВАНО

Руководитель  / (И.В. Кашин)

Исполнитель  / (М.А. Ветров)

Екатеринбург

2026

A1 ОБЩИЕ СВЕДЕНИЯ

A1.1 Полное название разработки

«Развитие информационной системы поддержки современного интернет-маркетинга на основе автоматического распознавания областей интереса на рекламных изображениях в видеопотоке».

A1.2 Наименование предприятий разработчика и заказчика системы

Заказчик – ФГАОУ ВО «Уральский федеральный университет имени первого Президента России Б. Н. Ельцина» Физико-технологический институт, кафедра «Технической физики».

Разработчик – студент ФГАОУ ВО «Уральский федеральный университет имени первого Президента России Б. Н. Ельцина» Физико-технологического института, кафедры «Технической физики», группы ФТ-420008 Ветров Максим Алексеевич.

A1.3 Плановые сроки начала и окончания работы

- начало - 22.01.2026;
- окончание - 01.06.2026.

A1.4 Конечный результат разработки

Конечным результатом является программное обеспечение, которое автоматически распознаёт и визуализирует области интереса пользователей на рекламных изображениях с использованием обученной свёрточной нейронной сети и агентного подхода к предобработке изображения.

A1.5 Порядок оформления и представления заказчику результатов работ

В соответствии с ГОСТ 19.201-79 оформляется, согласуется и утверждается техническое задание (ТЗ).

На основе ТЗ разрабатывается технорабочий проект.

По окончании работ по созданию программного обеспечения проверяется его соответствие назначению и функциям, изложенным в ТЗ.

После тестирования системы оформляются соответствующие документы.

Оформляется пояснительная записка к дипломной работе.

Защита результатов работы.

A2 НАЗНАЧЕНИЕ И ЦЕЛИ СОЗДАНИЯ СИСТЕМЫ

A2.1 Назначение системы

Автоматическое распознавание и визуализация областей интереса пользователей на рекламных изображениях средствами программного айтрекинга, машинного обучения и агентного подхода к предобработке изображения.

A2.2 Цели создания системы

Цель создания системы:

- создание информационной системы для автоматического анализа визуального внимания пользователей к рекламным изображениям;
- обеспечение точного прогнозирования областей интереса (тепловых карт внимания) с использованием свёрточной нейронной сети;
- обеспечение накопления контекста между последовательными кадрами видеопотока на основе агентного подхода И.В. Кашина к предобработке изображения.

A3 ХАРАКТЕРИСТИКИ СИСТЕМЫ

Объект – процесс анализа визуального внимания пользователей к рекламным изображениям. Система обрабатывает видеопоток с экрана компьютера или сохранённые видеофайлы, применяет агентную предобработку для накопления контекста между кадрами, затем с помощью обученной свёрточной нейронной сети (VGG16 + ASPP + декодер) предсказывает тепловую карту внимания и визуализирует её с наложением на исходное изображение.

A4 ТРЕБОВАНИЯ К СИСТЕМЕ

A4.1 Требования к системе в целом

Система должна реализовывать следующие функции:

- предобработка изображений с накоплением контекста предыдущих кадров на основе агентного подхода И.В. Кашина;
- применение обученной модели машинного обучения для предсказания областей интереса;
- визуализация результатов в виде тепловых карт с наложением на исходное изображение;
- переключение между режимами отображения (предсказанная карта взгляда, динамическое распределение агентов, эталонное распределение);
- обработка как сохранённых видео, так и видеопотока изображения с экрана компьютера.

Данный программный продукт должен обладать следующими характеристиками:

- языки программирования – Python 3.12.6 (основные модули, интерфейс, модель), C++ (высокопроизводительная библиотека `agent_sim` для агентной предобработки);
- среда разработки – PyCharm 3.

Требования к устройствам, на котором будет работать разработанное программное решение:

- операционная система: Windows 10 и выше;
- процессор: не менее 2 ГГц, 4 и более ядер;
- оперативная память (ОЗУ): не менее 4096 Мб (рекомендуется 8 Гб);
- свободное место на диске: не менее 5 Гб;
- устройства ввода/вывода: клавиатура, мышь.

Исходной информацией для системы является: видеопоток с экрана компьютера или сохранённые видеофайлы в формате MP4.

Выходной информацией должно быть: тепловая карта внимания, наложенная на исходное изображение.

A4.1.1 Требования к структуре и функционированию системы

С функциональной точки зрения система представляет собой совокупность подсистем, работающих последовательно друг за другом в цикле:

- Подсистема загрузки данных;
- Подсистема агентной предобработки с накоплением контекста предыдущих кадров;
- Подсистема применения модели машинного обучения;
- Подсистема визуализации внимания;
- Интерфейс пользователя.

A4.1.2 Требования к надежности

При возникновении нештатных ситуаций система должна сохранять работоспособность, выводить сообщение об ошибке пользователю и продолжать работу в доступных режимах.

А4.1.3 Требования к защите информации от несанкционированного доступа

Доступ к предлагаемой системе не требует авторизации. Система не осуществляет сбор, хранение или передачу персональных данных пользователей.

А4.1.4 Требования к численности и квалификации персонала

Пользователь должен обладать базовыми навыками работы с персональным компьютером (запуск приложения, работа с файлами, использование горячих клавиш). Специальная подготовка не требуется.

А4.2 Требования к функциям, выполняемым системой

А4.2.1 Подсистема загрузки данных

Подсистема выполняет следующие функции:

- захват текущего изображения с экрана компьютера;
- загрузка сохранённых видеофайлов в формате MP4;
- преобразование входных данных к единому формату для последующей обработки.

А4.2.2 Подсистема предобработки

Подсистема выполняет следующие функции:

- преобразование RGB-изображения в полутоновое (формула 11.1);
- разбиение на дискретную сетку ячеек (по 10 пикселей);
- нормализация эталонного распределения яркости;
- вычисление коэффициента привлекательности K_p для каждой ячейки;
- перемещение агентов в соседние ячейки с максимальным K_p ;
- нормализация динамического распределения для передачи в модель.

А4.2.3 Подсистема применения модели машинного обучения

Подсистема выполняет следующие функции:

- загрузка обученной модели;
- применение модели для предсказания тепловой карты внимания;
- получение предсказанной тепловой карты с интенсивностью внимания от 0 до 1;
- масштабирование тепловой карты к исходному разрешению экрана.

А4.2.4 Подсистема визуализации внимания

Подсистема выполняет следующие функции:

- построение цветовой тепловой карты на основе предсказанных значений;
- наложение тепловой карты на исходное изображение с регулируемой прозрачностью;
- переключение режимов отображения с помощью горячих клавиш;
- управление работой системы через графический интерфейс;
- отображение окна поверх всех окон для наложения тепловой карты.

4.3 Требования к видам обеспечения

А4.3.1 Информационное обеспечение системы

- данные поступают с экрана или из файлов;
- обученная модель хранится в директории `models` в формате `TensorFlow SavedModel`;
- результаты выводятся в графическом виде.

А4.3.2 Информационное обеспечение системы

Предоставляется руководство с описанием функционала и рекомендуется провести обучение пользователей работе с системой.

A5 СОСТАВ И СОДЕРЖАНИЕ РАБОТ ПО СОЗДАНИЮ СИСТЕМЫ

A5.1 Перечень стадий создания

Стадии:

- составление пояснительной записки;
- реализация.

A5.2 Перечень этапов создания

Этапы:

- литературный обзор;
- моделирование;
- техническое задание, внутреннее проектирование;
- реализация.

A5.3 Сроки выполнения работ по стадиям и этапам

1	литературный обзор, проектирование	ноябрь – декабрь
2	Моделирование	декабрь – январь
3	Проектирование и реализация	февраль – апрель
4	написание пояснительной записки	май – июнь

A5.4 Перечень исполнителей, консультантов и документы согласования

A5.4.1 Исполнители и консультанты

Исполнители и консультанты:

- разработчик – студент группы ФТ-420008 Ветров М.А.;
- руководитель – доцент, кандидат физико-математических наук
Кашин И. В.

A5.4.2 Документы согласования

1 Задание по дипломной работе студента

2 Пояснительная записка

A6 ПОРЯДОК КОНТРОЛЯ И ПРИЕМКИ СИСТЕМЫ

A6.1 Регламент

1 Постановки на разработку системы.

2 Защита дипломного проекта перед ГЭК.

A7 ТРЕБОВАНИЯ К СОСТАВУ И СОДЕРЖАНИЮ РАБОТ ПО ПОДГОТОВКЕ СИСТЕМЫ К ДЕЙСТВИЮ

Требования к составу и содержанию работ:

- проверить наличие технического обеспечения;
- установить необходимые программные продукты;
- сконфигурировать систему;
- проверить правильность функционирования.

A8 ТРЕБОВАНИЯ К ДОКУМЕНТИРОВАНИЮ

A8.1 Перечень подлежащих разработке комплектов и видов документов по ГОСТ 34.201-89 и отраслевым требованиям

Перечень подлежащих разработке комплектов и видов документов:

- задание на проектирование;
- пояснительная записка по ГОСТ 7.32-2017;
- слайд-презентация;
- доклад.

A8.2 Требования к предоставляемым документам

Требования к предоставляемым документам:

- продолжительность доклада не более 10 минут;

- пояснительная записка должна быть оформлена в соответствии с ГОСТ 34.602-89.

A9 ИСТОЧНИКИ РАЗРАБОТКИ

- Гольдштейн С.Л., Ткаченко Т.Я. Введение в системологию и системотехнику. Екатеринбург: ИРРО, 1994. 198 с.
- ГОСТ 34.601-90 Автоматизированные системы. Стадии создания. – Введ. 01.01.90. М.: Издательство стандартов, 1989.
- ГОСТ 34.602-89 Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. – Введ. 01.01.99. – М.: Издательство стандартов, 1989.
- ГОСТ 34.003-90 Автоматизированные системы. Термины и определения. – Введ. 01.01.90. – М.: Издательство стандартов, 1989.

ПРИЛОЖЕНИЕ Б

(Обязательное)

КОД БИБЛИОТЕКИ AGENT_SIM

```
#include <pybind11/pybind11.h> // Связывание C++ с Python
#include <pybind11/numpy.h>     // Работа с NumPy-массивами
#include <cstdint>               // Использование uint8_t
#include <algorithm>             // Стандартные алгоритмы
#include <vector>                // динамический массив
#include <cmath>                 // Математические функции
#include <mutex>                 // для многопоточной работы
#include <stdexcept>            // Исключения (для обработки ошибок)

namespace py = pybind11; // Создание псевдонима для удобства

// Глобальные переменные для хранения эталонной карты
static std::vector<float> g_reference;
static int g_rows = 0;
static int g_cols = 0;
static std::mutex g_ref_mutex;

// Вспомогательная функция для выполнения одного шага всех
агентов
void single_step(
    int* pos_ptr, // входные/выходные позиции
    (N,2)
    float* ref_ptr, // reference (rows, cols) -
нормализовано 0-255
    float* dyn_ptr, // dynamic (rows, cols) -
будет изменён (ненормализован)
    int* seq_ptr, // последовательность
направлений (8)
    bool allow_multiple,
    float coeff,
    int rows,
    int cols,
    int N,
    int dynamic_step
) {
    // Строим карту занятости на начало шага
    std::vector<uint8_t> occupied(rows * cols, 0);
    for (int i = 0; i < N; ++i) {
        int r = pos_ptr[i * 2];
        int c = pos_ptr[i * 2 + 1];
        if (r >= 0 && r < rows && c >= 0 && c < cols)
            occupied[r * cols + c] = 1;
    }

    // Направления (8 направлений)
    const int directions[8][2] = {
```

```

        {0, 1}, {1, 0}, {0, -1}, {-1, 0},
        {1, 1}, {1, -1}, {-1, 1}, {-1, -1}
    };

    // Временные массивы для новых позиций и обновлений
    std::vector<int> new_positions(N * 2);
    std::vector<int> updates(N);

#pragma omp parallel for
for (int idx = 0; idx < N; ++idx) {
    int r = pos_ptr[idx * 2];
    int c = pos_ptr[idx * 2 + 1];
    int self_linear = r * cols + c;

    int best_r = r;
    int best_c = c;
    float best_Kp = -1e9f;
    bool found = false;

    // Перебираем направления в заданном порядке
    for (int d = 0; d < 8; ++d) {
        int dir = seq_ptr[d];
        int nr = r + directions[dir][0];
        int nc = c + directions[dir][1];

        if (nr >= 0 && nr < rows && nc >= 0 && nc < cols) {
            int target_linear = nr * cols + nc;
            // Проверка занятости целевой клетки
            if (!allow_multiple && occupied[target_linear])
                continue;

            // Проверка соседей целевой клетки (8 соседей),
            // исключая самого агента
            bool neighbor_occupied = false;
            for (int di = -1; di <= 1 && !neighbor_occupied;
                ++di) {
                for (int dj = -1; dj <= 1; ++dj) {
                    if (di == 0 && dj == 0) continue;
                    int ni = nr + di;
                    int nj = nc + dj;
                    if (ni >= 0 && ni < rows && nj >= 0 &&
                        nj < cols) {
                        int neighbor_linear = ni * cols +
                            nj;
                        if (neighbor_linear == self_linear)
                            continue;
                        if (occupied[neighbor_linear]) {
                            neighbor_occupied = true;
                            break;
                        }
                    }
                }
            }
        }
    }
}

```

```

        if (neighbor_occupied)
            continue;

        float Kp = ref_ptr[target_linear] - coeff *
dyn_ptr[target_linear];
        if (Kp > best_Kp) {
            best_Kp = Kp;
            best_r = nr;
            best_c = nc;
            found = true;
        }
    }

    if (!found) {
        best_r = r;
        best_c = c;
    }

    new_positions[idx * 2] = best_r;
    new_positions[idx * 2 + 1] = best_c;
    updates[idx] = best_r * cols + best_c;
}

// Копируем новые позиции
for (int i = 0; i < N * 2; ++i) {
    pos_ptr[i] = new_positions[i];
}

// Обновляем dynamic distribution (суммируем по уникальным
клеткам)
std::vector<int> counts(rows * cols, 0);
for (int i = 0; i < N; ++i) {
    int idx = updates[i];
    counts[idx]++;
}
for (int lin = 0; lin < rows * cols; ++lin) {
    if (counts[lin] > 0) {
        dyn_ptr[lin] += counts[lin] * dynamic_step;
    }
}

}

// Функция обновления эталонной карты (вызывается из потока
захвата)
void update_reference(
    py::array_t<uint8_t> image,           // (H, W, 3) RGB
    изображение                           // (H, W, 3) RGB
    int cell_size
) {
    auto buf_img = image.request();
    if (buf_img.ndim != 3 || buf_img.shape[2] != 3) {
        throw std::runtime_error("Input image must be H x W x 3");
    }
}

```

```

    RGB");
}

int H = buf_img.shape[0];
int W = buf_img.shape[1];
int out_h = H / cell_size;
int out_w = W / cell_size;

if (out_h == 0 || out_w == 0) {
    // Слишком маленькое изображение - не обновляем карту
    return;
}

uint8_t* img_ptr = static_cast<uint8_t*>(buf_img.ptr);

// Вычисляем средние значения по ячейкам
std::vector<float> cell_means(out_h * out_w);
#pragma omp parallel for collapse(2)
for (int i = 0; i < out_h; ++i) {
    for (int j = 0; j < out_w; ++j) {
        float sum = 0.0f;
        for (int dy = 0; dy < cell_size; ++dy) {
            for (int dx = 0; dx < cell_size; ++dx) {
                int y = i * cell_size + dy;
                int x = j * cell_size + dx;
                if (y < H && x < W) {
                    uint8_t* pixel = img_ptr + (y * W + x) *
3;
                    float gray = 0.299f * pixel[0] + 0.587f
* pixel[1] + 0.114f * pixel[2];
                    sum += gray;
                }
            }
        }
        cell_means[i * out_w + j] = sum / (cell_size *
cell_size);
    }
}

// Находим максимальное значение для нормализации
float max_val = 0.0f;
for (int idx = 0; idx < out_h * out_w; ++idx) {
    if (cell_means[idx] > max_val) max_val =
cell_means[idx];
}

// Нормализуем к 0-255 и сохраняем в глобальную переменную
std::vector<float> new_ref(out_h * out_w);
if (max_val > 0) {
    for (int idx = 0; idx < out_h * out_w; ++idx) {
        new_ref[idx] = (cell_means[idx] / max_val) * 255.0f;
    }
} else {

```

```

        for (int idx = 0; idx < out_h * out_w; ++idx) {
            new_ref[idx] = 0.0f;
        }
    }

    // Заменяем глобальную карту под защитой мьютекса
    {
        std::lock_guard<std::mutex> lock(g_ref_mutex);
        g_reference = std::move(new_ref);
        g_rows = out_h;
        g_cols = out_w;
    }
}

// Функция для получения размеров эталонной карты
std::pair<int, int> get_reference_dims() {
    std::lock_guard<std::mutex> lock(g_ref_mutex);
    return {g_rows, g_cols};
}

// Функция для выполнения нескольких шагов подряд и возврата
// обоим распределений
void multi_step_agents(
    py::array_t<int> positions,           // (N, 2)
    входные/выходные позиции
    py::array_t<float> dynamic_in,        // (rows, cols) входное
    состояние dynamic (нормализовано 0-255)
    py::array_t<int> sequence,           // (8,) порядок
    направлений
    bool allow_multiple,
    int steps,
    int dynamic_step,
    py::array_t<float> dynamic_out,       // (rows, cols)
    выходное состояние dynamic (будет нормализовано)
    py::array_t<float> reference_out      // (rows, cols)
    выходная эталонная карта (копия)
) {
    auto buf_pos = positions.request();
    auto buf_dyn_in = dynamic_in.request();
    auto buf_seq = sequence.request();
    auto buf_dyn_out = dynamic_out.request();
    auto buf_ref_out = reference_out.request();

    int N = buf_pos.shape[0];

    // Получаем текущую эталонную карту из глобальной переменной
    std::vector<float> current_ref;
    int rows, cols;
    {
        std::lock_guard<std::mutex> lock(g_ref_mutex);
        if (g_rows == 0 || g_cols == 0) {
            throw std::runtime_error("Reference map not
initialized. Call update_reference first.");
        }
    }
}

```

```

    }
    rows = g_rows;
    cols = g_cols;
    current_ref = g_reference; // копируем
}

// Проверяем размеры входных массивов
if (buf_dyn_in.ndim != 2 || buf_dyn_in.shape[0] != rows ||
buf_dyn_in.shape[1] != cols) {
    throw std::runtime_error("dynamic_in dimensions do not
match reference");
}
if (buf_dyn_out.ndim != 2 || buf_dyn_out.shape[0] != rows ||
buf_dyn_out.shape[1] != cols) {
    throw std::runtime_error("dynamic_out dimensions do not
match reference");
}
if (buf_ref_out.ndim != 2 || buf_ref_out.shape[0] != rows ||
buf_ref_out.shape[1] != cols) {
    throw std::runtime_error("reference_out dimensions do
not match reference");
}

int* pos_ptr = static_cast<int*>(buf_pos.ptr);
float* dyn_in_ptr = static_cast<float*>(buf_dyn_in.ptr);
int* seq_ptr = static_cast<int*>(buf_seq.ptr);
float* dyn_out_ptr = static_cast<float*>(buf_dyn_out.ptr);
float* ref_out_ptr = static_cast<float*>(buf_ref_out.ptr);

// Копируем входной dynamic в выходной (будем работать с
ним)
for (int i = 0; i < rows * cols; ++i) {
    dyn_out_ptr[i] = dyn_in_ptr[i];
}

// Предвычисляем сумму reference (она постоянна)
float total_reference = 0.0f;
for (int i = 0; i < rows * cols; ++i) {
    total_reference += current_ref[i];
}

for (int step = 0; step < steps; ++step) {
    // Вычисляем коэффициент на основе текущего dynamic_out
    float total_dynamic = 0.0f;
    for (int i = 0; i < rows * cols; ++i) {
        total_dynamic += dyn_out_ptr[i];
    }
    float coeff = total_reference / (total_dynamic + 1e-
10f);

    // Выполняем один шаг
    single_step(pos_ptr, current_ref.data(), dyn_out_ptr,
seq_ptr, allow_multiple,

```

```

        coeff, rows, cols, N, dynamic_step);
    }

    // Нормализуем dynamic_out к 0-255
    float max_val = 0.0f;
    for (int i = 0; i < rows * cols; ++i) {
        if (dyn_out_ptr[i] > max_val) max_val = dyn_out_ptr[i];
    }
    if (max_val > 0) {
        for (int i = 0; i < rows * cols; ++i) {
            dyn_out_ptr[i] = (dyn_out_ptr[i] / max_val) *
255.0f;
        }
    } else {
        // Если все нули, оставляем как есть (уже нули)
    }

    // Копируем текущую эталонную карту в выходной массив
    reference_out
    for (int i = 0; i < rows * cols; ++i) {
        ref_out_ptr[i] = current_ref[i];
    }
}

PYBIND11_MODULE(agent_sim, m) {
    m.def("update_reference", &update_reference, "Update
internal reference map from screen image");
    m.def("get_reference_dims", &get_reference_dims, "Get
dimensions of the stored reference map");
    m.def("multi_step_agents", &multi_step_agents, "Perform
multiple simulation steps, return normalized dynamic and
reference");
}

```

ПРИЛОЖЕНИЕ В

(Обязательное)

КОД ОСНОВНОЙ ПРОГРАММЫ

```
import ctypes
import os
import queue
import subprocess
import sys
import threading
import time
from collections import deque
from pathlib import Path
import agent_sim
import cv2
import keyboard
import mss
import numpy as np
import pygame
import tensorflow as tf
from matplotlib.colors import LinearSegmentedColormap

# ===== КОНСТАНТЫ =====
MODELS_DIR = "models" # каталог моделей
MODEL_HASH = "1873d95a6db30af7b1bb69b0b8c44dfe8f78b000" # хеш
модели

DYNAMIC_STEP = 50 # шаг
NUM_AGENTS = 1000 # число агентов
NUM_ITERATIONS = 1000000 # макс. итераций
ALLOW_MULTIPLE_AGENTS_IN_CELL = True # много агентов в 1 ячейке
CELL_SIZE = 10 # размер ячейки (пикс)
SCREEN_CAPTURE_INTERVAL = 1.0 # интервал захвата (с)
CAPTURE_INTERVAL_STEP = 0.1 # шаг изменения прозрачности
MIN_CAPTURE_INTERVAL = 0.05 # мин. прозрачность
MAX_CAPTURE_INTERVAL = 1.0 # макс. прозрачность
INITIAL_ALPHA = 128 # нач. прозрачность
PREDICT_ENABLED = True # предсказание вкл
STEPS_PER_FRAME = 200 # шагов на кадр
TOPMOST_FORCE_INTERVAL = 0.05 # период поднятия окна
OUTPUT_DIR = "processed_videos" # папка результатов

if sys.platform == 'win32':
    ctypes.windll.user32.SetProcessDPIAware() # учёт DPI

# ===== ЦВЕТОВАЯ СХЕМА =====
def create_custom_colormap():
    # градиент: синий -> голубой -> жёлтый -> красный
    colors = [(1.0, 0.0, 0.0), (1.0, 1.0, 0.0), (0.0, 1.0, 1.0),
```



```

(0.0, 0.0, 1.0)]
    positions = [0.0, 0.33, 0.66, 1.0]
    return LinearSegmentedColormap.from_list('blue_to_red',
list(zip(positions, colors)))

def apply_custom_heatmap(gaze_map):
    # преобразование карты значимости в цветное BGR
    if gaze_map.max() > gaze_map.min():
        gaze_map_norm = (gaze_map - gaze_map.min()) /
(gaze_map.max() - gaze_map.min() + 1e-8)
    else:
        gaze_map_norm = np.zeros_like(gaze_map)
    cmap = create_custom_colormap()
    colored = cmap(gaze_map_norm)[:, :, :3] # RGB 0..1
    return (colored[:, :, ::-1] * 255).astype(np.uint8) # BGR

# ===== МОДЕЛЬ КАРТЫ ВЗГЛЯДА =====
class GazeMapModel:
    def __init__(self):
        self.model_fn = None
        self.load_model()

    def _find_model_path(self):
        # поиск директории с saved_model.pb
        script_dir = Path(__file__).parent.absolute()
        models_dir = script_dir / MODELS_DIR
        model_hash_dir = models_dir / MODEL_HASH
        if model_hash_dir.exists():
            return str(model_hash_dir)
        if models_dir.exists():
            for item in models_dir.iterdir():
                if item.is_dir() and (item /
"saved_model.pb").exists():
                    return str(item)
            raise FileNotFoundError("Модель не найдена")

    def load_model(self):
        try:
            model_path = self._find_model_path()
            model = tf.saved_model.load(model_path)
            self.model_fn =
model.signatures[list(model.signatures.keys())[0]]
            return True
        except Exception:
            return False

    def predict_gaze_map(self, frame):
        # возвращает карту значимости и тепловую карту
        if self.model_fn is None:
            return None, None

```

```

        original_shape = frame.shape[:2]
        frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) if
frame.shape[2] == 3 else frame

        input_tensor =
tf.expand_dims(frame_rgb.astype(np.float32), axis=0)
        input_tensor = tf.image.resize(input_tensor, (320, 320),
preserve_aspect_ratio=True)

        h_pad = 320 - input_tensor.shape[1]
        w_pad = 320 - input_tensor.shape[2]
        input_tensor = tf.pad(input_tensor,
                                [[0, 0], [h_pad // 2, h_pad -
h_pad // 2],
                                [w_pad // 2, w_pad - w_pad // 2],
                                [0, 0]])

        outputs = self.model_fn(input_tensor)
        out = outputs[list(outputs.keys())[0]]

        # удаление паддинга
        out = out[:, h_pad // 2:out.shape[1] - (h_pad - h_pad //
2),
        w_pad // 2:out.shape[2] - (w_pad - w_pad // 2), :]
        out = tf.image.resize(out, original_shape)
        gaze_map = out.numpy().squeeze()
        heatmap = apply_custom_heatmap(gaze_map)
        return gaze_map, heatmap

# ===== ФУНКЦИИ РАБОТЫ С ОКНАМИ (WIN32) =====
def force_window_topmost(hwnd):
    # поднять окно поверх всех
    styles = ctypes.windll.user32.GetWindowLongW(hwnd, -20)
    if not (styles & 0x00000008):
        ctypes.windll.user32.SetWindowPos(hwnd, -1, 0, 0, 0, 0,
0x0001 | 0x0002)

    foreground = ctypes.windll.user32.GetForegroundWindow()
    if foreground != hwnd:
        curr_thread =
ctypes.windll.kernel32.GetCurrentThreadId()
        fg_thread =
ctypes.windll.user32.GetWindowThreadProcessId(foreground, None)
        if fg_thread != curr_thread:
            ctypes.windll.user32.AttachThreadInput(fg_thread,
curr_thread, True)
            ctypes.windll.user32.SetForegroundWindow(hwnd)
            if fg_thread != curr_thread:
                ctypes.windll.user32.AttachThreadInput(fg_thread,
curr_thread, False)

        ctypes.windll.user32.BringWindowToTop(hwnd)

```

```

        ctypes.windll.user32.SetWindowPos(hwnd, 0, 0, 0, 0, 0x0002 |
0x0001)

def set_window_click_through(hwnd):
    # пропускать клики сквозь окно
    styles = ctypes.windll.user32.GetWindowLongW(hwnd, -20)
    ctypes.windll.user32.SetWindowLongW(hwnd, -20, styles |
0x00000020 | 0x00080000)
    ctypes.windll.user32.SetWindowPos(hwnd, 0, 0, 0, 0, 0,
0x0002 | 0x0001)

def set_window_alpha(hwnd, alpha):
    # установить прозрачность
    ctypes.windll.user32.SetLayeredWindowAttributes(hwnd, 0,
alpha, 0x00000002)

# ===== ЗАХВАТ ЭКРАНА И СИМУЛЯЦИЯ =====
class ScreenCaptureMode:
    def __init__(self, hwnd):
        self.hwnd = hwnd
        self.reference_distribution = None # эталонное
распределение
        self.dynamic_distribution = None # динамическое
распределение
        self.agents = None # позиции агентов
        self.sequences = None # последовательности
        self.running = True # флаг работы
        self.paused = False # флаг паузы
        self.current_iteration = 0 # текущая итерация
        self.alpha = INITIAL_ALPHA # прозрачность
        self.current_display = 1 # режим отображения
        self.minimized = False # окно свёрнуто
        self.capture_interval = SCREEN_CAPTURE_INTERVAL #
интервал захвата

        self.alpha_lock = threading.Lock()
        self.screen_capture_lock = threading.Lock()
        self.capturing_in_progress = False

        self.command_queue = deque()
        self.command_lock = threading.Lock()

        self.model = GazeMapModel()
        self.predicted_heatmap = None
        self.predict_enabled = PREDICT_ENABLED
        self.model_loaded = self.model.model_fn is not None

        self.frame_queue = queue.Queue(maxsize=30)
        self.processing_thread =
threading.Thread(target=self._process_frames_worker,

```

```

daemon=True)
    self.processing_thread.start()

    self._initialize_simulation()
    self.screen_capture_thread =
threading.Thread(target=self._continuous_screen_capture,
daemon=True)
    self.screen_capture_thread.start()

    self.topmost_thread =
threading.Thread(target=self._topmost_loop, daemon=True)
    self.topmost_thread.start()

    self.command_thread =
threading.Thread(target=self._process_commands, daemon=True)
    self.command_thread.start()

def _topmost_loop(self):
    while self.running:
        if not self.minimized:
            force_window_topmost(self.hwnd)
            time.sleep(TOPMOST_FORCE_INTERVAL)

def _process_commands(self):
    # обработка команд с приоритетом последней
    while self.running:
        if self.command_queue:
            with self.command_lock:
                while len(self.command_queue) > 1:
                    self.command_queue.popleft()
                    command = self.command_queue[0] if
self.command_queue else None

                if command:
                    cmd_type, value = command
                    if cmd_type == 'display':
                        self.current_display = value
                        force_window_topmost(self.hwnd)
                    elif cmd_type == 'alpha':
                        self.alpha = max(0, min(255, value))
                        self._set_window_alpha_safe(self.alpha)
                        force_window_topmost(self.hwnd)
                    elif cmd_type == 'capture_interval':
                        with self.screen_capture_lock:
                            self.capture_interval =
max(MIN_CAPTURE_INTERVAL, min(MAX_CAPTURE_INTERVAL, value))
                    elif cmd_type == 'predict':
                        self.predict_enabled = value
                        if not self.predict_enabled:
                            self.predicted_heatmap = None
                        elif not self.frame_queue.empty():

self._generate_gaze_heatmap(self.frame_queue.queue[-1])

```

```

        elif cmd_type == 'pause':
            self.paused = value
        elif cmd_type == 'minimize':
            self._toggle_minimize_internal()

        if self.command_queue:
            with self.command_lock:
                if self.command_queue and
self.command_queue[0] == command:
                    self.command_queue.popleft()
            time.sleep(0.01)

    def add_command(self, cmd_type, value):
        with self.command_lock:
            self.command_queue.append((cmd_type, value))

    def get_overlay(self, screen_shape):
        # формирование наложения
        if self.reference_distribution is None or
self.dynamic_distribution is None:
            return np.zeros((screen_shape[1], screen_shape[0],
4), dtype=np.uint8)

        screen_w, screen_h = screen_shape

        if self.current_display == 1:
            if self.predicted_heatmap is not None and
self.predict_enabled:
                img = self.predicted_heatmap
            else:
                img = np.ones((screen_h, screen_w, 3),
dtype=np.uint8) * 255
                cv2.putText(img, "Карта взгляда ВЫКЛ", (screen_w
// 4, screen_h // 2),
                           cv2.FONT_HERSHEY_SIMPLEX, 1.5, (0,
0, 0), 3)
        elif self.current_display == 2:
            dyn_norm = np.clip(self.dynamic_distribution, 0,
255).astype(np.uint8)
            img = apply_custom_heatmap(dyn_norm / 255.0)
        else: # 3 - эталон
            ref_norm = np.clip(self.reference_distribution, 0,
255).astype(np.uint8)
            img = apply_custom_heatmap(ref_norm / 255.0)

        if img.shape[:2] != (screen_h, screen_w):
            img = cv2.resize(img, (screen_w, screen_h))

        overlay = np.zeros((screen_h, screen_w, 4),
dtype=np.uint8)
        overlay[:, :, :3] = img
        overlay[:, :, 3] = 255
        return overlay

```

```

def _set_window_alpha_safe(self, alpha):
    with self.alpha_lock:
        if not self.capturing_in_progress:
            set_window_alpha(self.hwnd, alpha)

def _capture_screen_without_overlay(self):
    try:
        with self.alpha_lock:
            self.capturing_in_progress = True

        current_alpha = self.alpha
        set_window_alpha(self.hwnd, 0)
        time.sleep(0.01)

        with mss.mss() as sct:
            monitor = sct.monitors[1]
            screenshot = sct.grab(monitor)
            img = np.array(screenshot)
            result = cv2.cvtColor(img, cv2.COLOR_BGRA2RGB)

        set_window_alpha(self.hwnd, current_alpha)
        return result
    except Exception:
        return None
    finally:
        with self.alpha_lock:
            self.capturing_in_progress = False

def _update_reference_in_cpp(self, frame):
    try:
        agent_sim.update_reference(frame.astype(np.uint8),
CELL_SIZE)
        rows, cols = agent_sim.get_reference_dims()
        if rows > 0 and cols > 0:
            new_ref = np.zeros((rows, cols),
dtype=np.float32)
            agent_sim.get_reference(new_ref)
            self.reference_distribution = new_ref
    except Exception:
        pass

def _generate_gaze_heatmap(self, frame):
    if not self.predict_enabled or not self.model_loaded:
        self.predicted_heatmap = None
        return
    _, heatmap = self.model.predict_gaze_map(frame)
    self.predicted_heatmap = heatmap

def _capture_and_queue_frame(self):
    frame = self._capture_screen_without_overlay()
    if frame is not None:
        try:

```

```

        self.frame_queue.put(frame, timeout=0.5)
    except queue.Full:
        try:
            self.frame_queue.get_nowait()
        except queue.Empty:
            pass
        self.frame_queue.put(frame)

def _process_frames_worker(self):
    while self.running:
        try:
            frame = self.frame_queue.get(timeout=0.1)
        except queue.Empty:
            continue
        self._update_reference_in_cpp(frame)
        self._generate_gaze_heatmap(frame)
        self.frame_queue.task_done()

def _continuous_screen_capture(self):
    while self.running:
        with self.screen_capture_lock:
            interval = self.capture_interval
            time.sleep(interval)
            self._capture_and_queue_frame()

def _initialize_simulation(self):
    self._capture_and_queue_frame()
    time.sleep(0.5)
    rows, cols = 0, 0
    for _ in range(10):
        rows, cols = agent_sim.get_reference_dims()
        if rows > 0 and cols > 0:
            break
    time.sleep(0.5)
    self._capture_and_queue_frame()
    if rows == 0 or cols == 0:
        raise RuntimeError("Не удалось получить размеры")
    self.reference_distribution = np.zeros((rows, cols),
dtype=np.float32)
    self.dynamic_distribution = np.zeros((rows, cols),
dtype=np.float32)
    self.agents = np.array([(np.random.randint(rows),
np.random.randint(cols)) for _ in range(NUM_AGENTS)],
dtype=np.int32)
    self.sequences = [np.random.permutation(8) for _ in
range(1000)]

def update_simulation(self):
    if not self.running or self.paused or
self.current_iteration >= NUM_ITERATIONS:
        return False
    try:
        positions = self.agents.astype(np.int32)

```

```

        dyn_float =
self.dynamic_distribution.astype(np.float32)
        seq = self.sequences[self.current_iteration %
len(self.sequences)].astype(np.int32)
        rows, cols = self.dynamic_distribution.shape
        out_dyn = np.zeros_like(dyn_float)
        out_ref = np.zeros((rows, cols), dtype=np.float32)
        agent_sim.multi_step_agents(
            positions, dyn_float, seq,
            ALLOW_MULTIPLE_AGENTS_IN_CELL,
            STEPS_PER_FRAME, DYNAMIC_STEP,
            out_dyn, out_ref
        )
        self.agents = positions
        self.dynamic_distribution = out_dyn
        self.reference_distribution = out_ref
        self.current_iteration += STEPS_PER_FRAME
        return True
    except Exception:
        self.running = False
        return False

# интерфейсные методы
def toggle_predict(self):
    self.add_command('predict', not self.predict_enabled)
    return self.predict_enabled

def increase_capture_interval(self):
    self.add_command('capture_interval',
self.capture_interval + CAPTURE_INTERVAL_STEP)

def decrease_capture_interval(self):
    self.add_command('capture_interval',
self.capture_interval - CAPTURE_INTERVAL_STEP)

def set_display_mode(self, mode):
    self.add_command('display', mode)

def set_alpha(self, alpha):
    self.add_command('alpha', alpha)

def set_pause(self, paused):
    self.add_command('pause', paused)

def _toggle_minimize_internal(self):
    if not self.minimized:
        ctypes.windll.user32.ShowWindow(self.hwnd, 6) #
свернуть
        self.minimized = True
    else:
        ctypes.windll.user32.ShowWindow(self.hwnd, 9) #
восстановить
        self.minimized = False

```



```

        force_window_topmost(self.hwnd)
        set_window_click_through(self.hwnd)
        self._set_window_alpha_safe(self.alpha)

    def toggle_minimize(self):
        self.add_command('minimize', None)

# ===== ГРАФИЧЕСКИЙ ИНТЕРФЕЙС =====
class ModernButton:
    def __init__(self, x, y, w, h, text, color, hover_color,
text_color=(0, 0, 0)):
        self.rect = pygame.Rect(x, y, w, h)
        self.text = text
        self.color = color
        self.hover_color = hover_color
        self.text_color = text_color
        self.hover = False
        self.enabled = True

    def draw(self, screen, font):
        color = (180, 180, 180) if not self.enabled else
(self.hover_color if self.hover else self.color)
        pygame.draw.rect(screen, color, self.rect,
border_radius=12)
        pygame.draw.rect(screen, (150, 150, 170), self.rect, 3,
border_radius=12)
        surf = font.render(self.text, True, self.text_color)
        rect = surf.get_rect(center=self.rect.center)
        screen.blit(surf, rect)

    def handle_event(self, event):
        if not self.enabled:
            return False
        if event.type == pygame.MOUSEMOTION:
            self.hover = self.rect.collidepoint(event.pos)
        elif event.type == pygame.MOUSEBUTTONDOWN and
event.button == 1:
            return self.rect.collidepoint(event.pos)
        return False

class TextInput:
    def __init__(self, x, y, w, h, font):
        self.rect = pygame.Rect(x, y, w, h)
        self.text = "" # введённый текст
        self.cursor_pos = 0 # позиция курсора
        self.scroll_offset = 0 # смещение прокрутки
        self.active = False # активность поля
        self.color = (230, 230, 250)
        self.active_color = (200, 220, 255)
        self.cursor_blink = 0
        self.cursor_visible = True

```

```

self.backspace_timer = 0
self.backspace_delay = 500
self.backspace_interval = 50
self.font = font

def draw(self, screen):
    color = self.active_color if self.active else self.color
    pygame.draw.rect(screen, color, self.rect,
border_radius=10)
    pygame.draw.rect(screen, (150, 150, 170), self.rect, 3,
border_radius=10)

    display_text = self.text if self.text else "Введите путь
к видео на компьютере..."
    text_color = (80, 80, 80) if not self.text else (0, 0,
0)

    if self.active and self.cursor_pos < len(self.text):
        cx = self.font.size(self.text[:self.cursor_pos])[0]
        iw = self.rect.width - 20
        if cx - self.scroll_offset > iw:
            self.scroll_offset = cx - iw
        elif cx - self.scroll_offset < 0:
            self.scroll_offset = cx

    if self.scroll_offset > 0 and self.text:
        start = 0
        for i in range(len(self.text)):
            if self.font.size(self.text[:i])[0] >=
self.scroll_offset:
                start = i
                break
        visible = self.text[start:]
    else:
        visible = self.text
        self.scroll_offset = 0

    surf = self.font.render(visible, True, text_color)
    ty = self.rect.y + self.rect.height // 2 -
surf.get_height() // 2
    screen.blit(surf, (self.rect.x + 15, ty))

    if self.active and self.cursor_visible:
        cx = self.rect.x + 15 +
self.font.size(self.text[:self.cursor_pos])[0] -
self.scroll_offset
        if self.rect.x + 10 <= cx <= self.rect.x +
self.rect.width - 10:
            pygame.draw.line(screen, (0, 0, 0), (cx,
self.rect.y + 12),
                                (cx, self.rect.y +
self.rect.height - 12), 3)

```

```

def handle_event(self, event):
    if event.type == pygame.MOUSEBUTTONDOWN:
        self.active = self.rect.collidepoint(event.pos)
        if self.active:
            self.cursor_blink = 0
            self.cursor_visible = True
            mx = event.pos[0] - self.rect.x - 15 +
self.scroll_offset
            for i in range(len(self.text) + 1):
                if self.font.size(self.text[:i])[0] >= mx:
                    self.cursor_pos = i
                    break
            return False

    if event.type == pygame.KEYDOWN and self.active:
        self.cursor_blink = 0
        self.cursor_visible = True

        if event.key == pygame.K_RETURN:
            self.active = False
            return True
        if event.key == pygame.K_BACKSPACE:
            if self.cursor_pos > 0:
                self.text = self.text[:self.cursor_pos - 1]
+ self.text[self.cursor_pos:]
                self.cursor_pos -= 1
                self.backspace_timer =
pygame.time.get_ticks()
            elif event.key == pygame.K_DELETE:
                if self.cursor_pos < len(self.text):
                    self.text = self.text[:self.cursor_pos] +
self.text[self.cursor_pos + 1:]
                elif event.key == pygame.K_LEFT:
                    self.cursor_pos = max(0, self.cursor_pos - 1)
                elif event.key == pygame.K_RIGHT:
                    self.cursor_pos = min(len(self.text),
self.cursor_pos + 1)
                elif event.key == pygame.K_HOME:
                    self.cursor_pos = 0
                    self.scroll_offset = 0
                elif event.key == pygame.K_END:
                    self.cursor_pos = len(self.text)
                elif event.key == pygame.K_v and
pygame.key.get_mods() & pygame.KMOD_CTRL:
                    try:
                        import pyperclip
                        cb = pyperclip.paste()
                        if cb:
                            self.text = self.text[:self.cursor_pos]
+ cb + self.text[self.cursor_pos:]
                            self.cursor_pos += len(cb)
                    except ImportError:
                        pass

```

```

        elif event.key == pygame.K_c and
pygame.key.get_mods() & pygame.KMOD_CTRL:
            try:
                import pyperclip
                pyperclip.copy(self.text)
            except ImportError:
                pass
        elif event.unicode and event.unicode.isprintable():
            self.text = self.text[:self.cursor_pos] +
event.unicode + self.text[self.cursor_pos:]
            self.cursor_pos += 1
        return False

    if event.type == pygame.KEYUP and self.active and
event.key == pygame.K_BACKSPACE:
        self.backspace_timer = 0
        return False

    def update(self):
        if self.active:
            self.cursor_blink += 1
            if self.cursor_blink >= 30:
                self.cursor_blink = 0
                self.cursor_visible = not self.cursor_visible

            if self.backspace_timer > 0:
                now = pygame.time.get_ticks()
                if now - self.backspace_timer >
self.backspace_delay:
                    if now - self.backspace_timer >
self.backspace_delay + self.backspace_interval:
                        self.backspace_timer = now -
self.backspace_delay
                    if self.cursor_pos > 0:
                        self.text =
self.text[:self.cursor_pos - 1] + self.text[self.cursor_pos:]
                        self.cursor_pos -= 1

class ProgressBar:
    def __init__(self, x, y, w, h, font):
        self.rect = pygame.Rect(x, y, w, h)
        self.progress = 0.0
        self.font = font

    def draw(self, screen):
        pygame.draw.rect(screen, (200, 200, 210), self.rect,
border_radius=10)
        if self.progress > 0:
            pw = int(self.rect.width * self.progress)
            pr = pygame.Rect(self.rect.x, self.rect.y, pw,
self.rect.height)
            pygame.draw.rect(screen, (80, 150, 200), pr,

```

```

border_radius=10)
    text = self.font.render(f"{int(self.progress * 100)}%",
True, (0, 0, 0))
    tr = text.get_rect(center=self.rect.center)
    screen.blit(text, tr)

    def set_progress(self, value):
        self.progress = max(0.0, min(1.0, value))

class VideoProcessor:
    def __init__(self, video_path, output_path, status_cb,
progress_cb, complete_cb):
        self.video_path = video_path
        self.output_path = output_path
        self.status_cb = status_cb
        self.progress_cb = progress_cb
        self.complete_cb = complete_cb
        self.running = True
        self.thread = None

    def stop(self):
        self.running = False
        if self.thread and self.thread.is_alive():
            self.thread.join(timeout=1.0)

    def process(self):
        self.thread = threading.Thread(target=self._process,
daemon=True)
        self.thread.start()

    def _process(self):
        try:
            self.status_cb("Загрузка модели...")
            model = GazeMapModel()

            self.status_cb("Открытие видео...")
            cap = cv2.VideoCapture(self.video_path)
            if not cap.isOpened():
                self.status_cb("ОШИБКА: Не удалось открыть
видео")

                self.complete_cb(False, None)
                return

            fps = int(cap.get(cv2.CAP_PROP_FPS))
            w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
            h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
            total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

            os.makedirs(OUTPUT_DIR, exist_ok=True)
            fourcc = cv2.VideoWriter_fourcc(*'mp4v')
            out = cv2.VideoWriter(self.output_path, fourcc, fps,
(w, h))

```

```

        cnt = 0
        self.status_cb("Обработка видео...")
        while self.running and cnt < total:
            ret, frame = cap.read()
            if not ret:
                break
            _, hm = model.predict_gaze_map(frame)
            blended = cv2.addWeighted(frame, 0.6, hm, 0.4,
0) if hm is not None else frame
            out.write(blended)
            cnt += 1
            self.progress_cb(cnt / total)

        cap.release()
        out.release()

        if self.running and cnt == total:
            self.complete_cb(True, self.output_path)
        else:
            self.complete_cb(False, None)
    except Exception:
        self.complete_cb(False, None)

class MenuGUI:
    def __init__(self):
        pygame.init()
        pygame.font.init()
        self.WIDTH = int(1200 * 1.5)    # ширина окна
        self.HEIGHT = int(800 * 1.5)   # высота окна
        self.screen = pygame.display.set_mode((self.WIDTH,
self.HEIGHT))
        pygame.display.set_caption("Карта Взгляда - Система
Визуального Внимания")

        if sys.platform == 'win32':
            hwnd = pygame.display.get_wm_info()[0]["window"]
            force_window_topmost(hwnd)

        self.BUTTON_COLOR = (200, 200, 220)
        self.BUTTON_HOVER = (160, 160, 190)
        self.TEXT_COLOR = (0, 0, 0)
        self.ACCENT_COLOR = (80, 150, 200)
        self.SUCCESS_COLOR = (60, 180, 60)

        self.title_font = pygame.font.SysFont('arial', int(48 *
1.5))
        self.button_font = pygame.font.SysFont('arial', int(32 *
1.5))
        self.small_font = pygame.font.SysFont('arial', int(24 *
1.5))
        self.status_font = pygame.font.SysFont('arial', int(22 *

```

1.5))

```
self.current_screen = "main" # текущий экран
self.processing = False
self.processor = None
self.status_text = ""
self.status_color = self.TEXT_COLOR
self.progress_bar = None
self.completed = False
self.completed_path = ""

self.create_main_buttons()
self.create_video_buttons()
self.clock = pygame.time.Clock()
self.running = True

def create_main_buttons(self):
    bw = int(350 * 1.5)
    bh = int(80 * 1.5)
    bx = (self.WIDTH - bw) // 2
    self.main_buttons = [
        ModernButton(bx, int(250 * 1.5), bw, bh, "Захват
экрана",
                                self.BUTTON_COLOR, self.BUTTON_HOVER,
self.TEXT_COLOR),
        ModernButton(bx, int(360 * 1.5), bw, bh, "Обработка
видео",
                                self.BUTTON_COLOR, self.BUTTON_HOVER,
self.TEXT_COLOR),
        ModernButton(bx, int(470 * 1.5), bw, bh, "Выход",
                                self.BUTTON_COLOR, self.BUTTON_HOVER,
self.TEXT_COLOR)
    ]

def create_video_buttons(self):
    bw = int(220 * 1.5)
    bh = int(65 * 1.5)
    self.video_input = TextInput(int(200 * 1.5), int(250 *
1.5), int(800 * 1.5), int(65 * 1.5), self.small_font)
    self.back_btn = ModernButton(int(200 * 1.5), int(650 *
1.5), bw, bh, "Назад",
                                self.BUTTON_COLOR,
self.BUTTON_HOVER, self.TEXT_COLOR)
    self.process_btn = ModernButton(self.WIDTH - int(200 *
1.5) - bw, int(650 * 1.5), bw, bh, "Обработать",
                                self.ACCENT_COLOR, (100,
170, 220), self.TEXT_COLOR)
    self.stop_btn = ModernButton(self.WIDTH - int(200 * 1.5)
- bw, int(650 * 1.5), bw, bh, "Остановить",
                                (200, 100, 100), (220, 120,
120), self.TEXT_COLOR)
    self.stop_btn.enabled = False
    self.back_after_complete_btn = ModernButton(self.WIDTH
```

```

// 2 - int(120 * 1.5), int(550 * 1.5), int(240 * 1.5),
                                                    int(65 *
1.5),
                                                    "Назад",
self.ACCENT_COLOR, (100, 170, 220), self.TEXT_COLOR)

    def draw_gradient(self):
        for i in range(self.HEIGHT):
            val = 240 + int(i / self.HEIGHT * 15)
            pygame.draw.line(self.screen, (val, val, 250), (0,
i), (self.WIDTH, i))

    def draw_main_screen(self):
        self.draw_gradient()
        title = self.title_font.render("Карта Взгляда", True,
self.ACCENT_COLOR)
        tr = title.get_rect(center=(self.WIDTH // 2, int(120 *
1.5)))
        self.screen.blit(title, tr)
        sub = self.small_font.render("Система анализа
визуального внимания", True, self.TEXT_COLOR)
        sr = sub.get_rect(center=(self.WIDTH // 2, int(180 *
1.5)))
        self.screen.blit(sub, sr)
        for btn in self.main_buttons:
            btn.draw(self.screen, self.button_font)

    def draw_video_screen(self):
        self.draw_gradient()
        title = self.title_font.render("Обработка видео", True,
self.ACCENT_COLOR)
        tr = title.get_rect(center=(self.WIDTH // 2, int(80 *
1.5)))
        self.screen.blit(title, tr)

        if self.completed:
            comp = self.title_font.render("Видео обработано!",
True, self.SUCCESS_COLOR)
            cr = comp.get_rect(center=(self.WIDTH // 2, int(280
* 1.5)))
            self.screen.blit(comp, cr)
            if self.completed_path:
                path_txt = self.small_font.render(f"Сохранено:
{os.path.basename(self.completed_path)}", True,
self.TEXT_COLOR)
                pr = path_txt.get_rect(center=(self.WIDTH // 2,
int(360 * 1.5)))
                self.screen.blit(path_txt, pr)
                self.back_after_complete_btn.draw(self.screen,
self.button_font)
            elif self.processing:
                status = self.status_font.render(self.status_text,

```



```

True, self.status_color)
    sr = status.get_rect(center=(self.WIDTH // 2,
int(200 * 1.5)))
    self.screen.blit(status, sr)
    if self.progress_bar:
        self.progress_bar.draw(self.screen)
        self.stop_btn.draw(self.screen, self.button_font)
    else:
        label = self.small_font.render("Путь к видео на
компьютере:", True, self.TEXT_COLOR)
        lr = label.get_rect(center=(self.WIDTH // 2, int(190
* 1.5)))
        self.screen.blit(label, lr)
        self.video_input.draw(self.screen)
        if self.status_text:
            st = self.small_font.render(self.status_text,
True, self.status_color)
            sr = st.get_rect(center=(self.WIDTH // 2,
int(400 * 1.5)))
            self.screen.blit(st, sr)
            self.back_btn.draw(self.screen, self.button_font)
            self.process_btn.draw(self.screen, self.button_font)

    @staticmethod
    def get_video_path(inp):
        return inp if os.path.exists(inp) else None

    def process_video(self, path):
        self.processing = True
        self.completed = False
        self.status_text = "Загрузка модели..."
        self.status_color = self.ACCENT_COLOR

        out_name = f"processed_{Path(path).stem}.mp4"
        out_path = os.path.join(OUTPUT_DIR, out_name)
        self.progress_bar = ProgressBar(int(250 * 1.5), int(300
* 1.5), int(700 * 1.5), int(40 * 1.5), self.button_font)

        def upd_status(txt):
            self.status_text = txt
            self.status_color = self.ACCENT_COLOR

        def upd_prog(v):
            if self.progress_bar:
                self.progress_bar.set_progress(v)

        def on_complete(success, fpath):
            self.processing = False
            if success:
                self.completed = True
                self.completed_path = fpath
            else:
                self.status_text = "Ошибка обработки видео"

```

```

        self.status_color = (200, 50, 50)

        self.processor = VideoProcessor(path, out_path,
        upd_status, upd_prog, on_complete)
        self.processor.process()

    def run(self):
        while self.running:
            mp = pygame.mouse.get_pos()
            if self.current_screen == "main":
                for btn in self.main_buttons:
                    btn.hover = btn.rect.collidepoint(mp)
            elif self.current_screen == "video":
                if not self.processing and not self.completed:
                    self.back_btn.hover =
self.back_btn.rect.collidepoint(mp)
                    self.process_btn.hover =
self.process_btn.rect.collidepoint(mp)
                elif self.processing:
                    self.stop_btn.hover =
self.stop_btn.rect.collidepoint(mp)
                elif self.completed:
                    self.back_after_complete_btn.hover =
self.back_after_complete_btn.rect.collidepoint(mp)

            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    self.running = False
                    return "exit"

            if self.current_screen == "main":
                for i, btn in enumerate(self.main_buttons):
                    if btn.handle_event(event):
                        if i == 0:
                            return "screen"
                        elif i == 1:
                            self.current_screen = "video"
                            self.status_text = ""
                            self.processing = False
                            self.completed = False
                            self.video_input.text = ""
                            self.video_input.cursor_pos = 0
                            self.video_input.scroll_offset =
0

                        elif i == 2:
                            return "exit"
            elif self.current_screen == "video":
                if self.completed:
                    if
self.back_after_complete_btn.handle_event(event):
                        self.current_screen = "video"
                        self.completed = False
                        self.status_text = ""

```

```

        self.processing = False
        self.video_input.text = ""
        self.video_input.cursor_pos = 0
        self.video_input.scroll_offset = 0
    elif self.processing:
        if self.stop_btn.handle_event(event) and
self.processor:
            self.processor.stop()
            self.processing = False
            self.status_text = "Обработка
остановлена"

            self.status_color = (200, 130, 0)
            self.stop_btn.enabled = False
            self.process_btn.enabled = True
        else:
            self.video_input.handle_event(event)
            if self.back_btn.handle_event(event):
                self.current_screen = "main"
                self.status_text = ""
                self.completed = False
            if self.process_btn.handle_event(event):
                if not self.video_input.text:
                    self.status_text = "Пожалуйста,
введите путь к видео"

                    self.status_color = (200, 50,
50)

                else:
                    vpath =
self.get_video_path(self.video_input.text)
                    if vpath:
                        self.process_video(vpath)
                        self.stop_btn.enabled = True
                        self.process_btn.enabled =
False

                    else:
                        self.status_text = "Видео не
найдено! Проверьте путь"

                        self.status_color = (200,
50, 50)

            if self.current_screen == "main":
                self.draw_main_screen()
            else:
                if not self.processing and not self.completed:
                    self.video_input.update()
                    self.draw_video_screen()
                pygame.display.flip()
                self.clock.tick(60)
            return "exit"

# ===== РЕЖИМ ЗАХВАТА ЭКРАНА =====
def run_screen_capture():

```

```

pygame.init()
pygame.font.init()
info = pygame.display.Info()
screen = pygame.display.set_mode((info.current_w,
info.current_h), pygame.NOFRAME)
pygame.display.set_caption("Карта Взгляда - Наложение")

if sys.platform == 'win32':
    hwnd = pygame.display.get_wm_info()[ "window" ]
    force_window_topmost(hwnd)
    set_window_click_through(hwnd)
    set_window_alpha(hwnd, INITIAL_ALPHA)

clock = pygame.time.Clock()
sim = ScreenCaptureMode(hwnd)

def on_key_press(e):
    try:
        if e.name == '1' and keyboard.is_pressed('ctrl'):
            sim.set_display_mode(1)
        elif e.name == '2' and keyboard.is_pressed('ctrl'):
            sim.set_display_mode(2)
        elif e.name == '3' and keyboard.is_pressed('ctrl'):
            sim.set_display_mode(3)
        elif (e.name == '+' or e.name == '=') and
keyboard.is_pressed('ctrl'):
            sim.set_alpha(min(255, sim.alpha + 10))
        elif e.name == '-' and keyboard.is_pressed('ctrl'):
            sim.set_alpha(max(0, sim.alpha - 10))
        elif e.name == '[' and keyboard.is_pressed('ctrl'):
            sim.decrease_capture_interval()
        elif e.name == ']' and keyboard.is_pressed('ctrl'):
            sim.increase_capture_interval()
        elif e.name == 'space':
            sim.set_pause(not sim.paused)
        elif e.name == 'h' and keyboard.is_pressed('ctrl'):
            sim.toggle_predict()
        elif e.name == 'enter' and
keyboard.is_pressed('ctrl'):
            sim.toggle_minimize()
        elif e.name == 'esc' or e.name == 'q':
            sim.running = False
    except Exception:
        pass

keyboard.on_press(on_key_press)

while sim.running:
    for ev in pygame.event.get():
        if ev.type == pygame.QUIT:
            sim.running = False

    if not sim.paused:

```

```

sim.update_simulation()

if not sim.minimized:
    ov = sim.get_overlay((info.current_w,
info.current_h))
    if ov is not None:
        surf = pygame.image.frombuffer(ov.tobytes(),
ov.shape[1::-1], "RGBA")
        screen.fill((255, 255, 255, 0))
        screen.blit(surf, (0, 0))

        font = pygame.font.SysFont('arial', 24)
        mode_names = {1: "Предсказанная карта взгляда",
2: "Динамическое распределение",
3: "Эталонное распределение"}
        mode = mode_names.get(sim.current_display, "")
        model_stat = "Карта взгляда ВКЛ" if
(sim.predict_enabled and sim.model_loaded) else "Карта взгляда
ВЫКЛ"

        txt = f"Режим: {mode} | Прозрачность:
{sim.alpha} | Интервал: {sim.capture_interval:.2f}с |
{model_stat}"
        text = font.render(txt, True, (0, 0, 0))
        screen.blit(text, (10, 10))

        hint_font = pygame.font.SysFont('arial', 18)
        hint = hint_font.render(
            "Ctrl+1/2/3 - смена | Ctrl+/- - прозрачность
| Ctrl+[/] - частота | Ctrl+H - карта взгляда | Пробел - пауза",
            True, (50, 50, 50))
        screen.blit(hint, (10, 45))

        force_window_topmost(hwnd)
        pygame.display.flip()

    clock.tick(60)

keyboard.unhook_all()
pygame.quit()

# ===== ГЛАВНАЯ ФУНКЦИЯ =====
def main():
    try:
        import pyperclip
    except ImportError:
        subprocess.check_call([sys.executable, "-m", "pip",
"install", "pyperclip", "-q"])
        import pyperclip

    while True:
        menu = MenuGUI()
        choice = menu.run()

```

```
pygame.quit()
if choice == "screen":
    run_screen_capture()
elif choice == "exit":
    break

if __name__ == "__main__":
    main()
```